Course home

Transformers & LLMs

> Nineteen sections, from what a Transformer is to how LLMs are trained, served and fine-tuned. Each section has lecture notes; most also have an exercise sheet with worked solutions and a runnable Python lab.

Read the sections in order: each one builds on the notation and examples of the ones before it. Labs run with `uv run python <section>/lab.py` from the project folder.

Foundations

1 What a Transformer Is

The end-to-end pipeline from tokens to output, the two sub-layers in every block, and why Transformers replaced RNNs.

notes.md exercises solutions lab.py

2 Tokens & Embeddings

Cutting text into sub-word tokens with BPE, turning tokens into IDs, and IDs into learned embedding vectors.

notes.md exercises solutions lab.py

3 Self-Attention: the Intuition

Each token becomes a weighted average of all tokens: score, softmax, average. How "bat" gets its meaning from context.

notes.md exercises solutions lab.py

4 Queries, Keys & Values

Three learned projections that let a token ask, be found, and hand over information separately.

notes.md exercises solutions lab.py

5 Scaled Dot-Product Attention

The full formula softmax(QKᵀ/√d_k + M)V: why we scale by √d_k, and how causal and padding masks work.

notes.md exercises solutions lab.py

6 Multi-Head Attention

Several smaller attention heads side by side, concatenated and mixed by W_O. Plus GQA and MQA.

notes.md exercises solutions lab.py

7 Positional Information

Why attention is order-blind, and the fixes: learned and sinusoidal position vectors, RoPE and ALiBi.

notes.md exercises solutions lab.py

8 Residuals, LayerNorm & the FFN

The parts around attention: residual connections, LayerNorm and RMSNorm, and the per-token feed-forward network.

notes.md exercises solutions lab.py

Architectures

9 The Encoder Stack

Bidirectional blocks that produce contextual vectors, masked-LM pre-training, and how BERT-style outputs are used.

notes.md exercises solutions lab.py

10 The Decoder Stack

Causal self-attention, cross-attention, teacher forcing for parallel training, and token-by-token generation.

notes.md exercises solutions lab.py

11 Encoder vs Decoder vs Both

Same block, different mask and objective. Why decoder-only models became the default for LLMs.

notes.md exercises solutions lab.py

Large language models

12 Training an LLM

Pre-training, SFT and preference tuning; cross-entropy and perplexity, AdamW, learning-rate schedules and compute budgets.

notes.md exercises solutions lab.py

13 Inference & Decoding

Prefill and decode, greedy and beam search, and sampling with temperature, top-k, top-p and min-p.

notes.md exercises solutions lab.py

14 KV Cache & Fast Attention

Caching keys and values during generation, sizing the cache, and FlashAttention's tiled online softmax.

notes.md exercises solutions lab.py

Beyond text, and in practice

15 Vision Transformers (ViT)

Images as sequences of patches fed to a standard encoder; plus DeiT, Swin, MAE, CLIP and vision-language models.

notes.md exercises solutions lab.py

16 Fine-Tuning & LoRA

When to fine-tune, and parameter-efficient methods, above all LoRA's low-rank update $\Delta W = BA$ and QLoRA.

notes.md exercises solutions lab.py

17 Optimization & Quantization

Number formats, INT8/INT4 quantization, mixed precision, ZeRO sharding, model parallelism and speculative decoding.

notes.md exercises solutions lab.py

18 Serving & RAG

Running an LLM for many users (latency, batching, cost) and retrieval-augmented generation over your own documents.

notes.md exercises solutions lab.py

19 Interview Revision

The capstone: a formula sheet, numbers to remember, a question bank with model answers, and system-design practice.

notes.md exercises solutions lab.py

Also in this folder

Transformer_Notes_Akhil.pdf (printable notes) · COURSE.md (course log: conventions and the examples used in each section)

Akhil

What a Transformer Is

> **In one line:** a Transformer is a neural network that turns a sequence of tokens into a sequence of context-aware vectors by letting every token look directly at every other token, all at the same time.

1. Learning objectives
2. Prerequisites
3. Motivation
4. Intuition first
5. The formal idea
6. Worked example
7. Diagram
8. In practice
9. Common mistakes
10. Summary
11. Check your understanding
12. Further reading

1. Learning objectives

By the end of this section you should be able to:

1. **Explain** what problem the Transformer was designed to solve and why recurrent networks (RNNs) struggled with it.
2. **Describe** the end-to-end pipeline of a Transformer: tokens → embeddings → stacked blocks → output.
3. **Name** the two sub-layers inside every Transformer block and say, in one sentence each, what they do.
4. **Trace** the shape of the data as it flows through the model for a small example.
5. **Compare** the Transformer with an RNN on parallelism, path length between words, and cost.

2. Prerequisites

You should be comfortable with:

- **Python and NumPy**: arrays and `.shape`.
- **Matrix multiplication**: if A is $m \times k$ and B is $k \times p$, then AB is $m \times p$.
- **What a neural network layer does**: multiply by a weight matrix, add a bias, apply a non-linearity.
- **Gradient descent**: weights are adjusted step by step to reduce a loss.

> ***Quick refresher: a "vector representation"***
> Neural networks can't read words. Each word (or piece of a word) is replaced by a list of numbers called a **vector**, e.g. `[0.2, -1.3, 0.7, 0.05]`. Similar words end up with similar vectors. Everything in this course is about transforming such vectors into more useful ones.

No earlier sections are needed; this is the starting point.

3. Motivation: why do we need something new?

Read this sentence:

> *Riya poured water into the glass until **it** was full.*

What does **it** refer to? You instantly know it's *the glass*, not *Riya* and not *the water*. To work that out you had to connect "it" with a word five positions earlier, and use "full" (which comes *after* "it") as a clue.

Language is full of these long-range connections. A model that understands text must be able to:

1. **Link words that are far apart.**

2. **Use context from both sides** of a word (or at least everything before it).
3. **Do this quickly** on very long texts, because modern models train on trillions of words.

How RNNs handled it, and where they hit a wall

Before 2017 the standard tool was the **recurrent neural network (RNN)**, usually in its LSTM or GRU form. An RNN reads one token at a time and keeps a running summary called the **hidden state**:

$$h_t = f(h_{t-1},\ x_t)$$

In words: the new summary h_t is computed from the previous summary h_{t-1} and the current token x_t.

This design has three problems:

Problem	Why it happens
Slow training	Step t can't start until step $t-1$ is finished, so a 1,000-token sentence needs 1,000 steps one after another, and GPUs sit mostly idle.
Forgetting	Information about word 1 must survive being squeezed through every hidden state up to word 1,000. In practice it fades (the *vanishing gradient* problem).
Long paths	The signal between two words k positions apart has to pass through k steps, which makes long-range links hard to learn.

Researchers had already added an **attention mechanism** on top of RNNs (Bahdanau et al., 2014) so the model could "look back" at earlier words directly. The big idea of the Transformer paper was:

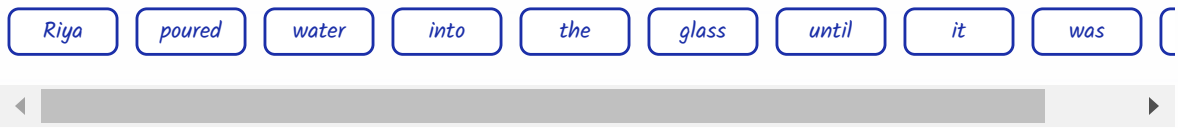
> ***Keep the attention, throw away the recurrence.***

4. Intuition first: the classroom discussion

Imagine a class of students, each holding one word of the sentence on a card.

- **The RNN classroom:** the first student whispers a summary to the second, who adds their word and whispers to the third, and so on. By the end of the line the message is long, slow to deliver and garbled, like a game of telephone.
- **The Transformer classroom:** everyone stands in a circle and can **talk to everyone else at once**. The student holding "it" asks the room, "Who here is a container that can be full?" The student holding "glass" answers loudly, the others quietly. "It" updates its understanding: *I mean the glass.* Every student does this at the same time, in parallel.

Try it: how information flows through one layer



Sequential steps for this layer: – Hops from "glass" to "it": –

Word pairs connected directly: –

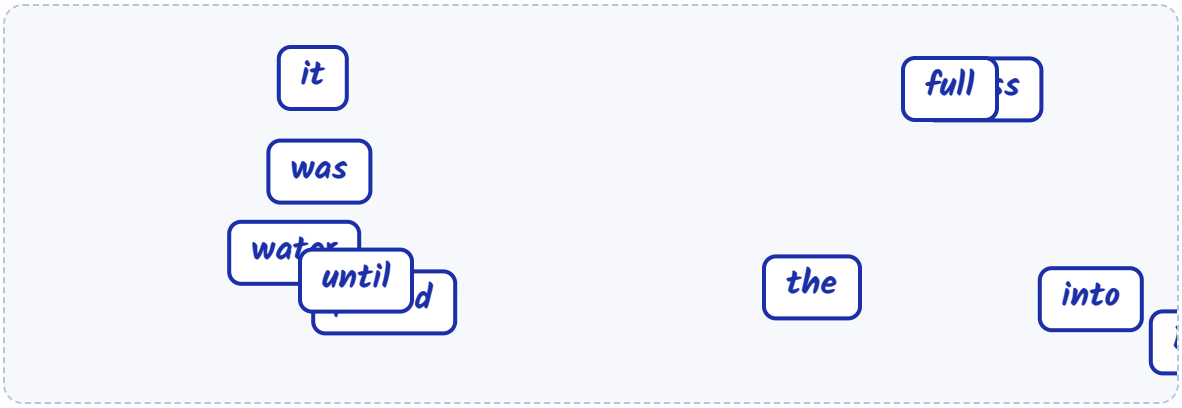
Bottom row: the 10 input tokens. Run a layer to see which computations depend on which, and how many steps it takes.

That room-wide question-and-answer is **self-attention**. After one round, each student has a better, context-aware understanding of their word. Repeat the discussion several times (several **layers**) and the understanding deepens.

Where the analogy breaks

Real students listen to the words in order. The Transformer, by default, has *no sense of order at all*: shuffle the students and it gives the same answers. We fix this by stamping each card with its position number (**positional information**, Section 7).

Try it: what the model sees without positions



No position information: the model receives these 10 tokens as a loose, unordered set, so they float freely. “Riya poured water into the glass” and “glass the into water poured Riya” would give exactly the same token vectors.

5. The formal idea

5.1 Key terms

- **Token**: the basic unit of text the model works with: a word, part of a word, or punctuation mark. (Section 2.)
- **Embedding**: the vector that represents a token. Its length is called the **model width**, written d_{model}.
- **Sequence length** n: the number of tokens in the input.
- **Transformer block** (also called a **layer**): one round of "discussion" plus "individual thinking". A model stacks N identical blocks.

- **Self-attention**: the operation that lets each token gather information from all other tokens. (Sections 3–6.)
- **Feed-forward network (FFN)**: a small two-layer neural network applied to each token *separately*. (Section 8.)

5.2 The input as a matrix

We stack the n token vectors as rows of a matrix:

$$X \in \mathbb{R}^{n \times d_{model}}$$

In words: X has one row per token, and each row is that token's d_{model}-long vector.

5.3 What one block does

Each block takes X in and returns a matrix **of the same shape** out:

$$X' = \text{Block}(X), \qquad X, X' \in \mathbb{R}^{n \times d_{model}}$$

Inside, there are two sub-layers, each wrapped with a **residual connection** (add the input back) and **layer normalization** (rescale the numbers to a stable range):

$$\begin{aligned} A &= X + \text{SelfAttention}(\text{LN}(X)) \\ X' &= A + \text{FFN}(\text{LN}(A)) \end{aligned}$$

In words:

1. **Self-attention**: tokens exchange information ("talk to each other"), and the result is added to what each token already knew.
2. **Feed-forward network**: each token processes what it has just learned on its own ("think individually"), and again the result is added on.

(This is the *pre-norm* arrangement used by most modern models. The 2017 paper normalized *after* adding; Section 8 compares the two.)

Because the output shape equals the input shape, we can stack as many blocks as we like:

$$X^{(N)} = \text{Block}_N(\cdots \text{Block}_2(\text{Block}_1(X^{(0)})))$$

5.4 The full pipeline

1. **Tokenize** the text into n token IDs (integers).
2. **Embed** each ID into a d_{model}-long vector and **add position information**, giving $X^{(0)}$ with shape $n \times d_{model}$.
3. **Pass through N blocks**, giving $X^{(N)}$ with the same shape.
4. **Output head**: a final linear layer turns each row into a score for every word in the vocabulary, shape $n \times \text{vocabulary size}$. A **softmax** turns the scores into probabilities.

5.5 Three flavours (preview of Section 11)

The original 2017 model had two halves:

- An **encoder**, which reads the input sentence and builds understanding.
- A **decoder**, which writes the output sentence one token at a time, looking at the encoder's result.

Since then, many models keep only one half:

Flavour	Example models	Good at
Encoder-only	BERT	Understanding: classification, search
Decoder-only	GPT, Claude, LLaMA	Generating text: chat, code
Encoder–decoder	Original Transformer, T5	Turning one text into another: translation

6. Worked example: tracing shapes

Let's follow a tiny, made-up model through the pipeline.

Setup

- Input text: *"glass is full"*
- Tokens (assume one token per word): `["glass", "is", "full"]`, so $n = 3$.
- Model width $d_{model} = 4$.
- Vocabulary size = 10 (a toy vocabulary).
- Number of blocks $N = 2$.

1. **Tokenize.** Suppose our vocabulary maps `glass → 7`, `is → 2`, `full → 5`.

$$\text{ids} = [7,\ 2,\ 5] \qquad \text{shape } (3,)$$

2. **Embed.** The embedding table has shape 10×4 (one row per vocabulary word). We pick rows 7, 2 and 5:

$$X^{(0)} = \begin{bmatrix} 0.10 & 0.50 & -0.20 & 0.30 \\ 0.40 & 0.00 & 0.10 & -0.10 \\ -0.30 & 0.20 & 0.60 & 0.00 \end{bmatrix} \qquad \text{shape } 3 \times 4$$

(The numbers are made up for illustration; position information is added here too.)

3. **Block 1.** Self-attention mixes information *across rows*, so row 1 (glass) now carries a little of rows 2 and 3. The FFN then transforms each row on its own. Shape in = 3×4, shape out = 3×4.

4. **Block 2.** Same again: $3 \times 4 \to 3 \times 4$.

5. **Output head.** Multiply by a weight matrix of shape 4×10:

$$(3 \times 4) \cdot (4 \times 10) = 3 \times 10$$

Each of the 3 rows now holds 10 scores, one per vocabulary word. After softmax, each row is a probability distribution that sums to 1.

Try it: run the worked example with real numbers

Token IDs shape (3,)

glass	7
is	2
full	5

positive negative darker = larger size

Stage 1, tokenize: each word becomes one integer ID from the 10-word vocabulary.

Shape summary

Stage	Shape
Token IDs	3
Embeddings $X^{(0)}$	3×4
After block 1	3×4
After block 2	3×4
Output scores	3×10

Key observation

The "width" stays at d_{model} through every block. Only the very first step (IDs → vectors) and the very last step (vectors → vocabulary scores) change it.

A second mini-calculation: RNN vs Transformer path length

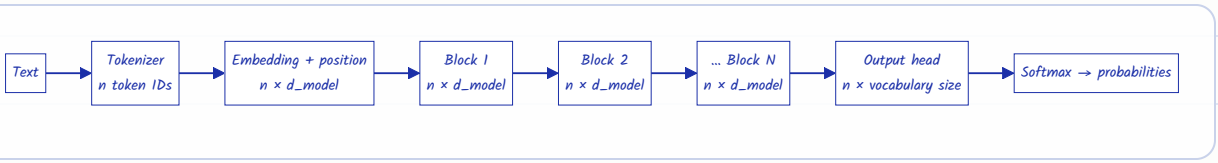
For $n = 3$ tokens, how many steps does information need to travel from token 1 to token 3?

- **RNN:** token 1 → hidden state 1 → hidden state 2 → hidden state 3. That's **2 steps** (in general $n - 1$).
- **Transformer:** token 3 attends to token 1 directly, so **1 step**, whatever n is.

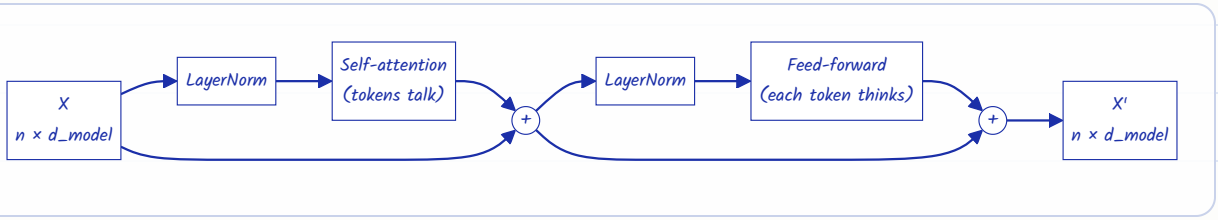
For a 1,000-token document that's 999 steps versus 1 step.

7. Diagram

The full pipeline:



Inside one block:



The arrows that skip around each sub-layer are the **residual connections**.

8. In practice

Real model sizes

Model	Year	Type	Blocks N	d_{model}	Heads h	Parameters
Original Transformer (base)	2017	Encoder–decoder	6 + 6	512	8	≈ 65 M
BERT-base	2018	Encoder-only	12	768	12	≈ 110 M
GPT-2 (small)	2019	Decoder-only	12	768	12	≈ 124 M
LLaMA-2 7B	2023	Decoder-only	32	4096	32	≈ 7 B

Notice the pattern: bigger models mostly get **wider** (larger d_{model}) and **deeper** (more blocks). The block design itself barely changes.

Where Transformers are used today

- **Language:** chatbots, translation, summarization, code completion.
- **Vision:** Vision Transformers split images into patches and treat each patch as a token (Section 15).
- **Speech:** e.g. Whisper turns audio frames into tokens.
- **Science:** e.g. protein-structure models such as AlphaFold 2 use attention-based blocks.

The one big cost

Self-attention compares every token with every other token, so the work grows with n^2. Going from 1,000 to 10,000 tokens makes the attention step about **100× more expensive**. Much of modern research (Section 14) is about reducing this cost.

9. Common mistakes and misconceptions

"A Transformer reads the sentence left to right like a person." Not by default. It processes all tokens at once and has no built-in notion of order. Order comes only from the position information we add.

"Transformer = GPT." GPT is *one* kind of Transformer (decoder-only). BERT (encoder-only) and T5 (encoder–decoder) are Transformers too.

"Attention is the whole model." Attention only moves information *between* tokens. Most of the parameters, roughly two-thirds in typical models, sit in the feed-forward networks, which process each token on its own.

"Each block changes the vector size." No. Every block maps $n \times d_{model}$ to $n \times d_{model}$, which is exactly why blocks can be stacked.

"Transformers have no weaknesses compared with RNNs." Their attention cost grows with n^2 while an RNN's grows with n. For very long sequences that matters, which is why RNN-like ideas (e.g. state-space models) are being revisited.

10. Summary

- The Transformer (Vaswani et al., 2017) replaced step-by-step recurrence with **self-attention**, letting every token look at every other token directly.
- This gives **parallel training** (fast on GPUs) and a **one-step path** between any two tokens (good for long-range meaning).
- The pipeline is: **tokenize → embed + position → N identical blocks → output head → softmax**.
- Each block = **self-attention** (tokens talk) + **feed-forward network** (each token thinks), each with a residual connection and layer normalization.
- Blocks keep the shape $n \times d_{model}$, so they stack cleanly. Bigger models are mainly wider and deeper.
- The main cost: attention work grows with n^2.

11. Check your understanding

1. An RNN and a Transformer each process a 500-token document. Roughly how many *sequential* steps does each need to process the tokens, and why does this matter for training on GPUs?
2. A Transformer block receives an input of shape 20×512. What is the shape of its output? Why must it be this shape?
3. If we shuffle the order of the input tokens and give a Transformer **no** position information, what happens to its output?

▶ **Show answers**

12. Further reading

1. Vaswani, A. et al. (2017). **"Attention Is All You Need."** *Advances in Neural Information Processing Systems 30 (NeurIPS 2017).* arXiv:1706.03762. The original Transformer paper. Read Sections 1–3 now; the rest will make sense after Section 6 of this course.
2. Bahdanau, D., Cho, K., & Bengio, Y. (2014). **"Neural Machine Translation by Jointly Learning to Align and Translate."** arXiv:1409.0473. The paper that introduced attention on top of RNNs; useful for seeing where the idea came from.
3. Jurafsky, D., & Martin, J. H. ***Speech and Language Processing*** (3rd edition draft), the chapter on Transformers and large language models. A clear textbook treatment at undergraduate level, freely available online from Stanford.
4. Alammar, J. **"The Illustrated Transformer"** (blog post, 2018). A highly visual walkthrough, good for building intuition alongside these notes.

Tokens & Embeddings

> **In one line:** before a Transformer can do anything, text must be cut into **tokens**, each token turned into an integer **ID**, and each ID turned into a learned vector called an **embedding**. This section is about those three steps.

1. Learning objectives
2. Prerequisites
3. Motivation
4. Intuition first
5. The formal idea
6. Worked examples
7. Diagram
8. In practice
9. Common mistakes
10. Summary
11. Check your understanding
12. Further reading

1. Learning objectives

By the end of this section you should be able to:

1. **Explain** why models use sub-word tokens instead of whole words or single characters.
2. **Run** the Byte-Pair Encoding (BPE) algorithm by hand on a small corpus, and use the learned merges to tokenize a new word.
3. **Show** that an embedding lookup is the same as multiplying a one-hot vector by the embedding matrix.
4. **Compute** the cosine similarity between two embedding vectors and interpret the result.
5. **Calculate** the size of an embedding table for a real model.

2. Prerequisites

- **Section 1**: the pipeline *tokenize → embed → blocks → output*, and the symbols n (number of tokens) and d_{model} (vector width).
- **Dot product**: $a \cdot b = \sum_i a_i b_i$. For $a = [1, 2]$ and $b = [3, 4]$: $1 \cdot 3 + 2 \cdot 4 = 11$.
- **Vector length (norm)**: $\|a\| = \sqrt{\sum_i a_i^2}$. For $a = [3, 4]$: $\sqrt{9 + 16} = 5$.

> **Quick refresher: one-hot vectors**
>
> A **one-hot vector** has a single 1 and zeros everywhere else. With 5 possible items, item number 3 (counting from 0) is `[0, 0, 0, 1, 0]`. It's the simplest way to say "this one, and nothing else" as numbers.

3. Motivation: computers only understand numbers

A neural network multiplies and adds numbers. It can't multiply the word "tea". So we need a reliable way to turn any text, in any language, including typos, emoji and code, into numbers.

The obvious idea is "give every word a number". It runs into trouble fast:

Approach	Example: "unhappiness"	Problem
Word-level	`["unhappiness"]`	The vocabulary must hold every word in every form: *happy*, *happier*, *happiest*, *unhappy*, *unhappiness*... Any word not in the list (a new name, a typo like "teh") becomes an **unknown** token and its meaning is lost.
Character-level	`["u","n","h","a","p","p","i","n","e","s","s"]`	Only about 100 symbols needed and nothing is ever unknown. But sequences become ~4× longer, and attention cost grows with n^2 (Section 1), so that's roughly 16× more work. Single letters also

		carry almost no meaning.
Sub-word	`["un", "happi", "ness"]`	The middle ground. Common words stay whole, rare words split into meaningful pieces, and nothing is ever unknown. **This is what every modern LLM uses.**

Sub-word tokenization reuses pieces. A model that has learned "un-" (not), "-ness" (state of being) and "happi" can make sense of "unhappiness" even if it saw that exact word rarely.

4. Intuition first

Tokens: LEGO bricks for text

Think of a tokenizer's vocabulary as a box of LEGO bricks. There are small bricks (single letters), medium bricks (common chunks like "ing", "pre", "tion") and big special bricks (very common whole words like "the"). To build any word, use the **biggest bricks that fit**. Frequent words are one big brick; a rare word like "neaten" is built from several smaller ones.

Embeddings: a map of meaning

Now give every brick a location on a map. Bricks with similar meanings sit close together: "tea" near "coffee", both far from "brick". The location is just a list of coordinates, and that list *is* the embedding vector.

Where the analogy breaks

- A real map has 2 dimensions; real embeddings have hundreds or thousands (d_{model} = 768, 4096, ...).
- On a real map the axes mean "north" and "east". Embedding axes have **no human-chosen meaning**. The model learns whatever directions are useful, and they're usually not interpretable one at a time.

5. The formal idea

5.1 Key terms

- **Tokenizer**: the fixed procedure that splits text into tokens and maps them to IDs. It's built *before* the model is trained and never changes afterwards.
- **Vocabulary**: the list of all tokens the tokenizer can produce. Its size is the **vocabulary size**, written $|\mathcal{V}|$ in this section so it isn't confused with the value matrix V later.
- **Token ID**: a token's position in the vocabulary, an integer from 0 to $|\mathcal{V}| - 1$.
- **Embedding matrix** E: a learned table with one row per token, shape $|\mathcal{V}| \times d_{model}$.
- **Embedding**: one row of E, the vector that represents a token.

5.2 Byte-Pair Encoding (BPE): how the vocabulary is learned

BPE builds the vocabulary from a training corpus by repeatedly gluing together the most common neighbouring pair:

1. Start with every word split into **single characters**. The initial vocabulary is the set of characters.
2. Count every **adjacent pair** of tokens across the corpus, weighting each word by how often it occurs.
3. **Merge** the most frequent pair into one new token, add it to the vocabulary, and record the merge rule.
4. Repeat steps 2–3 until the vocabulary reaches the target size (e.g. 32,000 or 50,000 tokens).

To tokenize new text, split it into characters and apply the recorded merges **in the order they were learned**.

5.3 From token IDs to vectors: the embedding lookup

Let token i have the one-hot vector o_i (length $|\mathcal{V}|$, with a 1 at position i). Its embedding is

$$x_i = o_i^\top E = E[i, :]$$

In words: multiplying a one-hot vector by E picks out exactly row i. In practice libraries skip the multiplication and simply **index** row i, which is much faster and gives the same result.

For a sequence of token IDs $(t_1, \ldots, t_n)$ we stack the rows:

$$X = \begin{bmatrix} E[t_1, :] \\ \vdots \\ E[t_n, :] \end{bmatrix} \in \mathbb{R}^{n \times d_{model}}$$

This is the matrix X from Section 1 (before position information is added).

5.4 Embeddings are learned

E starts as **small random numbers**. During training, gradients flow back into the rows of the tokens that appeared in each batch, nudging them so the model's predictions improve. Tokens used in similar contexts receive similar nudges, so they end up with **similar vectors**. Nobody tells the model that "tea" and "coffee" are related; it discovers this because they appear in similar sentences ("a hot cup of ___").

5.5 Measuring similarity: cosine similarity

$$\cos(a, b) = \frac{a \cdot b}{\|a\| \, \|b\|}$$

In words: the dot product divided by both lengths. It measures the **angle** between two vectors and ignores their lengths:

Value	Angle	Meaning
1	0°	Same direction: very similar
0	90°	Unrelated
-1	180°	Opposite directions

5.6 Two practical details

- **Special tokens.** Vocabularies include tokens that aren't text, such as `<bos>` (beginning of sequence), `<eos>` (end), `<pad>` (filler to make sequences the same length), and BERT's `[CLS]`, `[SEP]` and `[MASK]`. They get embeddings like any other token.
- **Weight tying.** The output head (Section 1) maps vectors back to vocabulary scores with a $d_{model} \times |\mathcal{V}|$ matrix. Many models simply reuse $E^\top$ for this, which saves $|\mathcal{V}| \cdot d_{model}$ parameters.

6. Worked examples

6.1 BPE by hand

Corpus (word: how many times it appears): `tea` × 4, `ten` × 3, `tent` × 2, `neat` × 1.

Start: split into characters. Vocabulary = `{t, e, a, n}`.

Word	Tokens	Count
tea	t · e · a	4
ten	t · e · n	3
tent	t · e · n · t	2
neat	n · e · a · t	1

Round 1: count the pairs.

- `t e` : 4 (tea) + 3 (ten) + 2 (tent) = **9**
- `e a` : 4 (tea) + 1 (neat) = 5
- `e n` : 3 (ten) + 2 (tent) = 5
- `n t` : 2 (tent) = 2
- `n e` : 1, `a t` : 1

Most frequent: `t e` (9), so **merge 1: t + e → te**.

Round 2. Words are now `te·a`, `te·n`, `te·n·t`, `n·e·a·t`. Pairs: `te n` = 3 + 2 = **5**, `te a` = 4, `n t` = 2, `n e` = `e a` = `a t` = 1. **Merge 2: te + n → ten**.

Round 3. Words: `te·a`, `ten`, `ten·t`, `n·e·a·t`. Pairs: `te a` = **4**, `ten t` = 2, others 1. **Merge 3: te + a → tea**.

Round 4. Words: `tea`, `ten`, `ten·t`, `n·e·a·t`. Pairs: `ten t` = **2**, others 1. **Merge 4: ten + t → tent**.

Result. Vocabulary = `{t, e, a, n, te, ten, tea, tent}` (8 tokens), with the merge rules in order: 1. t + e, 2. te + n, 3. te + a, 4. ten + t. Notice that `neat` was too rare to earn its own token; it's still four characters.

Tokenizing new words (apply the merges in order):

New word	After merging	Tokens
tent	t·e·n·t → te·n·t → ten·t → tent	`["tent"]`
teat	t·e·a·t → te·a·t → tea·t	`["tea", "t"]`
neaten	n·e·a·t·e·n → n·e·a·te·n → n·e·a·ten	`["n", "e", "a", "ten"]`

"neaten" never appeared in the corpus, yet it was tokenized with no unknown token. That is the main strength of sub-word tokenization.

Try it: train BPE yourself, one merge at a time

Corpus, split into current tokens

tea	t e a	× 4
ten	t e n	× 3
tent	t e n t	× 2
neat	n e a t	× 1

Pair counts (count × word frequency)

pair	count
t e	9
e a	5
e n	5
n t	2
n e	1
a t	1

Vocabulary

t e a n

Merge rules learned (in order)

(none yet)

Tokenize a new word with the rules learned so far

start: n e a t e n

Result: `["n","e","a","t","e","n"]` (6 tokens)

Every word starts as single characters. The table on the right counts each neighbouring pair, multiplied by how often the word appears.

6.2 Embedding lookup = one-hot × matrix

A toy vocabulary of 5 tokens with $d_{model} = 3$:

ID	Token	Row of E
0	`<pad>`	[0.00, 0.00, 0.00]
1	tea	[0.90, 0.10, 0.30]
2	is	[−0.20, 0.40, 0.10]
3	hot	[0.30, −0.50, 0.80]
4	cold	[0.20, −0.60, −0.70]

Look up "hot" (ID 3). One-hot: $o_3 = [0, 0, 0, 1, 0]$. First entry of $o_3^\top E$:

$$0(0.00) + 0(0.90) + 0(-0.20) + 1(0.30) + 0(0.20) = 0.30$$

Doing the same for the other two columns gives $[0.30, -0.50, 0.80]$, exactly row 3. Every zero in the one-hot vector wipes out a row, and the single 1 keeps row 3.

Whole sentence "tea is hot": IDs $[1, 2, 3]$, so

$$X = \begin{bmatrix} 0.90 & 0.10 & 0.30 \\ -0.20 & 0.40 & 0.10 \\ 0.30 & -0.50 & 0.80 \end{bmatrix} \qquad \text{shape } 3 \times 3 \ (n \times d_{model})$$

Try it: one-hot × E picks out one row

one-hot o_3 (1 × 5)

	0	1	2	3	4
"hot"	0	0	0	**1**	0

×

E (5 × 3)

0 <pad>	0.00	0.00	0.00
1 tea	0.90	0.10	0.30
2 is	-0.20	0.40	0.10
3 hot	0.30	-0.50	0.80
4 cold	0.20	-0.60	-0.70

=

embedding x (1 × 3)

"hot"

First entry: 0(0.00) + 0(0.90) + 0(−0.20) + 1(0.30) + 0(0.20) = **0.30**. Every 0 in the one-hot vector wipes out a row of E, so the result is exactly row 3 ("hot").

6.3 Cosine similarity by hand

Three 3-dimensional embeddings (toy values):

- tea = [0.9, 0.1, 0.3]
- coffee = [0.8, 0.2, 0.4]
- brick = [0.1, 0.9, −0.2]

tea vs coffee

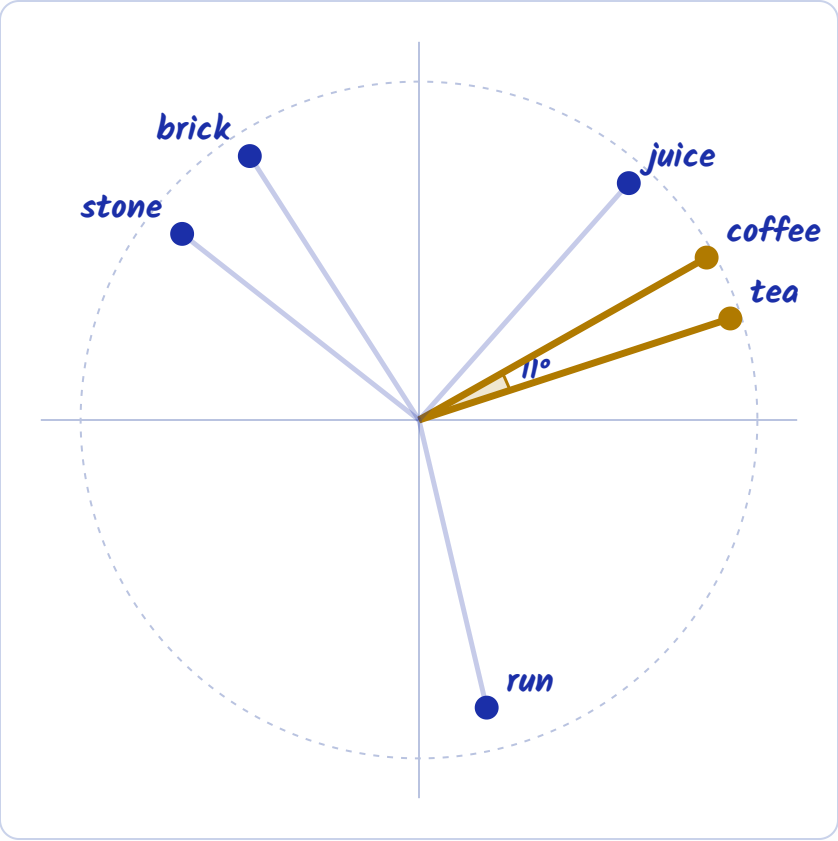
- Dot product: $0.9(0.8) + 0.1(0.2) + 0.3(0.4) = 0.72 + 0.02 + 0.12 = 0.86$
- Lengths: $\|\text{tea}\| = \sqrt{0.81 + 0.01 + 0.09} = \sqrt{0.91} = 0.954$ and $\|\text{coffee}\| = \sqrt{0.64 + 0.04 + 0.16} = \sqrt{0.84} = 0.917$
- Cosine: $0.86/(0.954 \times 0.917) = 0.86/0.874 = \mathbf{0.98}$, very similar.

tea vs brick

- Dot product: $0.9(0.1) + 0.1(0.9) + 0.3(-0.2) = 0.09 + 0.09 - 0.06 = 0.12$
- $\|\text{brick}\| = \sqrt{0.01 + 0.81 + 0.04} = \sqrt{0.86} = 0.927$
- Cosine: $0.12 / (0.954 \times 0.927) = 0.12 / 0.885 = \mathbf{0.14}$, nearly unrelated.

Try it: cosine similarity is about the angle

These are toy **2-dimensional** embeddings, so they can be drawn exactly. Real embeddings have hundreds of dimensions, but the formula is the same.



Click two words on the plot (or below).

tea | coffee | juice | brick | stone | run

cos = 0.98

`tea = [0.92, 0.30]`
`coffee = [0.85, 0.48]`
dot = 0.92×0.85 + 0.30×0.48 = **0.93**
lengths = 0.968 and 0.976
cos = 0.93 / (0.968 × 0.976) = **0.98** → angle 11.4°

“tea” and “coffee” are pointing almost the same way: very similar. Only the angle matters; the lengths of the arrows cancel out.

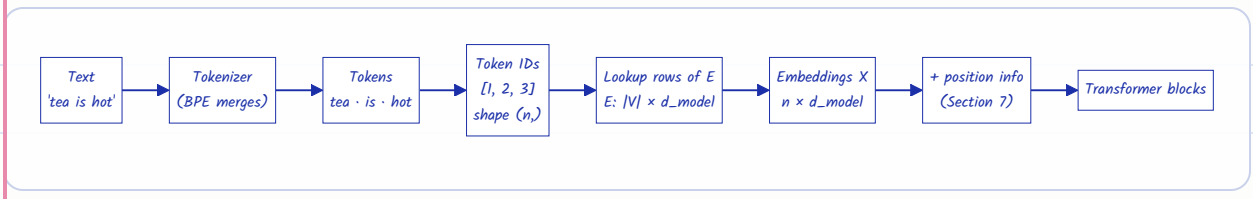
6.4 How big is an embedding table?

Parameters = vocabulary size × d_{model}.

- **GPT-2 (small):** $50{,}257 \times 768 = 38{,}597{,}376 \approx 38.6$ M parameters, about **31%** of its ≈124 M total. At 4 bytes per number (FP32) that's ≈154 MB.
- **LLaMA-2 7B:** $32{,}000 \times 4{,}096 = 131{,}072{,}000 \approx 131$ M parameters, under 2% of 7 B.

Small models spend a surprisingly large share of their parameters on the embedding table.

7. Diagram



8. In practice

Real tokenizers

Model	Tokenizer	Vocabulary size
BERT-base	WordPiece	30,522
GPT-2	Byte-level BPE	50,257
LLaMA-2	SentencePiece (BPE)	32,000
LLaMA-3	Byte-level BPE (tiktoken-based)	128,256

- **WordPiece** is a close cousin of BPE: it picks merges that most increase the likelihood of the training data, and marks word-internal pieces with `##` (e.g. `play`, `##ing`).

- **SentencePiece** works directly on raw text, treating spaces as a normal symbol (shown as `▁`). This suits languages without spaces, like Japanese or Chinese.
- **Byte-level BPE** starts from the 256 possible bytes instead of characters, so *any* input (emoji, any script, binary junk) can be tokenized with no unknown tokens at all.

Rules of thumb (English text)

- 1 token ≈ 4 characters ≈ ¾ of a word, so 1,000 tokens ≈ 750 words.
- Other languages, code and numbers usually need **more tokens per word**. That makes them more expensive and uses up the context window faster.

Typical embedding widths

d_{model} = 768 (BERT-base, GPT-2 small), 4,096 (LLaMA-2 7B), 8,192 (LLaMA-2 70B).

9. Common mistakes and misconceptions

"A token is a word." Often it's part of a word, a single character, a space plus a word (`" the"`), or punctuation. "Tokens" and "words" are different units, which is why API prices and context limits are counted in tokens.

"Token ID 500 is similar to token ID 501." IDs are just positions in a list and carry no meaning. All meaning lives in the embedding vectors.

"The embedding table is a fixed dictionary someone wrote." It's learned from data by gradient descent, exactly like the other weights.

"A bigger vocabulary is always better." Bigger vocabularies give shorter sequences but a bigger embedding table, and many rare tokens that are seldom trained. It's a trade-off, which is why sizes cluster around 30k–130k.

> **"LLMs read letters."** They read tokens. That's why tasks like "how many r's are in *strawberry*?" or reversing a word can be surprisingly hard: the model may never see the individual letters.

10. Summary

- Text → **tokens** → **IDs** → **embeddings**, giving the matrix X of shape $n \times d_{model}$.
- **Sub-word tokenization** sits between word-level (huge vocabulary, unknown words) and character-level (long sequences): common words stay whole, rare words split into pieces, nothing is unknown.
- **BPE** learns the vocabulary by repeatedly merging the most frequent adjacent pair, and tokenizes new text by replaying those merges in order.
- An **embedding lookup** picks row i of the matrix E, which is the same as one-hot $\times E$.
- Embeddings are **learned**; similar usage leads to similar vectors, measured with **cosine similarity**.
- The embedding table has vocabulary size $\times d_{model}$ parameters, a large share of a small model.

11. Check your understanding

1. Using the four merges learned in 6.1, tokenize the word **"tenet"**.
2. A model has a vocabulary of 32,000 tokens and $d_{model} = 512$. How many parameters are in its embedding table? If the output head uses weight tying, how many extra parameters does the output head's weight matrix add?
3. Two embeddings have cosine similarity -0.95. What does that tell you about the angle between them? Does it mean the two words have "opposite meanings"?

▶ *Show answers*

12. Further reading

1. Sennrich, R., Haddow, B., & Birch, A. (2016). **"Neural Machine Translation of Rare Words with Subword Units."** *ACL 2016.* arXiv:1508.07909. The paper that brought BPE to NLP.
2. Kudo, T., & Richardson, J. (2018). **"SentencePiece: A simple and language independent subword tokenizer and detokenizer for Neural Text Processing."** arXiv:1808.06226.
3. Mikolov, T. et al. (2013). **"Efficient Estimation of Word Representations in Vector Space."** arXiv:1301.3781. word2vec, the classic demonstration that learned word vectors capture meaning.
4. Jurafsky, D., & Martin, J. H. ***Speech and Language Processing*** (3rd edition draft), the chapter on vector semantics and embeddings.

Prepared by Akhil

Transformers & LLMs · Section 3

Self-Attention: the Intuition

> **In one line:** self-attention gives every token a new vector that is a **weighted average of all the tokens' vectors**, where the weights say how relevant each token is. This is how "bat" comes to mean *animal* in one sentence and *cricket bat* in another.

1. Learning objectives
2. Prerequisites
3. Motivation
4. Intuition first
5. The formal idea
6. Worked example
7. Diagram
8. In practice
9. Common mistakes
10. Summary
11. Check your understanding
12. Further reading

1. Learning objectives

By the end of this section you should be able to:

1. **Explain** why a token's embedding alone isn't enough, and why its meaning must depend on context.
2. **Describe** self-attention as three steps: *score* → *normalize (softmax)* → *weighted average*.
3. **Compute** a simple self-attention output by hand for a 3-token sentence.
4. **Identify** the key properties of self-attention: global, input-dependent, parallel, order-blind and $O(n^2)$.
5. **Explain** the weakness of the naive version (each token mostly attends to itself), which motivates queries, keys and values in Section 4.

2. Prerequisites

- **Section 2**: each token becomes an embedding vector, and the sentence becomes a matrix X ($n \times d_{model}$). The dot product measures how aligned two vectors are.
- **Weighted average**: $\sum_j w_j v_j$ with weights $w_j \geq 0$ that add up to 1. Example: with weights 0.75 and 0.25, the average of 10 and 20 is $0.75(10) + 0.25(20) = 12.5$.

> **Quick refresher: the softmax function**
>
> Softmax turns any list of numbers ("scores") into positive weights that sum to 1:
>
> $$\text{softmax}(s)_j = \frac{e^{s_j}}{\sum_k e^{s_k}}$$
>
> Example: scores $[2, 0, 0] \to e^2 = 7.39$, $e^0 = 1$, $e^0 = 1$, total $9.39 \to$ weights $[0.79, 0.11, 0.11]$. The largest score gets the largest weight, but every item keeps a little.

3. Motivation: the same word, different meanings

Compare:

> *The **bat** flew out of the cave.*
> *The **bat** hit the ball for six.*

The tokenizer gives "bat" the **same ID**, and the embedding table gives it the **same vector**, in both sentences. But the meaning is completely different: an animal versus a piece of cricket equipment.

So after the embedding step, every token's vector is **context-free**. The model needs a way to update each vector using the other words around it:

- "bat" next to "flew" and "cave" → shift toward *animal*.
- "bat" next to "hit" and "ball" → shift toward *sports equipment*.

This happens everywhere in language: pronouns ("it", "she"), words with several meanings ("bank", "light", "match"), and words whose role depends on grammar ("run" as a noun or a verb). **Self-attention is the mechanism that mixes context into each token's vector.**

4. Intuition first: asking the room for help

Each token is a student in a group discussion (as in Section 1). The student holding "bat" is unsure what their word means, so they:

1. **Score**: check how relevant each other student's word is to them. ("flew" feels very relevant; "the" barely matters.)
2. **Normalize**: decide how to split their attention, 100% in total, across everyone, giving more to relevant words.
3. **Blend**: form a new understanding by mixing everyone's information in those proportions.

After this, "bat" is no longer the bare dictionary word. It's "bat, *as used in this sentence*". Every student does the same thing at the same time.

Where the analogy breaks

- Real students decide relevance using knowledge and reasoning. Self-attention uses a simple, learned **similarity score** (a dot product), and nothing more.
- A student wouldn't usually listen mostly to themselves. In the naive version below, tokens often do (Section 6.3). Fixing that is exactly why Section 4 introduces queries, keys and values.

5. The formal idea

5.1 The three steps

For token i in a sentence with vectors $x_1, \ldots, x_n$:

Step 1, score every token j by similarity (dot product):

$$s_{ij} = x_i \cdot x_j$$

Step 2, normalize the scores into weights with softmax:

$$a_{ij} = \frac{e^{s_{ij}}}{\sum_{k=1}^{n} e^{s_{ik}}} \qquad \text{so } a_{ij} \geq 0 \text{ and } \sum_j a_{ij} = 1$$

Step 3, blend: the new vector is the weighted average:

$$y_i = \sum_{j=1}^{n} a_{ij}\, x_j$$

In words: *token i's new vector is made of all the token vectors, mixed in proportion to how similar each one is to token i.* The weights a_{ij} are called **attention weights**. "Token i attends to token j with weight a_{ij}."

5.2 Matrix form: all tokens at once

Stacking every token as a row of X ($n \times d_{model}$):

$$S = XX^\top \quad (n \times n), \qquad A = \text{softmax}_{\text{rows}}(S) \quad (n \times n), \qquad Y = AX$$

- Row i of S holds token i's scores against every token.
- Row i of A holds token i's attention weights, and **each row sums to 1**.
- Row i of Y is token i's new, context-aware vector. **Y has the same shape as X.**

This is **three matrix multiplications and a softmax**, with no loops over time steps, so the whole sentence is processed in parallel.

5.3 This is a simplified version

Real Transformers make two changes, covered next:

- **Section 4:** instead of comparing x_i with x_j directly, the model first projects each token into a **query**, a **key** and a **value** using learned matrices.
- **Section 5:** the scores are divided by $\sqrt{d_k}$ before the softmax to keep them in a sensible range.

The three-step skeleton (*score → softmax → weighted average*) stays the same.

5.4 Key properties

Property	What it means
Global	Every token can draw from every other token in a single step, however far apart.
Input-dependent	The weights are *computed* from the current sentence, not stored. A different sentence gives different weights.
Parallel	All n outputs come from a few matrix multiplications at once.
Order-blind	Shuffling the tokens just shuffles the outputs; each token's own output is unchanged. (Section 7 adds positions.)
Quadratic cost	S has $n \times n$ entries, so time and memory grow with n^2.

5.5 Two variants you will meet

- **Bidirectional self-attention**: every token may attend to every token, before and after it. Used in encoders such as BERT.
- **Causal (masked) self-attention**: token i may attend only to tokens $1, \ldots, i$, never the future. Used in GPT-style decoders that generate text left to right (Section 10).
- **Cross-attention** (different, but related): tokens of one sequence attend to tokens of *another* sequence, e.g. a translation decoder attending to the source sentence.

6. Worked example: "bat" in two sentences

To make the result easy to see, we use **toy 2-D embeddings whose two axes we chose to mean** *animal-ness* and *sport-ness*. (Real embedding axes have no such labels; this is only for teaching.)

Token	Vector
the	[0.2, 0.2] (weak, generic)
bat	[1, 1] (ambiguous: equally animal and sport)
flew	[2, 0] (strongly animal)
hit	[0, 2] (strongly sport)

6.1 Sentence A: "the bat flew"

Step 1, scores for "bat" (dot product with every token):

- bat · the = $1(0.2) + 1(0.2) = 0.4$
- bat · bat = $1(1) + 1(1) = 2$
- bat · flew = $1(2) + 1(0) = 2$

Step 2, softmax: $e^{0.4} = 1.49$, $e^{2} = 7.39$, $e^{2} = 7.39$, total $= 16.27$.

attends to	the	bat	flew
weight	1.49 / 16.27 = **0.09**	7.39 / 16.27 = **0.45**	7.39 / 16.27 = **0.45**

"the" gets almost no attention; "bat" and "flew" share the rest.

Step 3, weighted average:

$$y_{\text{bat}} = 0.09\,[0.2, 0.2] + 0.45\,[1, 1] + 0.45\,[2, 0] = [1.38,\ 0.47]$$

"bat" started at $[1, 1]$ and is now $[1.38, 0.47]$: it has **moved toward the animal axis**.

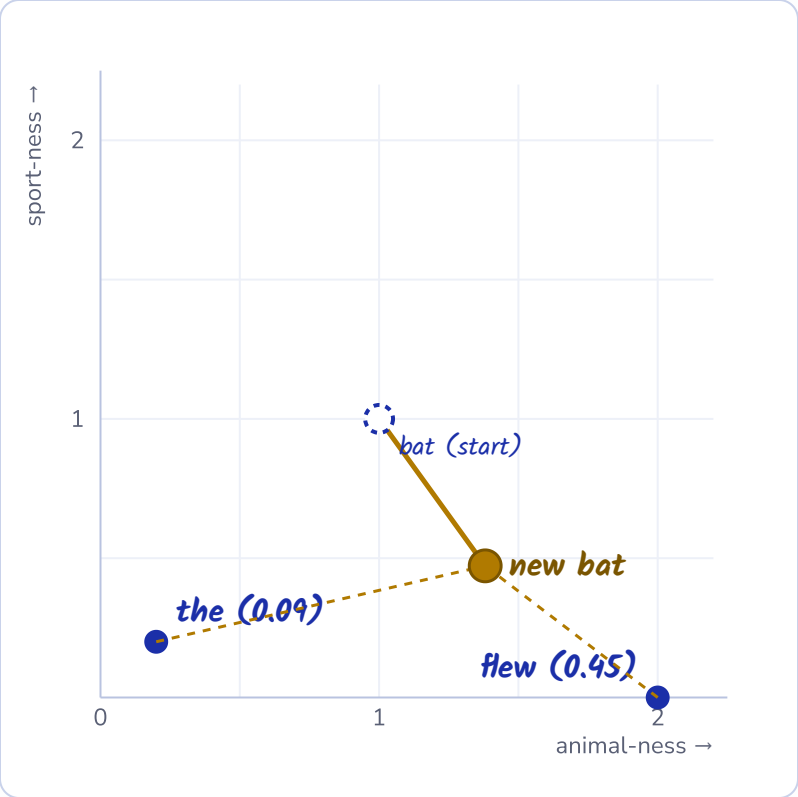
6.2 Sentence B: "the bat hit"

The scores are identical (bat · hit $= 1(0) + 1(2) = 2$), so the weights are the same: $0.09, 0.45, 0.45$. But the vectors being averaged are different:

$$y_{\text{bat}} = 0.09\,[0.2, 0.2] + 0.45\,[1, 1] + 0.45\,[0, 2] = [0.47,\ 1.38]$$

Now "bat" has **moved toward the sport axis**. The same word, with the same starting vector, ends up with a different meaning purely because of its neighbours.

Try it: watch context move "bat"



How much "bat" attends to each word

the	0.09
bat	0.45
flew	0.45

1. Scores (bat · each): the 0.40, bat 2.00, flew 2.00

2. Softmax: $e^{0.40}$=1.49, $e^{2.00}$=7.39, $e^{2.00}$=7.39 → weights 0.09, 0.45, 0.45

3. Blend: 0.09·[0.20, 0.20] + 0.45·[1.00, 1.00] + 0.45·[2.00, 0.00] = **[1.38, 0.47]**

In “the bat flew”, “bat” moves from [1.00, 1.00] to [1.38, 0.47]: toward animal. The new vector is a weighted average, so it always lands inside the region spanned by the sentence's vectors.

6.3 The whole sentence at once, and a problem

For sentence A, the full score matrix $S = XX^\top$ and attention matrix A (rows = the token doing the attending):

S	the	bat	flew
the	0.08	0.40	0.40
bat	0.40	2.00	2.00
flew	0.40	2.00	4.00

A (row softmax)	the	bat	flew	row sum
the	0.27	0.37	0.37	1
bat	0.09	0.45	0.45	1
flew	0.02	0.12	**0.86**	1

Look at the "flew" row: it puts **86% of its attention on itself**. That's because a vector's dot product with itself is its squared length, $||x||^2$, which is usually the largest score. In this naive version, long vectors mostly "listen to themselves" and learn little from context.

The fix (Section 4)

Compare a token's *question* (query) with other tokens' *labels* (keys), computed by separate learned matrices. Then "what I'm looking for" no longer has to equal "what I am", and a token can attend strongly to others.

Try it: the full attention matrix

Scores $S = XX^T$ $n \times n$

	the	bat	flew
the	0.08	0.40	0.40
bat	0.40	2.00	2.00
flew	0.40	2.00	4.00

Attention $A = \text{softmax}(S / 1.00)$ rows sum to 1

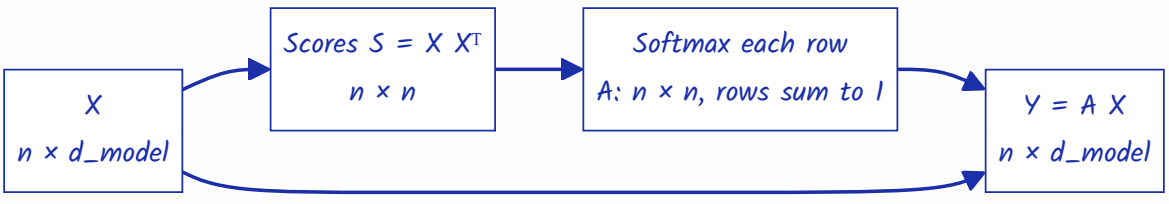
	the	bat	flew	sum
the	0.27	0.37	0.37	1.00
bat	0.09	0.45	0.45	1.00
flew	0.02	0.12	0.86	1.00

Output $Y = AX$ $n \times 2$

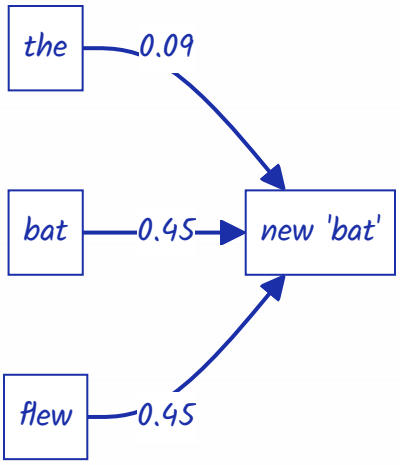
	dim 1	dim 2
the	1.15	0.42
bat	1.38	0.47
flew	1.84	0.12

Gold outlines mark the diagonal (each token attending to itself). Here “flew” gives 86% of its attention to itself. Move the slider to see softmax get sharper or flatter.

7. Diagram



One token's view:



8. In practice

What real attention heads learn

Studies of trained models find heads with recognisable jobs:

- **Previous-token heads**: mostly attend to the token just before.
- **Syntax heads**: e.g. a verb attending to its subject, or a noun to its article.
- **Coreference heads**: pronouns attending to the noun they refer to ("it" → "glass").
- **Delimiter heads**: attend to punctuation or special tokens like `[SEP]` when they have "nothing better to do".
- **Induction heads**: if "A B … A" appears, attend to what followed the earlier "A" and predict "B". Researchers believe these drive much of **in-context learning**.

Scale

- Each layer of a model has many attention heads (e.g. 12 in BERT-base, 32 in LLaMA-2 7B; Section 6) and many layers, so a token's vector is updated by attention dozens of times.
- For a 2,000-token input, one head in one layer computes $2{,}000^2 = 4{,}000{,}000$ scores. This n^2 cost is why long contexts are expensive (Section 14).

9. Common mistakes and misconceptions

"Attention weights are learned parameters stored in the model." No. The weights a_{ij} are **computed fresh for every input**. What is learned are the projection matrices (Section 4) that produce the scores.

"Attention only looks at nearby words." It looks at **all** tokens. Any preference for nearby words has to be learned or come from position information.

"Attention weights can be negative, so a token can subtract another's meaning." Softmax outputs are always **between 0 and 1** and sum to 1. The new vector is always a weighted *average* (a mixture).

"High attention weight means the model's decision was caused by that word." Not necessarily. Attention weights are one internal signal among many, and research shows they are not a reliable explanation of model decisions on their own.

"Self-attention understands word order." On its own it doesn't. "dog bites man" and "man bites dog" give each word the same output vector. Order comes from positional information (Section 7). Try "Shuffle word order" in the matrix demo above.

10. Summary

- Embeddings are **context-free**; self-attention makes them **context-aware**.
- Self-attention = **score** (dot products) → **softmax** (weights that sum to 1) → **weighted average** of vectors.
- Matrix form: $S = XX^\top$, $A = \mathrm{softmax}_{\mathrm{rows}}(S)$, $Y = AX$. The output has the same shape as the input.
- It is **global**, **input-dependent**, **parallel**, **order-blind**, and costs $O(n^2)$.

- The naive version tends to make tokens attend to **themselves**; queries, keys and values (Section 4) fix this.

11. Check your understanding

1. A token's scores against three tokens are $[2, 0, 0]$. Compute its attention weights. If all three scores were equal, what would the output vector be?
2. Using the vectors in Section 6, compute the new vector for "bat" if "the" were removed, i.e. the sentence is just "bat flew".
3. Why does the "flew" row in 6.3 give itself 0.86 of its attention? Suggest in one sentence what a model would need in order to make "flew" attend more to "bat".

▶ **Show answers**

12. Further reading

1. Vaswani, A. et al. (2017). **"Attention Is All You Need."** *NeurIPS 2017.* arXiv:1706.03762. Section 3.2 introduces the attention used in Transformers.
2. Bahdanau, D., Cho, K., & Bengio, Y. (2014). **"Neural Machine Translation by Jointly Learning to Align and Translate."** arXiv:1409.0473. The original "soft alignment" idea behind attention.
3. Clark, K., Khandelwal, U., Levy, O., & Manning, C. D. (2019). **"What Does BERT Look At? An Analysis of BERT's Attention."** arXiv:1906.04341. Finds heads for syntax, coreference and delimiters.
4. Jain, S., & Wallace, B. C. (2019). **"Attention is not Explanation."** *NAACL 2019.* arXiv:1902.10186. Why attention weights should be read with care.
5. Olsson, C. et al. (2022). **"In-context Learning and Induction Heads."** Anthropic, Transformer Circuits Thread. On the heads behind in-context learning.

Queries, Keys & Values

> **In one line:** instead of comparing tokens directly, each token is projected into three different vectors (a **query**: what it is looking for, a **key**: how it can be found, and a **value**: what it hands over), using three learned weight matrices.

1. Learning objectives
2. Prerequisites
3. Motivation
4. Intuition first
5. The formal idea
6. Worked examples
7. Diagram
8. In practice
9. Common mistakes
10. Summary
11. Check your understanding
12. Further reading

1. Learning objectives

By the end of this section you should be able to:

1. **Explain** why naive self-attention (Section 3) needs separate queries, keys and values.
2. **Compute** Q, K and V from an input matrix X and three weight matrices.
3. **State** the shape of every matrix involved, for any n, d_{model} and d_k.
4. **Show** that with $W_Q \neq W_K$ attention scores become asymmetric ($s_{ij} \neq s_{ji}$), and explain why that is useful.
5. **Count** the parameters in the Q/K/V projections for a real model.

2. Prerequisites

- **Section 3**: self-attention as *score → softmax → weighted average*, and its weakness: in the naive version, "flew" put 86% of its attention on itself.
- **Matrix shapes**: $(m \times k)(k \times p) = (m \times p)$. A 1×2 row vector times a 2×2 matrix is another 1×2 row vector.

> **Quick refresher: a linear projection**
>
> Multiplying a vector by a matrix, xW, is called a **linear projection**. It can rotate, stretch, shrink or mix the vector's entries, and it can change the vector's length (number of entries). Example: $[1, 2]\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} = [2, 1]$ swaps the two entries.

3. Motivation: one vector can't play three roles

In Section 3 each token's single vector x_i did **three jobs at once**:

1. As the thing **asking** "who is relevant to me?" (x_i on the left of $x_i \cdot x_j$)
2. As the thing **being matched** by others (x_j on the right of $x_i \cdot x_j$)
3. As the **content** that gets mixed into the output (x_j in $\sum_j a_{ij} x_j$)

Using one vector for all three causes two problems:

- **Self-obsession.** A vector always matches itself well ($x_i \cdot x_i = \|x_i\|^2$), so tokens over-attend to themselves. We saw "flew" give itself 86%.
- **Forced symmetry.** $x_i \cdot x_j = x_j \cdot x_i$, so if "it" finds "glass" relevant, "glass" must find "it" *equally* relevant. Language isn't like that: a pronoun badly needs its noun, but the noun usually doesn't need the pronoun.

The fix is simple: give each role its **own learned projection**.

4. Intuition first: a job portal

Think of a job-search website.

- A **recruiter** types a search: *"Python developer, 3 years, Hyderabad"*. That is the **query**: what they are looking for.
- Every candidate has a **profile headline**: *"Backend engineer · Python · Django"*. That is the **key**: a short summary designed to be *matched*.
- When the recruiter opens a profile, they get the **full CV**: projects, skills, experience. That is the **value**: the actual content handed over.

The portal ranks candidates by how well the **query matches each key**, then the recruiter reads the **values** of the best matches, paying more attention to higher-ranked ones.

Three important points:

- What you *search for* is different from what *you are*. A recruiter isn't a Python developer.
- A headline (key) is short and built for matching; the CV (value) is the rich content. They're related but not the same.
- In **self**-attention every token plays **both sides**: it is a recruiter (with a query) *and* a candidate (with a key and a value).

> **Where the analogy breaks**
>
> Nobody writes these queries, keys and values by hand. All three are produced by learned matrices, and the model discovers during training what is useful to search for and to advertise.

5. The formal idea

5.1 Three projections

Each token's vector x_i (a row of X) is multiplied by three **learned** weight matrices:

$$q_i = x_i W_Q, \qquad k_i = x_i W_K, \qquad v_i = x_i W_V$$

For the whole sentence at once:

$$Q = X W_Q, \qquad K = X W_K, \qquad V = X W_V$$

- **Query** q_i: "what token i is looking for".
- **Key** k_j: "what token j offers to be matched against".
- **Value** v_j: "the information token j passes on if attended to".

5.2 Attention with Q, K, V

The three steps from Section 3 stay the same, but each step now uses the right vector:

$$s_{ij} = q_i \cdot k_j \qquad a_{ij} = \mathrm{softmax}_j(s_{ij}) \qquad y_i = \sum_j a_{ij}\, v_j$$

In matrix form:

$$S = QK^\top, \qquad A = \mathrm{softmax}_{\mathrm{rows}}(S), \qquad Y = AV$$

In words: *match each query against every key, turn the matches into weights, and use the weights to mix the values.*

(Section 5 adds one more detail: dividing S by $\sqrt{d_k}$. We leave it out here to keep the numbers simple.)

5.3 Shapes

Matrix	Shape	Notes
X	$n \times d_{model}$	one row per token
W_Q, W_K	$d_{model} \times d_k$	learned, **shared by every position**
W_V	$d_{model} \times d_v$	learned
Q, K	$n \times d_k$	queries and keys must have the **same width** so their dot product exists
V	$n \times d_v$	
S, A	$n \times n$	one score per (query, key) pair
Y	$n \times d_v$	

Usually $d_k = d_v$, and they are **smaller** than d_{model} because the model runs several attention "heads" side by side (Section 6). For example, BERT-base has $d_{model} = 768$ and 12 heads, so $d_k = 768/12 = 64$.

5.4 Why this fixes both problems

- **No forced self-attention.** Token i's self-score is now $q_i \cdot k_i = (x_i W_Q) \cdot (x_i W_K)$. Because W_Q and W_K differ, this is no longer the squared length of anything, and it can easily be small.
- **Asymmetry.** $s_{ij} = x_i W_Q W_K^\top x_j^\top$, while $s_{ji} = x_j W_Q W_K^\top x_i^\top$. These are equal only if $W_Q W_K^\top$ happens to be symmetric, so in general token i can find j relevant without j finding i relevant.
- **Separate content.** Values are a third projection, so what a token *advertises* (its key) can differ from what it *delivers* (its value).

5.5 What is learned and what is computed

Learned once during training (parameters)	Computed fresh for every input
W_Q, W_K, W_V	Q, K, V, S, A, Y

The **same** W_Q is applied to every token at every position. That's why the layer works for any sentence length n.

6. Worked examples

6.1 Fixing "flew" from Section 3

We reuse the toy 2-D vectors from Section 3 ($d_{model} = 2$) and keep $d_k = d_v = 2$:

$$X = \begin{bmatrix} 0.2 & 0.2 \\ 1 & 1 \\ 2 & 0 \end{bmatrix} \begin{matrix} \leftarrow \text{the} \\ \leftarrow \text{bat} \\ \leftarrow \text{flew} \end{matrix}$$

We choose these weight matrices (in a real model they are learned):

$$W_Q = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \qquad W_K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \qquad W_V = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}$$

W_Q **swaps** the two axes. Its meaning in our toy world: an action word (strong on the animal axis) goes *looking for* something on the other axis, such as an ambiguous noun like "bat".

Step 1: project. Each row of X times each matrix:

Token	x	$q = xW_Q$	$k = xW_K$	$v = xW_V = [x_1 + x_2,\ x_2]$
the	[0.2, 0.2]	[0.2, 0.2]	[0.2, 0.2]	[0.4, 0.2]
bat	[1, 1]	[1, 1]	[1, 1]	[2, 1]
flew	[2, 0]	**[0, 2]**	[2, 0]	[2, 0]

The key change: flew's **query** is $[0, 2]$, very different from flew's **key** $[2, 0]$.

Step 2: scores for "flew" (its query against every key):

- $q_{\text{flew}} \cdot k_{\text{the}} = 0(0.2) + 2(0.2) = 0.4$
- $q_{\text{flew}} \cdot k_{\text{bat}} = 0(1) + 2(1) = 2$
- $q_{\text{flew}} \cdot k_{\text{flew}} = 0(2) + 2(0) = \mathbf{0}$

Step 3: softmax. $e^{0.4} = 1.49$, $e^{2} = 7.39$, $e^{0} = 1$, total $= 9.88$:

"flew" attends to	the	bat	flew
Naive (Section 3)	0.02	0.12	**0.86**
With Q/K (now)	0.15	**0.75**	0.10

"flew" now spends 75% of its attention on "bat", the word it actually needs, and only 10% on itself.

Step 4: mix the values.

$$y_{\text{flew}} = 0.15\,[0.4, 0.2] + 0.75\,[2, 1] + 0.10\,[2, 0] = [1.76,\ 0.78]$$

(Using unrounded weights: $0.151, 0.748, 0.101$.)

The full matrices for all three tokens:

$S = QK^{\top}$	the	bat	flew
the	0.08	0.40	0.40
bat	0.40	2.00	2.00
flew	0.40	2.00	**0.00**

A	the	bat	flew
the	0.27	0.37	0.37
bat	0.09	0.45	0.45
flew	0.15	**0.75**	0.10

$Y = AV$	dim 1	dim 2
the	1.57	0.42
bat	1.85	0.47
flew	1.76	0.78

Note

In this particular example S still happens to be symmetric (e.g. $s_{\text{the,flew}} = s_{\text{flew,the}} = 0.4$), because $W_Q W_K^\top = W_Q$ is itself a symmetric matrix. This example fixes the *self-attention* problem; the next one shows *asymmetry*. In the demo below, change the top-right entry of W_Q to 2 and watch S become asymmetric.

Try it: change the weight matrices and see attention change

W_Q

0	1
1	0

W_K

1	0
0	1

W_V

1	0
1	1

$Q = XW_Q$

	d1	d2
the	0.20	0.20
bat	1.00	1.00
flew	0.00	2.00

$K = XW_K$

	d1	d2
the	0.20	0.20
bat	1.00	1.00
flew	2.00	0.00

$V = XW_V$

	d1	d2
the	0.40	0.20
bat	2.00	1.00
flew	2.00	0.00

$S = QK^T$

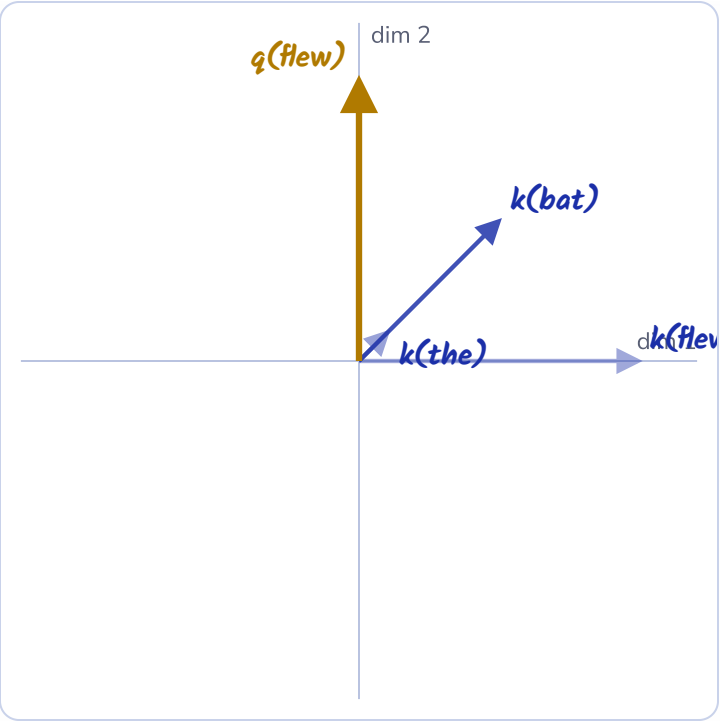
	the	bat	flew
the	0.08	0.40	0.40
bat	0.40	2.00	2.00
flew	0.40	2.00	0.00

$A = softmax(S)$

	the	bat	flew
the	0.27	0.37	0.37
bat	0.09	0.45	0.45
flew	0.15	0.75	0.10

$Y = AV$

	d1	d2
the	1.57	0.42
bat	1.85	0.47
flew	1.76	0.78



"flew"'s query against every key

the		0.15
bat		0.75
flew		0.10

q(flew) = [0.00, 2.00]

q·k(the) = [0.00, 2.00]·[0.20, 0.20] = **0.40**

q·k(bat) = [0.00, 2.00]·[1.00, 1.00] = **2.00**

q·k(flew) = [0.00, 2.00]·[2.00, 0.00] = **0.00**

output y(flew) = **[1.76, 0.78]**

"flew" gives 10% of its attention to itself and the most to "bat" (75%). The score matrix S is symmetric ($s_{ij} = s_{ji}$), so relevance is forced to be mutual.

6.2 Asymmetry in two lines

Take two tokens $a = [1, 0]$ and $b = [0, 1]$, with $W_K = I$ (identity) and $W_Q = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}$.

- $q_a = [1, 0]W_Q = [0, 1]$ and $q_b = [0, 1]W_Q = [0, 0]$
- $s_{ab} = q_a \cdot k_b = [0, 1] \cdot [0, 1] = \mathbf{1}$, so a is looking for b.
- $s_{ba} = q_b \cdot k_a = [0, 0] \cdot [1, 0] = \mathbf{0}$, so b is not looking for a.

Like a pronoun and its noun: "it" searches for "glass", but "glass" isn't searching for "it". With a single shared vector (Section 3), this is impossible. In the demo above, compare entry (flew, the) with entry (the, flew) in S after editing W_Q.

6.3 Shapes in a real model

BERT-base, one attention head, a 128-token input: $n = 128$, $d_{model} = 768$, $d_k = d_v = 64$.

Matrix	Shape
X	128×768
W_Q, W_K, W_V	768×64 each
Q, K, V	128×64 each
S, A	128×128
Y	128×64

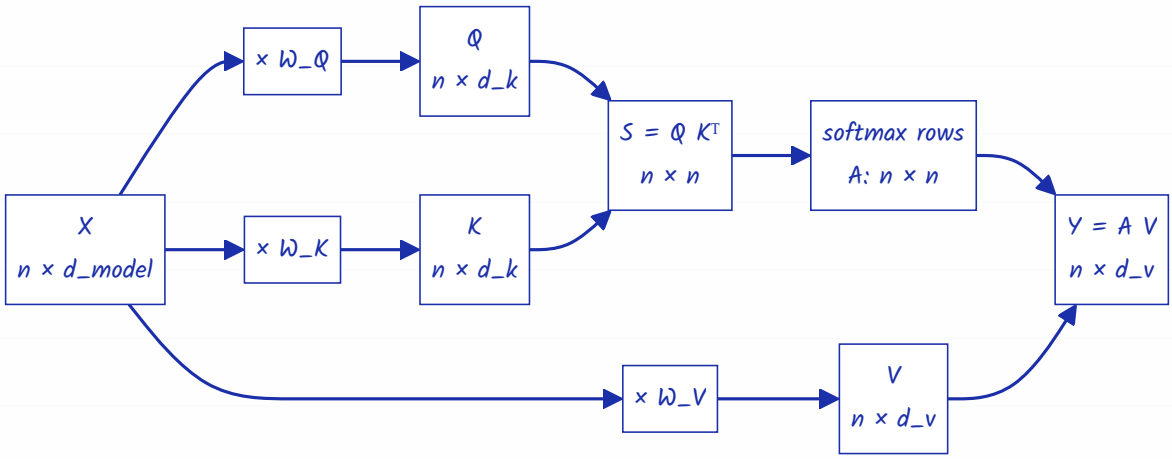
Parameters per head: $3 \times 768 \times 64 = 147{,}456$. Across all 12 heads that's $3 \times 768 \times 768 = 1{,}769{,}472$ per layer (biases ignored). For **LLaMA-2 7B** ($d_{model} = 4096$): $3 \times 4096^2 = 50{,}331{,}648$ per layer.

Try it: shapes and parameter counts for any model

What	Shape	Size
X (input)	`128 × 768`	`98,304 numbers`
W_Q, W_K, W_V (one head)	`768 × 64`	`147,456 parameters`
Q, K, V (one head)	`128 × 64 each`	`24,576 numbers`
S and A (one head)	`128 × 128`	`16,384 scores`
Y (one head)	`128 × 64`	`8,192 numbers`
All 12 heads: Q/K/V weights	`768 × 2304`	**`1,769,472 parameters per layer`**
All 12 heads: attention scores	`12 × 128 × 128`	`196,608 scores per layer`

$d_k = d_{model} / h = 768 / 12 =$ **64**. Notice the scores grow with n^2: doubling n to 256 makes S 65,536 entries (4×), while the weight matrices don't change at all.

7. Diagram



8. In practice

- **One big matrix.** Implementations often store W_Q, W_K and W_V side by side as one $d_{model} \times 3d_k$ matrix (one multiplication on the GPU, then split the result into three).
- **Biases.** The original Transformer and BERT add a bias vector ($xW_Q + b_Q$). Many modern LLMs (e.g. the LLaMA family) leave the biases out.
- **Many heads.** A layer has h sets of (W_Q, W_K, W_V), one per head, each with $d_k = d_{model}/h$ (Section 6).
- **Sharing keys and values.** To save memory at inference time, some models let several query heads share one set of keys and values: multi-query attention (MQA) and grouped-query attention (GQA), Sections 6 and 14.
- **Cross-attention.** In an encoder–decoder model, Q comes from the decoder's tokens while K and V come from the encoder's output (Section 10). The formulas are identical; only the source of X differs.

9. Common mistakes and misconceptions

"Q, K and V come from three different inputs." In *self*-attention all three are projections of the **same** X. Only in cross-attention do Q and K, V come from different sequences.

"Keys and values are the same thing." They come from the same tokens, but through **different** matrices. Keys decide *where* to look; values decide *what* is copied.

"Each token has its own W_Q." No. One W_Q (per head, per layer) is shared by all tokens and all positions.

"The attention weights are the values." The weights are A (how much to take); the values are V (what is taken). The output is AV.

"d_k must equal d_{model}." It can be any size; queries and keys just need to match each other. In practice $d_k = d_{model}/h$.

10. Summary

- Each token gets three learned projections: **query** (what I look for), **key** (how I'm found), **value** (what I hand over).
- $Q = XW_Q, K = XW_K, V = XW_V$; then $S = QK^\top, A = \mathrm{softmax}_{\mathrm{rows}}(S), Y = AV$.
- Separate projections remove forced self-attention and allow **asymmetric** relevance ($s_{ij} \neq s_{ji}$).
- The weights W_Q, W_K, W_V are **learned and shared** across positions; Q, K, V, A are **computed per input**.
- Shapes: Q, K are $n \times d_k$, V is $n \times d_v$, A is $n \times n$. Parameters per layer: $3 \cdot d_{model} \cdot d_k \cdot h$ (= $3d_{model}^2$ when $d_k = d_{model}/h$).

11. Check your understanding

1. A layer has $n = 10$ tokens, $d_{model} = 512$ and $d_k = d_v = 64$. Give the shapes of W_Q, Q, S and Y.
2. With X, W_K and W_V as in Section 6.1 but $W_Q = W_K = I$ (the identity), what are the attention weights of "flew"? Which section's result do you get back?
3. If $W_Q = W_K$, is the score matrix S symmetric? What does that mean for "it" and "glass"?

▶ **Show answers**

12. Further reading

1. Vaswani, A. et al. (2017). **"Attention Is All You Need."** *NeurIPS 2017.* arXiv:1706.03762. Section 3.2.1 defines attention with queries, keys and values.
2. Rush, A. (2018). **"The Annotated Transformer."** Harvard NLP. The original paper with line-by-line PyTorch code; see the attention function.
3. Alammar, J. **"The Illustrated Transformer"** (blog post, 2018). The section on self-attention in detail shows Q, K, V vectors visually.
4. Jurafsky, D., & Martin, J. H. ***Speech and Language Processing*** (3rd edition draft), the chapter on Transformers: the query/key/value formulation of self-attention.

Prepared by Akhil

Transformers & LLMs · Section 5

Scaled Dot-Product Attention

> **In one line:** the attention used in every Transformer is $\mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V$. This section explains the two pieces we haven't met yet: **dividing by $\sqrt{d_k}$** (to stop softmax from "freezing") and the **mask M** (to forbid certain connections).

1. Learning objectives
2. Prerequisites
3. Motivation
4. Intuition first
5. The formal idea
6. Worked examples
7. Diagram
8. In practice
9. Common mistakes
10. Summary
11. Check your understanding
12. Further reading

1. Learning objectives

By the end of this section you should be able to:

1. **Write** the full scaled dot-product attention formula and say what each symbol is.
2. **Explain**, using variance, why scores must be divided by $\sqrt{d_k}$, and what goes wrong if they aren't.
3. **Apply** a causal mask and a padding mask by hand, and explain why masked scores are set to $-\infty$.
4. **Compute** scaled, masked attention for a small example, step by step.
5. **Implement** attention in a few lines of NumPy, including the numerically stable softmax.

2. Prerequisites

- **Section 4**: $Q = XW_Q, K = XW_K, V = XW_V; S = QK^\top, A = \mathrm{softmax}_{\mathrm{rows}}(S), Y = AV$.
- **Variance**: if a quantity has variance σ^2, its typical size (standard deviation) is σ. Two facts:
  - For independent quantities, **variances add**: $\mathrm{Var}(a + b) = \mathrm{Var}(a) + \mathrm{Var}(b)$.
  - If q and k are independent with mean 0 and variance 1, then $\mathrm{Var}(q \cdot k) = 1$ for their product.

> ***Quick refresher: what $-\infty$ does in softmax***
>
> Since $e^{-\infty} = 0$, a score of $-\infty$ gets weight exactly 0, and the other weights still sum to 1. Example: scores $[1, -\infty, 1] \to e^1, 0, e^1 \to$ weights $[0.5, 0, 0.5]$.

3. Motivation: two problems left over

Section 4's attention, $\mathrm{softmax}(QK^\top)V$, works for our 2-D toy example. Real models break it in two ways.

Problem 1: big vectors make softmax freeze. Real heads use $d_k = 64$ or 128. A dot product adds up d_k products, so with more dimensions the scores get bigger: typically around ± 8 for $d_k = 64$, and ± 23 for $d_k = 512$. Softmax of such large numbers puts almost all the weight on one token, and its **gradient becomes nearly zero**, so the model stops learning.

Problem 2: some connections must be forbidden.

- A text *generator* must not peek at future words during training. If it could see the word it's supposed to predict, it would just copy it.
- Padding tokens (fillers added to make sequences in a batch the same length) carry no information and must be ignored.

The fixes: **divide the scores by $\sqrt{d_k}$**, and **add a mask M** before the softmax.

4. Intuition first

Scaling: turning down the volume

Imagine asking a room of people to vote by shouting. In a small room, the loudest voice wins by a little. In a stadium of 60,000, the combined noise is so loud that one section completely drowns out everyone else. Attention with large d_k is the stadium: sums of many products produce huge scores, and softmax hands everything to the loudest one.

Dividing by $\sqrt{d_k}$ is a **volume knob** that brings the typical score back to a normal size, so softmax can still express "mostly this one, but some of that one too".

Masking: blindfolds

A mask is a set of blindfolds put on before the vote. A **causal mask** blindfolds each token to every token *after* it, like an exam where you may only look at questions you have already answered. A **padding mask** blindfolds everyone to the filler tokens.

Where the analogy breaks

A blindfold sounds like "less information". Masking doesn't remove any token from the computation; it only sets certain *weights* to exactly zero. The masked tokens still compute their own outputs (their own rows).

5. The formal idea

5.1 The full formula

$$\boxed{\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V}$$

Symbol	Shape	Meaning
Q, K	$n \times d_k$	queries and keys (Section 4)
V	$n \times d_v$	values
$\sqrt{d_k}$	scalar	the scaling factor; d_k is the key/query width
M	$n \times n$	mask: 0 where attention is allowed, $-\infty$ where it is forbidden
output	$n \times d_v$	one new vector per token

5.2 The five steps

1. **Scores:** $S = QK^\top$
2. **Scale:** $S' = S/\sqrt{d_k}$
3. **Mask:** $S'' = S' + M$
4. **Softmax** each row: $A = \text{softmax}_{\text{rows}}(S'')$
5. **Mix the values:** $Y = AV$

5.3 Why exactly $\sqrt{d_k}$?

Assume each entry of a query q and a key k is independent, with mean 0 and variance 1 (roughly true at initialization). Then

$$q \cdot k = \sum_{i=1}^{d_k} q_i k_i$$

is a sum of d_k independent terms, each with variance 1. Variances add, so

$$\mathrm{Var}(q \cdot k) = d_k, \qquad \text{typical size} = \sqrt{d_k}.$$

Dividing by $\sqrt{d_k}$ gives

$$\mathrm{Var}\left(\frac{q \cdot k}{\sqrt{d_k}}\right) = \frac{d_k}{d_k} = 1.$$

In words: **whatever d_k is, the scaled scores have a typical size of about 1.** Dividing by d_k would over-shrink them as d_k grows, and not dividing lets them grow without limit; $\sqrt{d_k}$ is the amount that exactly cancels the growth.

5.4 Why large scores stop learning

For a two-way softmax with weights p and $1 - p$, the derivative of p with respect to its score is $p(1 - p)$.

Scores	Weights	Gradient $p(1-p)$
$[8, 0]$ (unscaled, $d_k = 64$)	$[0.9997, 0.0003]$	**0.0003**
$[1, 0]$ (same, ÷ 8 = $\sqrt{64}$)	$[0.73, 0.27]$	**0.197**

The unscaled version has a gradient about **590 times smaller**, so training barely moves the weights. This is called **softmax saturation**.

Try it: why $\sqrt{d_k}$? Watch softmax freeze

One query is compared with 6 keys. Every entry is a random number with mean 0 and variance 1, as in 5.3. The statistics on the right come from 2,000 fresh random draws at the chosen d_k.

Scores and softmax weights

key 1	0.229	score 0.09
key 2	0.169	score -0.21
key 3	0.399	score 0.65
key 4	0.040	score -1.66
key 5	0.040	score -1.65
key 6	0.123	score -0.53

Over 2,000 random draws

typical score size (std)	0.99
predicted by 5.3	1.00
average top weight	0.410
average gradient p(1−p)	0.2265

Learning signal (average gradient of the top weight)

Scaled: the typical score stays near 1 whatever d_k is, so softmax stays soft and the gradient stays healthy. Try $d_k = 1024$ and compare with “no scaling”.

5.5 Masks

M is added to the scores, entry by entry. Two common kinds, shown for 3 tokens:

Causal (look-ahead) mask: token i may attend to positions $j \le i$ only:

$$M_{\text{causal}} = \begin{bmatrix} 0 & -\infty & -\infty \\ 0 & 0 & -\infty \\ 0 & 0 & 0 \end{bmatrix}$$

Padding mask: if token 3 is padding, nobody may attend to it (column 3 blocked):

$$M_{\text{pad}} = \begin{bmatrix} 0 & 0 & -\infty \\ 0 & 0 & -\infty \\ 0 & 0 & -\infty \end{bmatrix}$$

Masks can be combined by adding them (any $-\infty$ wins).

Why $-\infty$ and not 0? A score of 0 still gets weight $e^0 = 1$, which is often a *large* weight. Only $-\infty$ guarantees weight exactly 0 **and** keeps every row summing to 1. (Setting weights to 0 *after* the softmax would break the "sums to 1" property.) In code, a very large negative number such as -10^9 is often used instead.

5.6 Numerically stable softmax

e^{1000} overflows a computer's floating-point numbers. Softmax gives the same answer if you subtract any constant from every score in a row, so implementations subtract the row maximum first:

$$\text{softmax}(s)_j = \frac{e^{s_j - \max(s)}}{\sum_k e^{s_k - \max(s)}}$$

Now the largest exponent is $e^0 = 1$ and nothing overflows.

6. Worked examples

We continue with Section 4's matrices ($d_k = 2$, so $\sqrt{d_k} = \sqrt{2} \approx 1.414$):

Token	q	k	v
the	[0.2, 0.2]	[0.2, 0.2]	[0.4, 0.2]
bat	[1, 1]	[1, 1]	[2, 1]
flew	[0, 2]	[2, 0]	[2, 0]

6.1 Scaling, step by step (no mask)

Step 1: scores $S = QK^\top$ (from Section 4):

S	the	bat	flew
the	0.08	0.40	0.40
bat	0.40	2.00	2.00
flew	0.40	2.00	0.00

Step 2: scale by $1/\sqrt{2}$:

$S/\sqrt{2}$	the	bat	flew
the	0.06	0.28	0.28
bat	0.28	1.41	1.41
flew	0.28	1.41	0.00

Step 3: softmax the "flew" row: $e^{0.28} = 1.33$, $e^{1.41} = 4.11$, $e^{0} = 1$, total 6.44 → weights $[0.21, 0.64, 0.16]$ (rounded).

All rows:

A	the	bat	flew
the	0.29	0.36	0.36
bat	0.14	0.43	0.43
flew	0.21	0.64	0.16

Step 4: output $Y = AV$:

Y	dim 1	dim 2
the	1.54	0.41
bat	1.78	0.46
flew	1.67	0.68

Compare with Section 4 (unscaled): "flew" went from $[0.15, 0.75, 0.10]$ to $[0.21, 0.64, 0.16]$. Scaling made the distribution **softer**. Here $d_k = 2$, so the

effect is mild; with $d_k = 64$ it's the difference between learning and not learning (Section 6.3).

6.2 Adding a causal mask

Same scaled scores, plus M_{causal}:

$S/\sqrt{2} + M$	the	bat	flew
the	0.06	$-\infty$	$-\infty$
bat	0.28	1.41	$-\infty$
flew	0.28	1.41	0.00

- **the** row: only one allowed entry → weights $[1, 0, 0]$. "the" can only see itself.
- **bat** row: $e^{0.28} = 1.33$, $e^{1.41} = 4.11$, total 5.44 → $[0.24, 0.76, 0]$.
- **flew** row: nothing masked → $[0.21, 0.64, 0.16]$, as before.

A_{causal}	the	bat	flew
the	**1.00**	0	0
bat	0.24	0.76	0
flew	0.21	0.64	0.16

The matrix is now **lower-triangular**: everything above the diagonal is zero. The output for "the" is simply its own value, $y_{\text{the}} = v_{\text{the}} = [0.4, 0.2]$.

Why this matters for training: in "the bat flew", the model can now be trained to predict "bat" from position 1 and "flew" from position 2 **at the same time, in one pass**. No position can see the answer it is meant to predict.

Try it: the five steps, one at a time

$S = QK^T$

	the	bat	flew
the	0.08	0.40	0.40
bat	0.40	2.00	2.00
flew	0.40		

Each entry is one query · one key. E.g. flew's query [0, 2] · bat's key [1, 1] = 2.00.

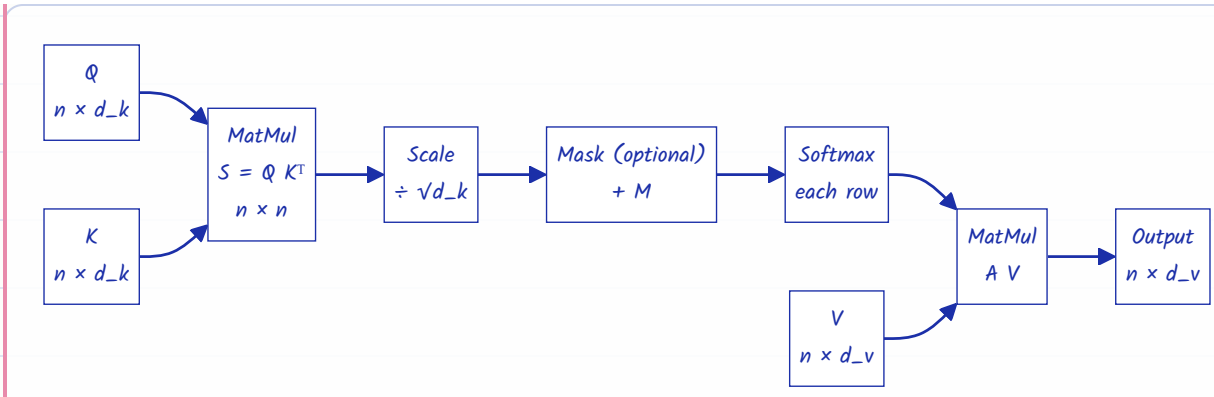
6.3 Why scaling matters at real sizes

Draw random queries and keys with independent standard-normal entries, 4,000 pairs for each d_k (simulation, seed 0):

d_k	variance of $q \cdot k$	typical size (std)	after ÷ $\sqrt{d_k}$
4	4.0	2.00	1.00
64	65.5	8.09	1.01
512	518.4	22.77	1.01

The variance tracks d_k closely, as the formula in 5.3 predicts, and scaling brings the typical size back to about 1 every time. The "why $\sqrt{d_k}$" demo in 5.4 runs the same experiment live.

7. Diagram



8. In practice

A complete implementation (NumPy)

```
import numpy as np

def attention(Q, K, V, mask=None):
    """Scaled dot-product attention.
    Q, K: (n, d_k)   V: (n, d_v)   mask: (n, n) booleans, True = allowed
    Returns the output (n, d_v) and the weights (n, n)."""
    d_k = Q.shape[-1]
    S = Q @ K.T / np.sqrt(d_k)                  # steps 1-2: scores, scaled
    if mask is not None:
        S = np.where(mask, S, -np.inf)          # step 3: blocked → -inf
    S = S - S.max(axis=-1, keepdims=True)       # stable softmax
    A = np.exp(S)
    A = A / A.sum(axis=-1, keepdims=True)       # step 4: rows sum to 1
    return A @ V, A                             # step 5

causal = np.tril(np.ones((3, 3), dtype=bool))  # lower-triangular = causal
```

In real libraries

- **PyTorch** has a built-in `torch.nn.functional.scaled_dot_product_attention(q, k, v, attn_mask=None, is_causal=False)`. It applies exactly this formula and automatically picks a fast kernel when one is available.

- **FlashAttention** computes the *same* result without ever storing the full $n \times n$ matrix, by working in small tiles that fit in the GPU's fast on-chip memory. It is much faster and uses memory proportional to n instead of n^2 (Section 14).

Cost

For one head: computing $QK^\top$ takes about $n^2 d_k$ multiply-adds, and AV another $n^2 d_v$. Storing A needs n^2 numbers: for $n = 1{,}000$ tokens at 2 bytes each, that's 2 MB per head per layer, and it grows 4× every time n doubles.

9. Common mistakes and misconceptions

"We divide by d_k." No: by $\sqrt{d_k}$, the standard deviation of the dot product, not its variance.

"Scaling changes which token gets the most attention." It doesn't. Dividing every score by the same positive number keeps their order, so the top token stays on top. It only changes **how sharp** the distribution is.

"Masking means setting attention weights to 0 after the softmax." If you zero weights after the softmax, the row no longer sums to 1. The mask is applied **before** the softmax, as $-\infty$ on the scores.

"Masked tokens are removed from the computation." They still have their own rows and outputs. They just can't be *attended to* by the tokens that are blocked from them.

"Encoders like BERT use the causal mask." Encoders are bidirectional and use only padding masks. The causal mask belongs to decoders that generate text (GPT, LLaMA, Claude).

10. Summary

- **Scaled dot-product attention:** $\mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V$, in five steps: scores → scale → mask → softmax → mix values.
- $q \cdot k$ has variance $\approx d_k$; dividing by $\sqrt{d_k}$ brings scores back to a typical size of about 1 and prevents **softmax saturation** (vanishing gradients).
- Scaling never changes which token ranks first; it controls how **sharp** the weights are.
- A **mask** adds $-\infty$ to forbidden scores: **causal** masks hide the future (decoders), **padding** masks hide filler tokens.
- Implementations use a **stable softmax** (subtract the row max) and fast kernels like **FlashAttention**.

11. Check your understanding

1. A head has $d_k = 256$. A query's raw scores against three keys are $[32, 16, 0]$. Compute the scaled scores and the attention weights.
2. In a 4-token sentence with a causal mask, how many of the 16 entries of A are forced to be 0? Which token can attend to the most tokens?
3. A student masks by setting forbidden **scores** to 0 instead of $-\infty$. Using the "bat" row of 6.2 (scaled scores $[0.28, 1.41, 1.41]$, with "flew" forbidden), show what goes wrong.

▶ **Show answers**

12. Further reading

1. Vaswani, A. et al. (2017). **"Attention Is All You Need."** *NeurIPS 2017.* arXiv:1706.03762. Section 3.2.1 and its footnote give the variance argument for $\sqrt{d_k}$.
2. Dao, T. et al. (2022). **"FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness."** arXiv:2205.14135. How the same formula is

computed without storing the $n \times n$ matrix.

3. PyTorch documentation, `torch.nn.functional.scaled_dot_product_attention`. The library version, including `attn_mask` and `is_causal`.
4. Goodfellow, I., Bengio, Y., & Courville, A. (2016). ***Deep Learning***, MIT Press. Chapter 4 (numerical computation: overflow and the stable softmax) and Chapter 6 (softmax units and saturation).

Prepared by Akhil

Multi-Head Attention

> **In one line:** instead of one attention operation, a Transformer layer runs h **smaller attentions ("heads") side by side**, each with its own W_Q, W_K, W_V, then glues their outputs together and mixes them with one more matrix W_O. Each head can look for a different kind of relationship.

1. Learning objectives
2. Prerequisites
3. Motivation
4. Intuition first
5. The formal idea
6. Worked examples
7. Diagram
8. In practice
9. Common mistakes
10. Summary
11. Check your understanding
12. Further reading

1. Learning objectives

By the end of this section you should be able to:

1. **Explain** why a single attention head is limited, and what multiple heads add.
2. **Write** the multi-head attention formula, including the concatenation and the output projection W_O.
3. **Trace** every shape through multi-head attention, including the "split into heads" reshape.
4. **Compute** a small two-head example by hand.
5. **Count** parameters and compare MHA with its memory-saving variants, MQA and GQA.

2. Prerequisites

- **Section 5**: $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) V$.
- **Section 4**: $Q = XW_Q, K = XW_K, V = XW_V$, with W_Q of shape $d_{model} \times d_k$.
- **Concatenation**: placing matrices side by side. Two 3×1 matrices concatenated along columns give a 3×2 matrix: $[\, a \mid b \,]$.

> ***Quick refresher: one weighted average = one choice***
>
> In Section 5, each token's output is *one* weighted average of the values, using *one* set of weights that sum to 1. If the weights favour token A, they must take weight away from token B. A single head can't say "100% A for one purpose and 100% B for another" at the same time.

3. Motivation: one head has to compromise

Take the word **"it"** in

> *The phone didn't fit in the pocket because **it** was too small.*

To understand "it", the model needs several things **at once**:

- **Which noun** "it" refers to (the pocket), a coreference relationship.
- **What is being said about it** ("was too small"), a nearby-words relationship.
- **Grammar**: "it" is the subject of "was".

A single head produces a single weighted average. If it puts most of its weight on "pocket", it has little left for "small" and "was". It must **compromise**, blending everything into one muddled mixture.

The fix: run **several heads in parallel**. Each head has its own projections, so each can learn to look for something different, and the layer combines all their findings.

4. Intuition first: a study group with coloured highlighters

Four friends prepare for an exam from the same textbook page. Each uses a different coloured highlighter:

- **Yellow** marks definitions.
- **Green** marks formulas.
- **Pink** marks examples.
- **Blue** marks dates and names.

Every friend reads the **whole page**, but each pays attention to different parts. At the end they **combine their highlights** into one set of notes that is richer than any single friend's.

That is multi-head attention:

- Each **friend** = one **head**.
- Each friend's **highlighter rule** = that head's own W_Q, W_K, W_V.
- **Combining the highlights** = concatenating the head outputs and mixing them with W_O.

Where the analogy breaks

- Nobody assigns the heads their jobs. Each head's "highlighter rule" is **learned**, and many heads end up with jobs humans can't easily name (or with no useful job at all).
- Friends read at different speeds; heads run **truly in parallel**, as one batched matrix operation.

5. The formal idea

5.1 The formula

For heads $i = 1, \ldots, h$, each with its own weight matrices:

$$\text{head}_i = \text{Attention}\Big(XW_Q^{(i)},\ XW_K^{(i)},\ XW_V^{(i)}\Big)$$

Then concatenate and project:

$$\text{MultiHead}(X) = \big[\,\text{head}_1 \,\big|\, \text{head}_2 \,\big|\, \cdots \,\big|\, \text{head}_h\,\big]\, W_O$$

In words: *run h independent attentions, stack their outputs side by side, then mix them with one learned matrix.*

5.2 Shapes

The standard choice is to split the width evenly: $d_k = d_v = d_{model}/h$.

Matrix	Shape	Count
X	$n \times d_{model}$	1
$W_Q^{(i)}, W_K^{(i)}, W_V^{(i)}$	$d_{model} \times d_k$	h of each
head_i	$n \times d_k$	h
Concatenation	$n \times (h \cdot d_k) = n \times d_{model}$	1
W_O	$d_{model} \times d_{model}$	1
Output	$n \times d_{model}$	1

The output has the **same shape as the input**, which is exactly what a Transformer block needs (Section 1).

5.3 Why W_O?

Without W_O, the output would just be the heads' results placed side by side: dimensions 1 to d_k would come only from head 1, the next d_k only from head 2, and so on. W_O lets **every output dimension draw on every head**, so the layer can combine findings ("head 3 found the noun, head 7 found the adjective") into one representation.

5.4 Cost: more heads, same price

Splitting d_{model} into h heads of width d_{model}/h means:

- **Parameters:** W_Q, W_K, W_V for all heads together are $3 \times d_{model} \times d_{model}$, plus W_O at $d_{model} \times d_{model}$, so $4\,d_{model}^2$ **per layer** (ignoring biases), whatever h is.
- **Compute:** about the same as one big head of width d_{model}.
- **What does grow with h:** the number of $n \times n$ attention matrices, $h \cdot n^2$ scores in total.

So multiple heads are almost a **free upgrade**: same parameters, similar compute, but h different attention patterns instead of one.

5.5 How it is really computed: one big matrix, then reshape

In practice nobody multiplies by h separate small matrices. Implementations:

1. Multiply once by a big W_Q of shape $d_{model} \times d_{model}$ (the h small ones side by side) → Q is $n \times d_{model}$.
2. **Reshape** Q to $h \times n \times d_k$: split its columns into h groups of d_k.
3. Run attention on all heads at once as a batched operation → $h \times n \times d_k$.
4. Reshape back to $n \times d_{model}$ (this *is* the concatenation) and multiply by W_O.

5.6 Saving memory: MQA and GQA

During text generation, models store every past token's keys and values (the **KV cache**, Section 14). With standard multi-head attention (**MHA**), every head has its own keys and values. Two popular variants share them:

Variant	Query heads	Key/value heads	KV cache vs MHA
MHA (multi-head)	h	h	1×
GQA (grouped-query)	h	g (e.g. 8)	g/h
MQA (multi-query)	h	1	$1/h$

In GQA, each group of h/g query heads shares one key head and one value head. Quality stays close to MHA, while the cache shrinks a lot. Most recent LLMs use GQA.

6. Worked examples

6.1 Two heads on "the bat flew"

We reuse the toy 2-D vectors (axis 1 = *animal-ness*, axis 2 = *sport-ness*), with $d_{model} = 2$ and $h = 2$ heads, so $d_k = d_v = 1$:

Token	x
the	[0.2, 0.2]
bat	[1, 1]
flew	[2, 0]

To keep the arithmetic short, each head simply **reads one axis** (in a real model these weights are learned, and $W_Q \neq W_K$ as in Section 4):

- **Head 1, the "animal" head:** $W_Q^{(1)} = W_K^{(1)} = W_V^{(1)} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$, so $q = k = v = x_1$.
- **Head 2, the "sport" head:** $W_Q^{(2)} = W_K^{(2)} = W_V^{(2)} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$, so $q = k = v = x_2$.
- $W_O = I$ (the identity), so the output is just the concatenation.

Since $d_k = 1$, the scaling factor is $\sqrt{1} = 1$.

Head 1 for "bat". $q = 1$; keys (animal values) $= [0.2, 1, 2]$.

- Scores $= [0.2, 1, 2]$
- $e^{0.2} = 1.22$, $e^1 = 2.72$, $e^2 = 7.39$, total 11.33 → weights $[0.11, 0.24, 0.65]$
- Output $= 0.11(0.2) + 0.24(1) + 0.65(2) = \mathbf{1.57}$

Head 2 for "bat". $q = 1$; keys (sport values) $= [0.2, 1, 0]$.

- Scores $= [0.2, 1, 0]$
- $e^{0.2} = 1.22$, $e^1 = 2.72$, $e^0 = 1$, total 4.94 → weights $[0.25, 0.55, 0.20]$
- Output $= 0.25(0.2) + 0.55(1) + 0.20(0) = \mathbf{0.60}$

Concatenate (and $W_O = I$): $y_{\text{bat}} = [1.57,\ 0.60]$.

The two heads looked at the **same sentence differently**: head 1 sent 65% of its attention to "flew", while head 2 kept 55% on "bat" itself.

All three tokens:

Token	Head 1 weights (the, bat, flew)	Head 2 weights (the, bat, flew)	Output $[\text{head}_1 \mid \text{head}_2]$
the	0.28, 0.33, 0.40	0.32, 0.37, 0.31	[1.18, 0.44]
bat	0.11, 0.24, **0.65**	0.25, **0.55**, 0.20	[1.57, 0.60]
flew	0.02, 0.12, **0.86**	**0.33, 0.33, 0.33**	[1.84, 0.40]

> ***Something to notice***
>
> For "flew", head 2's query is 0 (flew has no sport-ness), so all its scores are 0 and it spreads attention **evenly**. That's a head with "nothing to look for". Real models have this problem too, and trained heads often handle it by attending to a harmless token such as the first token or punctuation.

6.2 Compared with one head

A single head with $d_k = 2$ and identity projections (scaled by $\sqrt{2}$) gives "bat" **one** set of weights, $[0.14, 0.43, 0.43]$, and output $[1.32, 0.46]$. Both dimensions are forced to use the *same* weights. With two heads, the animal dimension and the sport dimension each get their own weights. This is the "no compromise" benefit from Section 3.

Try it: two heads, two different views

Head 1, "animal" (reads axis 1)

the	0.11
bat	0.24
flew	0.65

q(bat) = 1.00; keys = [0.20, 1.00, 2.00]
scores = [0.20, 1.00, 2.00] → output =
0.11·0.20 + 0.24·1.00 + 0.65·2.00 = **1.57**

	the	bat	flew
the	0.28	0.33	0.40
bat	0.11	0.24	0.65
flew	0.02	0.12	0.86

Head 2, "sport" (reads axis 2)

the	0.25
bat	0.55
flew	0.20

q(bat) = 1.00; keys = [0.20, 1.00, 0.00]
scores = [0.20, 1.00, 0.00] → output =
0.25·0.20 + 0.55·1.00 + 0.20·0.00 = **0.60**

	the	bat	flew
the	0.32	0.37	0.31
bat	0.25	0.55	0.20
flew	0.33	0.33	0.33

concat [1.57, 0.60] × W_O $\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$ = [1.57, 0.60]

W_O = identity | W_O = mix 50/50

One head instead (d_k = 2, identity projections, ÷√2): "bat" gets one set of weights [0.14, 0.43, 0.43] for both dimensions and output [1.32, 0.46].

Head 1 puts the most weight on "flew", head 2 on "bat". With W_o = identity, the output is just the two heads' results side by side.

6.3 Shape trace for BERT-base

$n = 128$ tokens, $d_{model} = 768$, $h = 12$ heads, $d_k = 768/12 = 64$.

Step	Shape
X	128×768
$Q = XW_Q$ (all heads at once)	128×768
Split into heads	$12 \times 128 \times 64$
Scores $QK^\top$ per head	$12 \times 128 \times 128$
Head outputs	$12 \times 128 \times 64$
Concatenate	128×768
$\times W_O$ (768×768)	128×768

Parameters per layer: $4 \times 768^2 = 2{,}359{,}296$. For LLaMA-2 7B ($d_{model} = 4096$, 32 heads, $d_k = 128$): $4 \times 4096^2 = 67{,}108{,}864$ per layer.

6.4 How much GQA saves

The KV cache stores, per token per layer, $2 \times (\text{KV heads}) \times d_k$ numbers (keys + values):

Model	Query heads	KV heads	Per token per layer	Saving vs MHA
LLaMA-2 7B (MHA)	32	32	$2 \times 32 \times 128 = 8{,}192$	1×
Mistral 7B (GQA)	32	8	$2 \times 8 \times 128 = 2{,}048$	4× smaller
LLaMA-2 70B (GQA)	64	8	$2 \times 8 \times 128 = 2{,}048$	8× smaller

Try it: split into heads, then share keys and values

Query heads (each block is one head's slice of the d_{model} columns; colour = which key/value head it uses)



32 query heads × d_k = 128 columns each = 4,096 = d_model

Key/value heads

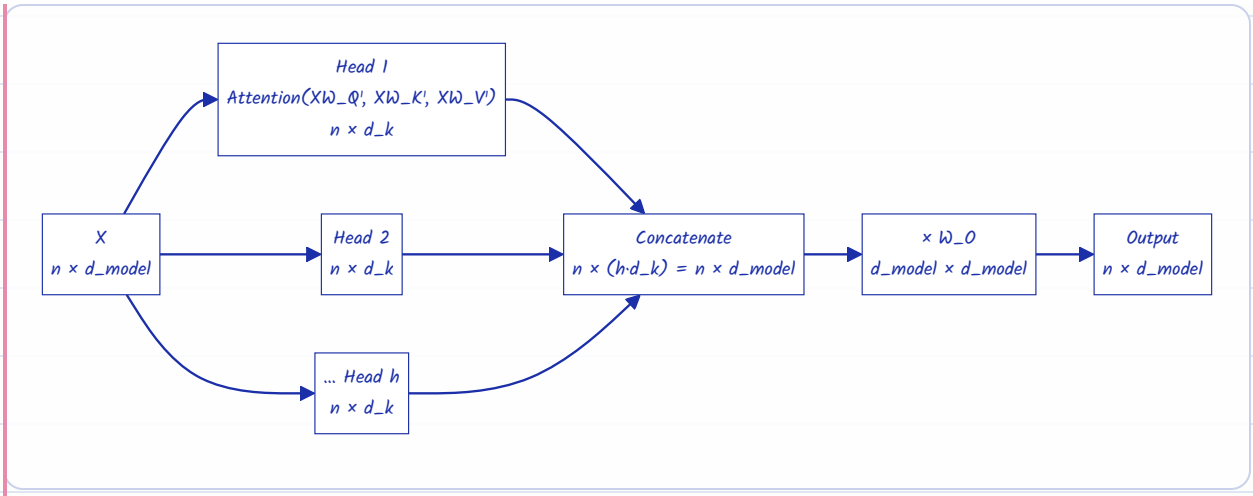


32 key/value heads, each shared by 1 query head

Quantity	Value
d_k = d_model / h	`4,096 / 32 = 128`
Attention weights per layer (W_Q, W_O: d_model × d_model; W_K, W_V: d_model × KV heads·d_k)	`2×4,096² + 2×4,096×4,096 = 67,108,864`
KV cache per token per layer	`2 × 32 × 128 = 8,192 numbers`
KV cache per token, all 32 layers (FP16)	`524,288 bytes ≈ 0.500 MB`
KV cache for a 4,096-token context	`2.00 GB`
Saving vs MHA (32 KV heads)	`1× smaller`

MHA: every query head has its own keys and values. Slide the key/value heads down to see GQA and MQA.

7. Diagram



8. In practice

Typical head counts

Model	d_{model}	Heads h	d_k
Original Transformer (base)	512	8	64
BERT-base / GPT-2 small	768	12	64
LLaMA-2 7B	4096	32	128
LLaMA-2 70B	8192	64 (8 KV heads)	128

d_k is almost always **64 or 128**. Bigger models add more heads rather than making each head wider.

What heads learn

Researchers who inspect trained models find heads that track the previous token, syntactic relations, coreference, rare words, or copying patterns (induction heads, Section 3). They also find that **many heads can be removed after training with little loss in accuracy**: a few specialised heads do most of the work.

PyTorch

`torch.nn.MultiheadAttention(embed_dim=768, num_heads=12)` implements this whole section, including W_O. Internally it uses the reshape trick from 5.5.

9. Common mistakes and misconceptions

"Each head sees only part of the input." Every head sees the **full** d_{model}-wide input. Each head *projects* it into its own smaller d_k-dimensional space.

"More heads means more parameters and compute." With $d_k = d_{model}/h$, the parameter count stays $4d_{model}^2$ and compute stays about the same. Only the number of attention maps grows.

"Heads are assigned roles like 'syntax head'." Nothing assigns roles. Any specialisation **emerges** from training, and many heads don't have a clean, nameable role.

"Concatenation is the final step." It's followed by W_O, which mixes information across heads. Without it, each output dimension would only ever come from one head.

"GQA means fewer query heads." GQA keeps all the **query** heads and shares **key/value** heads between groups of them.

10. Summary

- **Multi-head attention** runs h attention heads in parallel, each with its own $W_Q^{(i)}, W_K^{(i)}, W_V^{(i)}$, so each can focus on a different relationship.
- $\text{MultiHead}(X) = [\text{head}_1 | \cdots | \text{head}_h]\, W_O$; with $d_k = d_{model}/h$ the output is $n \times d_{model}$, the same as the input.
- Same parameters ($4d_{model}^2$ per layer) and similar compute as one big head, but h attention patterns.

- Implemented as one big matrix multiply plus a **reshape** to $h \times n \times d_k$.
- **MQA/GQA** share key/value heads to shrink the KV cache; most recent LLMs use GQA.

11. Check your understanding

1. A model has $d_{model} = 1024$ and $h = 16$ heads. What is d_k? How many attention parameters (Q, K, V and O, no biases) does one layer have?
2. Using 6.1, what would "bat"'s output be if $W_O = \begin{bmatrix} 0.5 & 0.5 \\ 0.5 & 0.5 \end{bmatrix}$ instead of the identity? What has W_O done?
3. A model has 32 query heads and uses GQA with 4 key/value heads. By what factor is its KV cache smaller than MHA's? How many query heads share each key/value head?

▶ **Show answers**

12. Further reading

1. Vaswani, A. et al. (2017). **"Attention Is All You Need."** *NeurIPS 2017.* arXiv:1706.03762. Section 3.2.2 introduces multi-head attention.
2. Michel, P., Levy, O., & Neubig, G. (2019). **"Are Sixteen Heads Really Better than One?"** *NeurIPS 2019.* arXiv:1905.10650. Many heads can be pruned after training.
3. Voita, E. et al. (2019). **"Analyzing Multi-Head Self-Attention: Specialized Heads Do the Heavy Lifting, the Rest Can Be Pruned."** *ACL 2019.* arXiv:1905.09418.
4. Shazeer, N. (2019). **"Fast Transformer Decoding: One Write-Head is All You Need."** arXiv:1911.02150. Introduces multi-query attention.
5. Ainslie, J. et al. (2023). **"GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints."** arXiv:2305.13245.

Positional Information

> **In one line:** self-attention can't tell word order apart, so Transformers inject position information, either by **adding a position vector** to each token's embedding (learned or sinusoidal) or by building position directly into the attention scores (**RoPE**, **ALiBi**). Modern LLMs mostly use RoPE.

1. Learning objectives
2. Prerequisites
3. Motivation
4. Intuition first
5. The formal idea
6. Worked examples
7. Diagram
8. In practice
9. Common mistakes
10. Summary
11. Check your understanding
12. Further reading

1. Learning objectives

By the end of this section you should be able to:

1. **Explain** why self-attention needs position information, using the "shuffle" result from Section 3.
2. **Compute** sinusoidal positional encodings by hand and add them to token embeddings.
3. **Describe** learned absolute positions and their maximum-length limit.
4. **Show** with a small example that RoPE makes attention scores depend only on **relative** position.

5. **Compare** the main schemes (learned, sinusoidal, RoPE, ALiBi) and say which real models use each.

2. Prerequisites

- **Section 3**: shuffling the input tokens only shuffles the outputs; each token's output vector stays exactly the same.
- **Section 2**: X is the matrix of token embeddings, one row per token.
- **Section 5**: attention scores $q \cdot k / \sqrt{d_k}$, and adding a bias to scores before the softmax (as the mask did).

> ***Quick refresher: sine, cosine and rotation***
>
> The point $(\cos\phi, \sin\phi)$ lies on a circle of radius 1 at angle ϕ. Rotating a 2-D vector $[x_1, x_2]$ by angle ϕ gives
>
> $$[\, x_1 \cos\phi - x_2 \sin\phi, \quad x_1 \sin\phi + x_2 \cos\phi \,]$$
>
> Rotation changes direction but not length. The dot product of two vectors depends only on their lengths and the **angle between them**.

3. Motivation: "dog bites man" = "man bites dog"?

Section 3 showed that self-attention is **order-blind**: the score between two tokens depends only on their vectors, never on where they sit. So for a Transformer without position information:

- *"dog bites man"* and *"man bites dog"* give each word the **same** output vector.
- *"not good, actually bad"* and *"not bad, actually good"* look identical.

Word order carries a lot of meaning, so the model needs to know **where** each token is. There are two broad strategies:

1. **Tell each token its absolute position**: add a "position vector" to its embedding ("I am word number 5").
2. **Tell attention the relative distance**: adjust each score according to how far apart the two tokens are ("you are 3 words before me").

4. Intuition first: seat numbers and clocks

Absolute positions: seat numbers. At an exam, every student's answer sheet has their **seat number** stamped on it. The examiner now knows not just *what* each student wrote but *where* they sat. Adding a position vector to each token is that stamp.

Sinusoidal positions: a set of clocks. How do you stamp a position using numbers that stay small? Use a wall of clocks running at different speeds: a seconds hand, a minutes hand, an hours hand, a days hand. Read together, the hands identify any moment uniquely, and every hand stays between −1 and 1 however much time passes. Sinusoidal encodings give each position the readings of many "clocks" ticking at different speeds.

Relative positions: "how far apart?" When you read *"it"*, you care that the noun was "two words ago", not that it was "word 1,847 of the book". Relative schemes such as RoPE and ALiBi give attention exactly this: the **distance** between tokens.

Where the analogy breaks

A seat number is a label someone reads. The model never "reads" a position; position just changes the numbers it computes with, and the model has to *learn* how to use those changes.

5. The formal idea

5.1 Absolute positions, added to the embeddings

For the token at position pos (counting from 0):

$$x'_{pos} = x_{pos} + p_{pos}$$

where p_{pos} is a d_{model}-long **position vector**. Now the same word at different positions enters the model as different vectors.

Option A, learned positions. Keep a trainable table P of shape $L_{max} \times d_{model}$ (one row per position) and learn it like the embedding table. Used in BERT ($L_{max} = 512$) and GPT-2 ($L_{max} = 1024$).

- *Pro:* simple and flexible.
- *Con:* there is **no row for position L_{max} or beyond**, so the model can't handle longer inputs at all.

Option B, sinusoidal positions (the original 2017 Transformer). No parameters; use fixed waves:

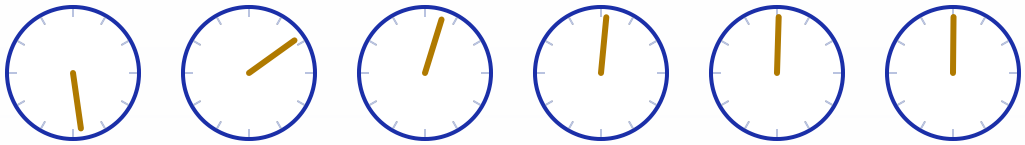
$$p_{pos,\,2i} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right), \qquad p_{pos,\,2i+1} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

In words: dimensions come in **(sin, cos) pairs**. Pair i is a clock hand that turns at speed $\omega_i = 1/10000^{2i/d_{model}}$ radians per position. The first pair turns fast (1 radian per step); the last pairs turn very slowly.

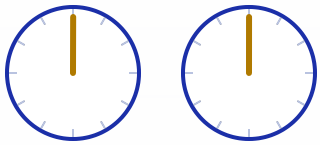
- Every value stays between −1 and 1.
- Each position gets a unique pattern.
- Moving forward k positions **rotates** each (sin, cos) pair by the same angle $k\omega_i$, whatever the starting position. This makes relative offsets easy for the model to detect, and it is the idea RoPE builds on.

Try it: positions as a wall of clocks

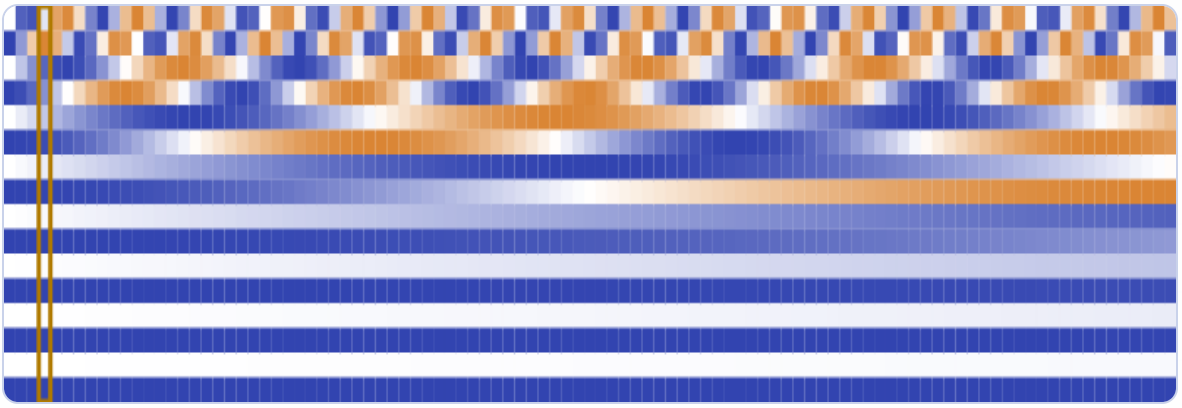
Each clock is one (sin, cos) pair. The hand's angle is position × ω_i.



pair 0 ω=1.000 (0.14, −0.99) | pair 1 ω=0.316 (0.81, 0.58) | pair 2 ω=0.100 (0.30, 0.96) | pair 3 ω=0.032 (0.09, 1.00) | pair 4 ω=0.010 (0.03, 1.00) | pair 5 ω=3.2e-3 (0.01, 1.00)



pair 6 ω=1.0e-3 (0.00, 1.00) | pair 7 ω=3.2e-4 (0.00, 1.00)



Heat map: position 0 to 100 across, dimension 0 to d_{model}−1 down. Blue = +1, orange = −1. The gold line marks the chosen position. Fast pairs (top) stripe quickly; slow pairs (bottom) barely change.

Position 3 → first entries [0.141, −0.990, 0.813, 0.583] .... The fast clock (pair 0) has turned 3 rad; the slowest shown has turned only 0.0009 rad. Together they give every position a unique fingerprint.

5.2 RoPE: rotate the queries and keys

Rotary Position Embedding (RoPE) doesn't add anything to x. Instead, inside every attention layer it **rotates** each query and key according to its position, just before the dot product:

- Split q and k into pairs of dimensions.

- Rotate pair i of the query at position m by angle $m\,\theta_i$, and pair i of the key at position n by angle $n\,\theta_i$, where $\theta_i = 10000^{-2i/d_k}$.

The key fact: rotating q by $m\theta$ and k by $n\theta$ changes the angle *between* them by $(m-n)\theta$. So

$$\text{score}(q \text{ at } m,\ k \text{ at } n) \ \text{ depends only on } \ q,\ k \ \text{ and } \ (m-n).$$

Attention automatically sees **relative distance**. Shifting a whole phrase further along the text doesn't change its internal scores.

RoPE is used by LLaMA, Mistral, Qwen, Gemma and most open LLMs today.

5.3 ALiBi: a distance penalty on the scores

ALiBi (Attention with Linear Biases) adds no vectors at all. It subtracts a penalty proportional to distance from each score:

$$s_{ij} = \frac{q_i \cdot k_j}{\sqrt{d_k}} - m \cdot (i-j) \qquad (j \le i)$$

where m is a fixed slope (each head gets a different slope). Far-away tokens get a bigger penalty, so heads prefer nearby context, some strongly and some weakly. Used in BLOOM and MPT.

5.4 Comparison

Scheme	Where it acts	Parameters	Relative?	Beyond training length?	Used in
Learned absolute	added to x	$L_{max} \times d_{model}$	no	**no** (no row exists)	BERT, GPT-2
Sinusoidal	added to x	0	indirectly	defined, but works poorly in practice	Original Transformer
RoPE	rotates q, k in every layer	0	**yes**	with tricks (5.5)	LLaMA, Mistral, Qwen
ALiBi	bias on scores	0	**yes**	fairly well	BLOOM, MPT

5.5 Stretching the context window (brief)

A model trained on 4,096-token texts sees RoPE angles only up to $4096\,\theta_i$. To use longer contexts, practitioners:

- **Position interpolation:** squeeze the positions (e.g. divide by 4) so a 16k-token text uses the angles seen during training, then fine-tune briefly.
- **Change the base:** use a larger base than 10,000, so the clocks turn more slowly (e.g. LLaMA-3 uses 500,000).
- **NTK-aware scaling / YaRN:** smarter mixtures of the two, treating fast and slow pairs differently.

6. Worked examples

6.1 Sinusoidal encodings for $d_{model} = 4$

With $d_{model} = 4$ there are two (sin, cos) pairs:

- Pair 0: $\omega_0 = 1/10000^0 = 1$, so $(\sin pos, \cos pos)$.
- Pair 1: $\omega_1 = 1/10000^{2/4} = 1/100 = 0.01$, so $(\sin 0.01pos, \cos 0.01pos)$.

pos	$\sin(pos)$	$\cos(pos)$	$\sin(0.01\,pos)$	$\cos(0.01\,pos)$
0	0.000	1.000	0.000	1.000
1	0.841	0.540	0.010	1.000
2	0.909	−0.416	0.020	1.000
3	0.141	−0.990	0.030	1.000

The first pair changes a lot from one position to the next (the fast hand); the second barely moves (the slow hand). The slow pairs matter for telling apart positions that are far apart.

6.2 Order finally matters

Use our toy 2-D vectors and $d_{model} = 2$, so $p_{pos} = [\sin pos,\ \cos pos]$:

- $p_0 = [0.000, 1.000], p_1 = [0.841, 0.540], p_2 = [0.909, -0.416]$

"the bat flew" with positions added:

Token (pos)	x	$+\,p_{pos}$	x'
the (0)	[0.2, 0.2]	[0.000, 1.000]	[0.20, 1.20]
bat (1)	[1, 1]	[0.841, 0.540]	[1.84, 1.54]
flew (2)	[2, 0]	[0.909, −0.416]	[2.91, −0.42]

"flew bat the": now "flew" is at position 0 and "the" at position 2, so they get *different* additions: flew → [2.00, 1.00], the → [1.11, −0.22].

Running the naive self-attention from Section 3 on both orders, "bat"'s output is:

Order	Without positions	With sinusoidal positions
the bat flew	[1.38, 0.47]	[2.08, 1.04]
flew bat the	[1.38, 0.47]	[1.89, 1.32]

Without positions the outputs are identical, as Section 3 found. **With positions** they differ: the model can now tell the two orders apart.

Try it: does word order change "bat"?

"the bat flew"

token (pos)	x	input
the (0)	0.20, 0.20	[0.20, 0.20]
bat (1)	1.00, 1.00	[1.00, 1.00]
flew (2)	2.00, 0.00	[2.00, 0.00]

bat's output: **[1.38, 0.47]**

"flew bat the"

token (pos)	x	input
flew (0)	2.00, 0.00	[2.00, 0.00]
bat (1)	1.00, 1.00	[1.00, 1.00]
the (2)	0.20, 0.20	[0.20, 0.20]

bat's output: **[1.38, 0.47]**

Same output in both orders: the model can't tell them apart.

6.3 RoPE depends only on the distance

Take 2-D queries and keys and a rotation of $\theta = 30^\circ$ per position.

Simple case: $q = k = [1, 0]$.

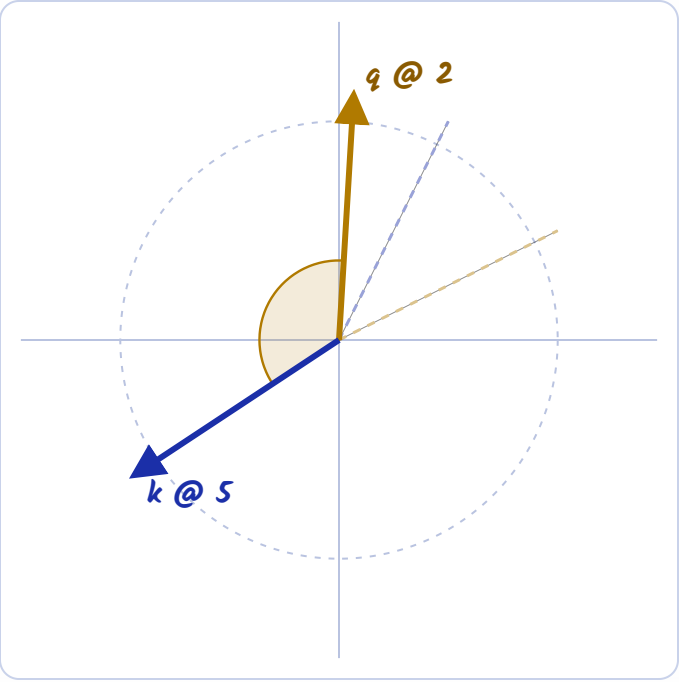
- Query at $m = 1$ → rotated by 30°; key at $n = 3$ → rotated by 90°. Angle between them $= 60^\circ$, so the score is $\cos 60^\circ = \mathbf{0.5}$.
- Query at $m = 4$ (120°), key at $n = 6$ (180°). Angle between them is again 60°, so the score is $\mathbf{0.5}$.
- Query at $m = 3$, key at $n = 3$: angle 0°, so the score is $\mathbf{1}$.

A general case: $q = [1, 0.5], k = [0.5, 1]$.

- Rotating q to position $m = 2$ (60°): $[1\cos 60^\circ - 0.5\sin 60^\circ,\ 1\sin 60^\circ + 0.5\cos 60^\circ] = [0.07, 1.12]$.
- Doing the same for k at $n = 5$ (150°) and taking the dot product gives $\mathbf{-0.75}$.
- Moving both 5 positions later ($m = 7, n = 10$) gives **exactly** -0.75 again.

The score changed only when the **gap** $m - n$ changed. That's the property that makes RoPE a relative scheme.

Try it: RoPE rotates by position; only the gap matters



score = −0.75

q = [1, 0.5] rotated by 2×30° = 60° → [0.07, 1.12]
k = [0.5, 1] rotated by 5×30° = 150° → [−0.93, −0.62]
gap m − n = **-3**; q·k = −0.750

Scores you have produced

```
1. m=2, n=5 (gap -3) → -0.750
```

Dashed arrows: q and k before rotation. Solid: after rotating by their positions. Press “Shift both by +1”: both arrows turn by the same angle, the gap stays -3, and the score doesn’t change.

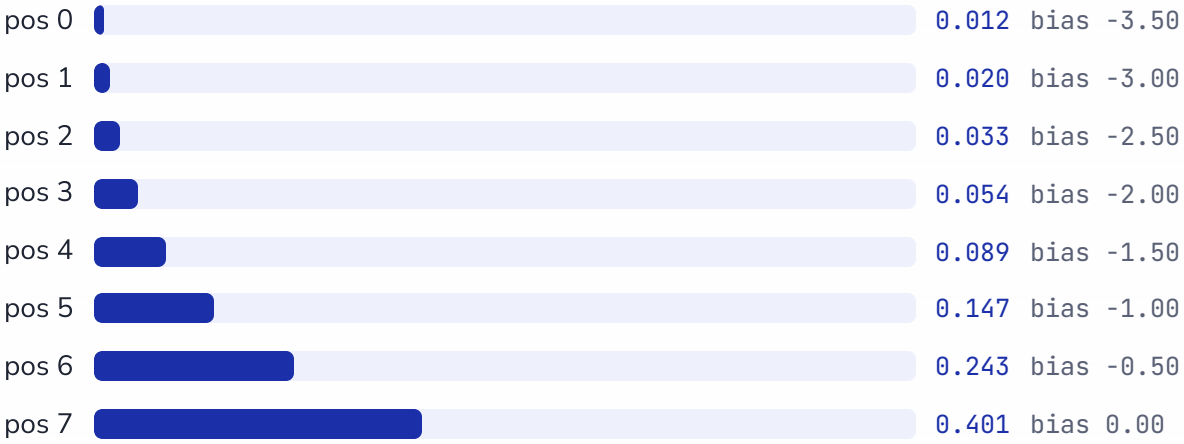
6.4 ALiBi in one row

The token at position 3 (causal, so it sees positions 0–3), with slope $m = 0.5$. Suppose all four content scores are equal (say 1). The biases are $-0.5 \times (3 - j)$:

j	0	1	2	3
Distance $3 - j$	3	2	1	0
Bias	−1.5	−1.0	−0.5	0
Weight after softmax	0.10	0.17	0.28	**0.46**

With nothing else to go on, the head prefers **nearby** tokens. (Adding the same content score to every entry doesn't change the softmax, so the result is the same for any equal content scores.)

Try it: ALiBi slopes, from short-sighted to long-sighted



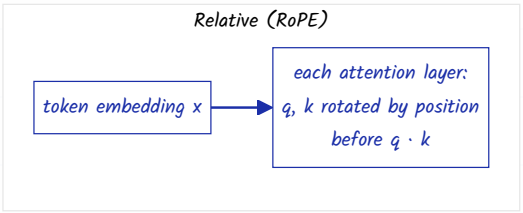
Slope 0.50: the newest token gets 40% and the oldest only 1.2%. Steep slopes make short-sighted heads; gentle slopes make long-sighted ones. ALiBi gives each head a different slope.

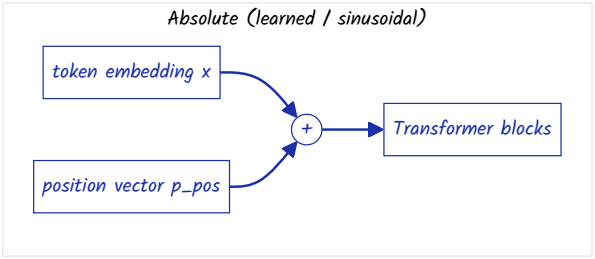
6.5 Cost of learned positions

- GPT-2: $1024 \times 768 = 786{,}432$ position parameters.
- BERT-base: $512 \times 768 = 393{,}216$.

Small next to the full models, but they **hard-limit** the input length.

7. Diagram





8. In practice

Model	Position scheme	Trained context
Original Transformer (2017)	Sinusoidal	—
BERT	Learned absolute	512
GPT-2	Learned absolute	1,024
T5	Learned relative biases (bucketed distances)	512
BLOOM	ALiBi	2,048
LLaMA-2	RoPE	4,096
LLaMA-3	RoPE (base 500,000)	8,192

- **Causal masks give a hint of position too.** In a decoder, token i sees exactly i earlier tokens, so a model can partly infer position even with no explicit scheme. Every production model still uses one.
- **Long-context models** (100k+ tokens) are usually RoPE models extended with position interpolation, a larger base, or YaRN, plus fine-tuning on long documents.
- **Positions in other data:** Vision Transformers use 2-D positions for image patches (Section 15). Audio models use positions along time.

9. Common mistakes and misconceptions

"Transformers read left to right, so they already know order." Only the causal mask is directional. Attention scores ignore order unless position information is added.

"Positional encoding is concatenated to the embedding." In the learned and sinusoidal schemes it is **added**, element by element, so the width stays d_{model}.

"RoPE adds a vector to the input like sinusoidal encoding." RoPE **rotates queries and keys inside each attention layer**. Nothing is added to x, and values are not rotated.

"Sinusoidal encodings let models handle any length perfectly." They're defined for any position, but models trained on short texts usually perform poorly on much longer ones.

"Position 0 is always the first word of the document." It's the first token of the **input sequence**, which may be a special token like `<bos>` or the middle of a document chunk.

10. Summary

- Self-attention is order-blind; **position information** makes word order count.
- **Absolute** schemes add a position vector to each embedding: **learned** (BERT, GPT-2; hard length limit) or **sinusoidal** (fixed (sin, cos) "clocks" at different speeds).
- **RoPE** rotates queries and keys by position-dependent angles, so scores depend only on the **relative distance** $m - n$. It's the standard in modern LLMs.

- **ALiBi** subtracts a distance-proportional penalty from scores, favouring nearby tokens.
- Context windows can be stretched with position interpolation, a larger RoPE base, or YaRN.

11. Check your understanding

1. Using the $d_{model} = 4$ formula in 6.1, compute the sinusoidal encoding for position 4. (Use $\sin 4 = -0.757$, $\cos 4 = -0.654$.)
2. A learned-position model has $L_{max} = 512$. What happens if you feed it 600 tokens? What could a RoPE model do instead?
3. With RoPE and $\theta = 30^\circ$ per position, a query at position 10 and a key at position 7 have score s. Give two other (query position, key position) pairs with the same score, and explain why.

▶ **Show answers**

12. Further reading

1. Vaswani, A. et al. (2017). **"Attention Is All You Need."** *NeurIPS 2017.* arXiv:1706.03762. Section 3.5 introduces sinusoidal positional encoding.
2. Su, J. et al. (2021). **"RoFormer: Enhanced Transformer with Rotary Position Embedding."** arXiv:2104.09864. The RoPE paper.
3. Press, O., Smith, N. A., & Lewis, M. (2021). **"Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation."** arXiv:2108.12409. The ALiBi paper.
4. Shaw, P., Uszkoreit, J., & Vaswani, A. (2018). **"Self-Attention with Relative Position Representations."** arXiv:1803.02155. An early relative-position method.
5. Chen, S. et al. (2023). **"Extending Context Window of Large Language Models via Positional Interpolation."** arXiv:2306.15595. And Peng, B. et al.

(2023). **"YaRN: Efficient Context Window Extension of Large Language Models."** arXiv:2309.00071.

Prepared by Akhil

Transformers & LLMs · Section 8

Residuals, LayerNorm & the Feed-Forward Network

> **In one line:** a Transformer block is attention plus three supporting parts: **residual connections** (add the input back so information and gradients can flow through deep stacks), **layer normalization** (keep each token's numbers in a stable range), and a **feed-forward network** (a small two-layer network that processes each token on its own).

1. Learning objectives
2. Prerequisites
3. Motivation
4. Intuition first
5. The formal idea
6. Worked examples
7. Diagram
8. In practice
9. Common mistakes
10. Summary
11. Check your understanding
12. Further reading

1. Learning objectives

By the end of this section you should be able to:

1. **Explain** why residual connections make deep networks trainable, using the gradient of $x + F(x)$.
2. **Compute** LayerNorm and RMSNorm by hand for a single token vector.
3. **Compute** a feed-forward network's output for a small example, with ReLU and GELU.

4. **Write** a full pre-norm Transformer block and compare pre-norm with post-norm.
5. **Count** the parameters of a complete model (GPT-2 small, LLaMA-2 7B) from its dimensions.

2. Prerequisites

- **Section 6**: multi-head attention maps $n \times d_{model}$ to $n \times d_{model}$ and has $4d_{model}^2$ parameters.
- **Section 1**: the block outline $A = X + \mathrm{Attn}(\mathrm{LN}(X))$, $X' = A + \mathrm{FFN}(\mathrm{LN}(A))$, now explained piece by piece.
- **Mean and variance** of a list of numbers: $\mu = \frac{1}{d}\sum x_i$ and $\sigma^2 = \frac{1}{d}\sum(x_i - \mu)^2$.
- **Chain rule**: the derivative of $f(g(x))$ is $f'(g(x))\,g'(x)$. Gradients through a stack of layers **multiply**.

> ***Quick refresher: activation functions***
>
> **ReLU** is $\max(0, z)$: negatives become 0. **GELU** is a smooth version, $z \cdot \Phi(z)$, where Φ is the standard normal CDF. It lets small negatives through a little: GELU(1) ≈ 0.84, GELU(−0.5) ≈ −0.15.

3. Motivation: attention alone isn't enough

Stacking many attention layers by themselves runs into three problems:

1. **Deep stacks are hard to train.** Gradients pass through every layer by multiplication (chain rule). If each layer shrinks them a little, then after 24, 48 or 96 layers they **vanish**.
2. **Numbers drift.** Each layer adds and mixes vectors, so values can grow or shrink layer after layer, making training unstable.

3. **Attention only *moves* information.** Its output is a weighted average of value vectors, which is a fairly simple, almost linear operation. The model also needs a place to **transform** information non-linearly and to store what it has learned ("Paris is the capital of France").

The three fixes, one for each problem:

Problem	Fix
Vanishing gradients in deep stacks	**Residual connections**
Drifting values	**Layer normalization**
Attention can mix but not really "think"	**Feed-forward network (FFN)**

4. Intuition first: editing a shared document

Think of each token's vector as a **shared document** that passes through many editors, one per layer.

- **Residual connection:** an editor doesn't rewrite the document from scratch; they **add tracked changes** on top. If an editor has nothing useful to add, the document passes through unchanged. Many small, safe edits instead of 48 complete rewrites. This running document is often called the **residual stream**.
- **LayerNorm:** before each editor reads the document, it's reformatted to a standard font size, so no editor is thrown off by text that has become huge or tiny.
- **Attention vs FFN:** the attention editor **gathers quotes from other documents** (other tokens). The FFN editor then **thinks privately** about their own document, using their background knowledge.

Where the analogy breaks

The "reformatting" (LayerNorm) is applied to the copy the editor reads, not to the document itself. In pre-norm Transformers the residual stream itself is never normalized until the very end.

5. The formal idea

5.1 Residual connections

Instead of $y = F(x)$, a residual (skip) connection computes

$$y = x + F(x)$$

where F is a sub-layer (attention or FFN). Its derivative is

$$\frac{\partial y}{\partial x} = 1 + F'(x)$$

In words: the **"1" is a gradient highway**. Even if $F'(x)$ is tiny, the gradient through this layer is at least about 1, so it doesn't vanish as layers are stacked. This idea came from ResNets for images (2015), and it is what made 100+-layer networks trainable.

Residuals also require $F(x)$ to have **the same shape as x**. This is why attention and the FFN both map d_{model} back to d_{model}.

5.2 Layer normalization

For **one token's** vector $x \in \mathbb{R}^{d_{model}}$:

$$\mu = \frac{1}{d}\sum_i x_i, \qquad \sigma^2 = \frac{1}{d}\sum_i (x_i - \mu)^2, \qquad \mathrm{LN}(x)_i = \gamma_i \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta_i$$

- Subtract the mean and divide by the standard deviation, so the token's features have mean 0 and variance 1.
- ϵ (e.g. 10^{-5}) avoids dividing by zero.
- γ (scale) and β (shift) are **learned**, one value per feature, so the model can undo the normalization where useful.

LayerNorm vs BatchNorm. LayerNorm normalizes **across the features of one token**. BatchNorm (common in CNNs) normalizes each feature **across the examples in a batch**. LayerNorm doesn't depend on batch size or on other sequences, which suits variable-length text.

RMSNorm (used in LLaMA, Mistral) skips the mean subtraction and β:

$$\mathrm{RMSNorm}(x)_i = \gamma_i \frac{x_i}{\sqrt{\frac{1}{d}\sum_j x_j^2 + \epsilon}}$$

It's slightly cheaper and works just as well in practice.

5.3 The feed-forward network (FFN)

Applied to **each token separately**, with the **same weights** for every position:

$$\mathrm{FFN}(x) = \phi(xW_1 + b_1)\,W_2 + b_2$$

Part	Shape	Role
W_1	$d_{model} \times d_{ff}$	expand into a wider hidden layer
ϕ	—	non-linear activation (ReLU, GELU, ...)
W_2	$d_{ff} \times d_{model}$	project back to the model width

Usually $d_{ff} = 4\,d_{model}$, giving $2 \times 4d_{model}^2 = 8\,d_{model}^2$ parameters, **twice the attention's** $4d_{model}^2$. So roughly two-thirds of each block's parameters are in the FFN.

A useful way to picture it: each of the d_{ff} hidden units is a **detector**. A column of W_1 is a pattern; if the token matches it, that unit fires, and the matching row of W_2 is written into the token's vector. Research suggests these "detector → write" pairs store much of a model's factual knowledge.

SwiGLU (LLaMA, PaLM) uses three matrices and a gate:

$$\mathrm{FFN}_{\mathrm{SwiGLU}}(x) = \left(\mathrm{SiLU}(xW_1) \odot xW_3\right) W_2$$

where $\odot$ is element-wise multiplication. To keep the parameter count similar, d_{ff} is set to about $\frac{8}{3}d_{model}$ (LLaMA-2 7B: $d_{ff} = 11{,}008$ for $d_{model} = 4{,}096$).

5.4 The complete block (pre-norm)

$$A = X + \mathrm{MultiHead}(\mathrm{LN}_1(X))$$
$$X' = A + \mathrm{FFN}(\mathrm{LN}_2(A))$$

Pre-norm models also apply one **final LayerNorm** after the last block, before the output head.

5.5 Pre-norm vs post-norm

	Post-norm (2017 original, BERT)	Pre-norm (GPT-2 onwards, most LLMs)
Formula	$\mathrm{LN}(x + F(x))$	$x + F(\mathrm{LN}(x))$
Residual path	passes through LN every layer	**clean**: x is never normalized on the highway
Training	needs careful learning-rate warm-up; unstable when very deep	stable even for very deep models

6. Worked examples

6.1 LayerNorm by hand

One token with $d_{model} = 4$: $x = [2, 4, 6, 8]$ (take $\gamma = 1$, $\beta = 0$, $\epsilon \approx 0$).

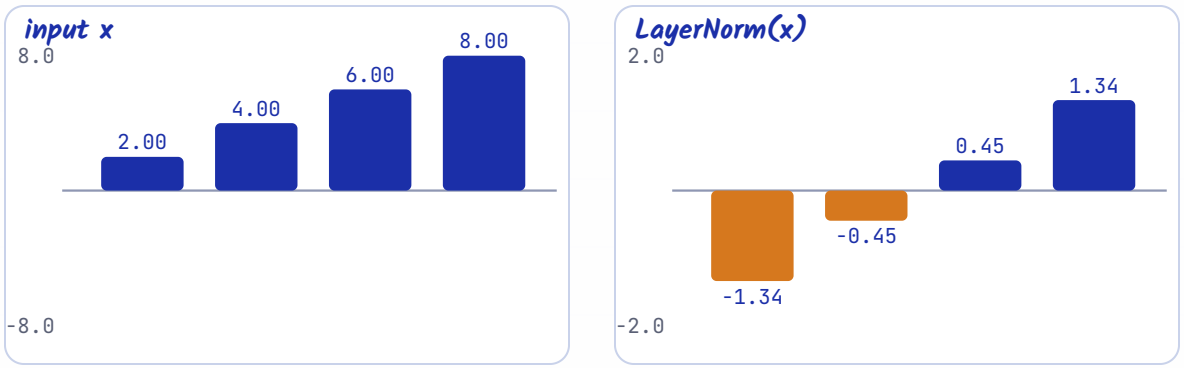
1. Mean: $\mu = (2 + 4 + 6 + 8)/4 = 5$
2. Variance: $\sigma^2 = \left((-3)^2 + (-1)^2 + 1^2 + 3^2\right)/4 = 20/4 = 5$, so $\sigma = \sqrt{5} = 2.236$
3. Normalize: $\left[\frac{-3}{2.236}, \frac{-1}{2.236}, \frac{1}{2.236}, \frac{3}{2.236}\right] = [\mathbf{-1.34}, \mathbf{-0.45}, \mathbf{0.45}, \mathbf{1.34}]$

The result has mean 0 and variance 1. The pattern (increasing, evenly spaced) is kept; only the scale and offset changed.

With learned $\gamma = [1, 1, 2, 2]$ and $\beta = [0, 0, 0, 1]$:
$[-1.34, -0.45, 2(0.45), 2(1.34) + 1] = [-1.34, -0.45, 0.89, 3.68]$.

RMSNorm of the same x: $\text{rms} = \sqrt{(4 + 16 + 36 + 64)/4} = \sqrt{30} = 5.477$, so the result is $x/5.477 = [0.37, 0.73, 1.10, 1.46]$. There is no centring, so all the values stay positive.

Try it: normalize one token



`μ = 5.00, σ² = 5.00, σ = 2.236 → output [-1.34, -0.45, 0.45, 1.34]`
`(mean 0.00, variance 1.00)`

Try “Multiply x by 10”: the input bars grow, but the normalized output doesn’t change at all. Normalization removes the overall scale and offset and keeps the pattern.

6.2 A feed-forward network by hand

A tiny FFN with $d_{model} = 2$, $d_{ff} = 4$, no biases:

$$W_1 = \begin{bmatrix} 1 & -1 & 0.5 & 0 \\ 0 & 1 & 1 & -1 \end{bmatrix}, \qquad W_2 = \begin{bmatrix} 0.5 & 0 \\ 0 & 1 \\ 0 & 0.5 \\ 1 & 1 \end{bmatrix}$$

Input token $x = [1, 0.5]$.

Step 1, expand: $h = xW_1 = [1(1) + 0.5(0),\ 1(-1) + 0.5(1),\ 1(0.5) + 0.5(1),\ 1(0) + 0.5(-1)] = [1, -0.5, 1, -0.5]$

Step 2, activate (ReLU): $\phi(h) = [1, 0, 1, 0]$. Hidden units 1 and 3 "fire"; units 2 and 4 stay silent.

Step 3, project back: only the rows of W_2 for the firing units contribute:

$$1 \cdot [0.5, 0] + 0 \cdot [0, 1] + 1 \cdot [0, 0.5] + 0 \cdot [1, 1] = [0.5, 0.5]$$

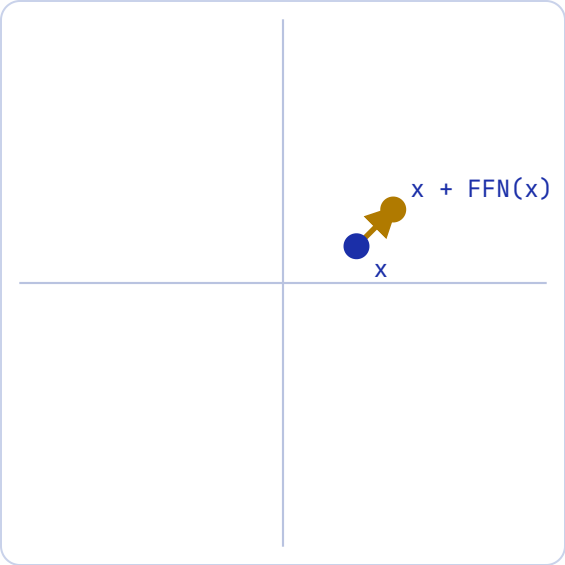
Step 4, residual: $x + \mathrm{FFN}(x) = [1, 0.5] + [0.5, 0.5] = [\mathbf{1.5}, \mathbf{1.0}]$

With GELU instead of ReLU: $\phi(h) = [0.84, -0.15, 0.84, -0.15]$ (silent units now leak a little). The FFN output becomes $[0.27, 0.11]$ and the residual sum $[1.27, 0.61]$. The activation function changes the result noticeably.

Try it: hidden units as detectors

unit 1	unit 2	unit 3	unit 4
pattern [1.0, 0.0]	pattern [−1.0, 1.0]	pattern [0.5, 1.0]	pattern [0.0, −1.0]
`h = 1.00`	`h = -0.50`	`h = 1.00`	`h = -0.50`
`φ(h) = 1.00`	`φ(h) = 0.00`	`φ(h) = 1.00`	`φ(h) = 0.00`
writes [0.5, 0.0]	writes [0.0, 1.0]	writes [0.0, 0.5]	writes [1.0, 1.0]

1. expand: h = xW_1 = `[1.00, -0.50, 1.00, -0.50]`
2. ReLU: φ(h) = `[1.00, 0.00, 1.00, 0.00]`
3. project: φ(h)W_2 = 1.00·[0.5, 0.0] + 1.00·[0.0, 0.5] = `[0.50, 0.50]`
4. residual: x + FFN(x) = **`[1.50, 1.00]`**



Units 1 and 3 recognise this input and write their rows of W_2 into the token. The arrow is the FFN's edit to the residual stream.

6.3 Why residuals rescue gradients

Suppose every sub-layer has derivative $F'(x) = 0.1$ (a layer that changes its input only a little).

Depth	Without residuals (0.1^L)	With residuals (1.1^L)
3 layers	0.001	1.331
24 layers	10^{-24}	$1.1^{24} \approx 9.8$

Without residuals, the gradient reaching the first layer is effectively **zero**, so early layers never learn. With residuals, it stays a usable size. (It can also grow, which is one reason normalization and careful initialization are still needed.)

Try it: the gradient highway

without residuals $(F')^L$ 1.0 × 10^-24

with residuals $(1 + F')^L$ 9.850

Bars use a log scale from 10^−30 to 10^10. After 24 layers the gradient reaching layer 1 is 1.0 × 10^-24 without residuals, which is effectively zero, so the early layers can't learn. With residuals it is 9.850, a usable size.

6.4 Counting a whole model

Per block (ignoring biases and norms): attention $4d^2$ + FFN $8d^2$ = $\mathbf{12\,d^2_{model}}$.

GPT-2 small ($d = 768$, 12 blocks, vocabulary 50,257, 1,024 positions, tied output head):

Part	Formula	Parameters
12 blocks	$12 \times 12 \times 768^2$	84,934,656
Token embeddings (Section 2)	$50{,}257 \times 768$	38,597,376
Position embeddings (Section 7)	$1{,}024 \times 768$	786,432
Biases	$12 \times (3d + d + 4d + d)$	82,944
LayerNorm γ, β	$12 \times 2 \times 2 \times 768 + 2 \times 768$	38,400
Total		**124,439,808**

That's exactly GPT-2 small's reported size of **124M**.

LLaMA-2 7B ($d = 4096$, 32 blocks, SwiGLU with $d_{ff} = 11{,}008$, vocabulary 32,000, separate output head, RMSNorm, no biases):

Part	Formula	Parameters
Attention per block	4×4096^2	67,108,864
SwiGLU FFN per block	$3 \times 4096 \times 11{,}008$	135,266,304
32 blocks	$32 \times 202{,}375{,}168$	6,476,005,376
Embeddings + output head	$2 \times 32{,}000 \times 4096$	262,144,000
RMSNorm γ	$32 \times 2 \times 4096 + 4096$	266,240
Total		**6,738,415,616**

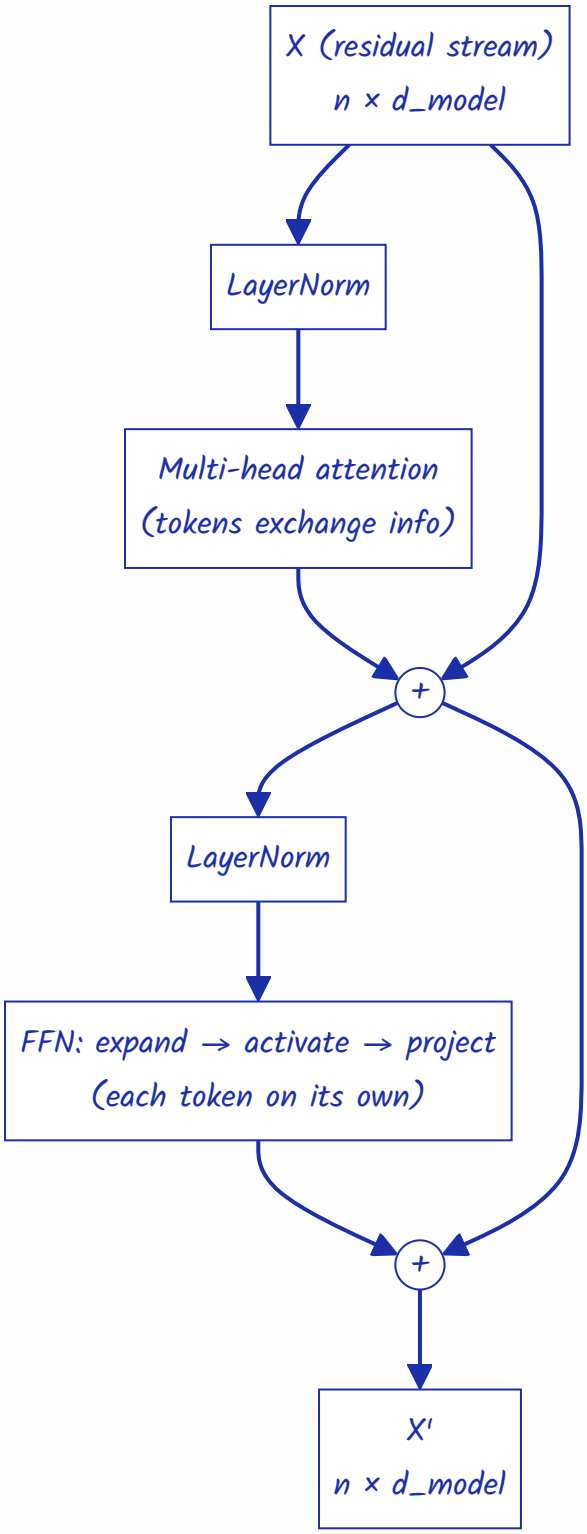
That's the reported **6.74B**. Every parameter in these models is accounted for by Sections 2–8.

Try it: count any model's parameters

Part	Formula	Parameters
Attention weights per block	`4 × 768²`	`2,359,296`
FFN weights per block	`2 × 768 × 3,072`	`4,718,592`
Biases per block	`attention 4d + FFN`	`6,912`
Norms per block	`2 × (γ, β) × d`	`3,072`
All 12 blocks	`12 × per-block total`	`85,054,464`
Token embeddings	`50,257 × 768`	`38,597,376`
Output head	`tied: reuses embeddings`	`0`
Learned positions	`1,024 × 768`	`786,432`
Final norm	`2 × d`	`1,536`
Total		**`124,439,808 (124.4 M)`**

✓ Exactly matches GPT-2 small's reported 124,439,808 parameters.

7. Diagram



8. In practice

Model	Norm	Placement	FFN activation	d_{ff}
Original Transformer	LayerNorm	post-norm	ReLU	$4d$ (2,048)
BERT	LayerNorm	post-norm	GELU	$4d$ (3,072)
GPT-2	LayerNorm	pre-norm (+ final LN)	GELU	$4d$ (3,072)
LLaMA-2 / Mistral	RMSNorm	pre-norm (+ final norm)	SwiGLU	$\approx \frac{8}{3}d$ (11,008 / 14,336)

- **Dropout** is often applied to each sub-layer's output before it's added to the residual stream (0.1 in the original Transformer and BERT). Large LLMs trained on huge data often use none.
- The **FFN is where most compute goes** for short inputs: for each token, about $2 \times 8d^2$ operations for the FFN against $2 \times 4d^2$ for the attention projections.
- **Mixture-of-Experts (MoE)** models (e.g. Mixtral) replace the single FFN with several "expert" FFNs and send each token to only a few of them: more parameters, similar compute per token.

9. Common mistakes and misconceptions

> **"LayerNorm normalizes across the batch."** That's BatchNorm. LayerNorm normalizes **each token's own features**, independently of other tokens and other examples.

> **"The FFN mixes information between tokens."** It doesn't; it's applied to every token **separately** (with shared weights). Only attention moves information between positions.

> **"Residual connections are an optional extra."** Without them, deep Transformers essentially fail to train. They also require every sub-layer to

keep the d_{model} shape.

"Normalization removes information." It removes the token's overall **offset and scale**. The pattern across features is kept, and the learned γ, β can restore scale where it's useful.

"Attention holds most of the parameters." In standard blocks the FFN holds about **two-thirds** ($8d^2$ of $12d^2$).

10. Summary

- **Residual connections** $y = x + F(x)$ give gradients a highway ($\partial y/\partial x = 1 + F'$) and turn each layer into a small edit of the **residual stream**.
- **LayerNorm** rescales each token's features to mean 0, variance 1, then applies learned γ, β; **RMSNorm** skips the centring.
- The **FFN** expands each token to d_{ff} (often $4d$), applies a non-linearity, and projects back. It holds about 2/3 of the block's parameters and acts like a bank of detectors.
- **Pre-norm** blocks, $x + F(\mathrm{LN}(x))$, train more stably than **post-norm**, $\mathrm{LN}(x + F(x))$.
- A block has about $12d_{model}^2$ parameters; adding embeddings reproduces GPT-2's 124M and LLaMA-2 7B's 6.74B exactly.

11. Check your understanding

1. Compute LayerNorm (with $\gamma = 1$, $\beta = 0$) of $x = [1, 1, 1, 5]$.
2. In 6.2, which hidden units fire for the input $x = [0, 1]$ with ReLU, and what is $x + \mathrm{FFN}(x)$?
3. A model has $d_{model} = 1024$, 24 blocks, standard FFN ($d_{ff} = 4d$) and a tied vocabulary of 50,000. Estimate its parameter count, ignoring biases, norms and positions.

▶ Show answers

12. Further reading

1. He, K., Zhang, X., Ren, S., & Sun, J. (2015). **"Deep Residual Learning for Image Recognition."** arXiv:1512.03385. The origin of residual connections.
2. Ba, J. L., Kiros, J. R., & Hinton, G. E. (2016). **"Layer Normalization."** arXiv:1607.06450.
3. Zhang, B., & Sennrich, R. (2019). **"Root Mean Square Layer Normalization."** arXiv:1910.07467. RMSNorm.
4. Xiong, R. et al. (2020). **"On Layer Normalization in the Transformer Architecture."** arXiv:2002.04745. Why pre-norm trains more stably.
5. Shazeer, N. (2020). **"GLU Variants Improve Transformer."** arXiv:2002.05202. SwiGLU and related FFNs.
6. Geva, M. et al. (2021). **"Transformer Feed-Forward Layers Are Key-Value Memories."** arXiv:2012.14913. The "detectors that write" view of the FFN.

Prepared by Akhil

The Encoder Stack

> **In one line:** an **encoder** is a stack of N identical Transformer blocks with **bidirectional** self-attention (every token sees every other token). It turns a sequence of token embeddings into a sequence of **contextual vectors** — one per token — that downstream tasks like classification, tagging and search are built on.

1. Learning objectives
2. Prerequisites
3. Motivation
4. Intuition first
5. The formal idea
6. Worked examples
7. Diagram
8. In practice
9. Common mistakes
10. Summary
11. Check your understanding
12. Further reading

1. Learning objectives

By the end of this section you should be able to:

1. **Describe** the encoder's input, its stack of blocks, and its output (shapes included).
2. **Explain** what "bidirectional" means and how it differs from the causal attention of decoders.
3. **Trace** how a token's vector changes layer by layer, using a small worked example.
4. **Explain** masked language modelling (MLM) — including the 15% and 80/10/10 rules — and compute its loss.

5. **Choose** how to use encoder outputs for sentence classification, token tagging and sentence embeddings.

2. Prerequisites

- **Section 8:** a pre-norm block $X' = A + \mathrm{FFN}(\mathrm{LN}(A))$ with $A = X + \mathrm{MHA}(\mathrm{LN}(X))$; each block maps $n \times d_{model} \to n \times d_{model}$.
- **Section 5:** masks — padding masks hide filler tokens; causal masks hide future tokens.
- **Section 2:** cosine similarity between vectors.

> **Quick refresher — cross-entropy loss**
> If a model gives the correct answer probability p, its loss is $-\log p$. Confident and right ($p = 0.9$) → small loss 0.105; unsure ($p = 0.2$) → larger loss 1.609. Training minimizes the average loss.

3. Motivation: understanding needs both sides

Consider:

> *I went to the **bank** to deposit my salary.*
> *I sat on the **bank** and watched the river.*

The words that disambiguate "bank" ("deposit", "river") come **after** it. A model that reads only left-to-right — like a text generator — must decide what "bank" means before seeing those clues.

For **understanding** tasks — "is this review positive?", "which words are names?", "which document answers this question?" — the whole text is available at once.

There is no reason to hide the future. So we want a model where every token can look **both left and right**: an **encoder**.

4. Intuition first: a committee drafting a report in rounds

A committee of 12 people is writing a joint report. Each member holds one paragraph.

- **Round 1:** everyone reads everyone else's paragraph and updates their own to fit the whole report.
- **Round 2:** they read the *updated* paragraphs and refine again — now each paragraph reflects not only the others, but the others' first-round understanding.
- After **many rounds**, every paragraph is consistent with — and informed by — the entire report.

That is an encoder:

- Each **member/paragraph** = one token's vector.
- Each **round** = one Transformer block (layer).
- **Reading everyone's paragraph** = bidirectional self-attention.
- **Updating your own** = the residual + FFN.

Where the analogy breaks

Committee members could choose to read only some paragraphs. In an encoder, *every* token attends to *every* token in every layer (except padding) — the only choice is how much weight to give each one.

5. The formal idea

5.1 The pipeline

$$
\begin{aligned}
X^{(0)} &= \mathrm{Embed}(\mathrm{tokens}) + \mathrm{Positions} && (n \times d_{model}) \\
X^{(\ell)} &= \mathrm{Block}_{\ell}\left(X^{(\ell-1)}\right), \quad \ell = 1, \ldots, N && (n \times d_{model}) \\
H &= \mathrm{LN}_{\mathrm{final}}\left(X^{(N)}\right) && (n \times d_{model})
\end{aligned}
$$

H holds one **contextual vector** per input token — the encoder's output.

5.2 "Bidirectional" = no causal mask

In each block, self-attention uses only a **padding mask** (Section 5). So token i can attend to every real token j, before or after it:

	Encoder (bidirectional)	Decoder (causal)
Token i sees	all tokens $1 \ldots n$	only tokens $1 \ldots i$
Attention matrix	full	lower-triangular
Good for	understanding the whole input	generating text left-to-right

"Bidirectional" does **not** mean reading the text twice (once each way). All tokens see all others **simultaneously**, in one attention step.

Try it: who can see whom?

	I	sat	on	the	bank	and	watched	the	river
I	✓	✓	✓	✓	✓	✓	✓	✓	✓
sat	✓	✓	✓	✓	✓	✓	✓	✓	✓
on	✓	✓	✓	✓	✓	✓	✓	✓	✓
the	✓	✓	✓	✓	✓	✓	✓	✓	✓
bank	✓	✓	✓	✓	✓	✓	✓	✓	✓
and	✓	✓	✓	✓	✓	✓	✓	✓	✓
watched	✓	✓	✓	✓	✓	✓	✓	✓	✓
the	✓	✓	✓	✓	✓	✓	✓	✓	✓
river	✓	✓	✓	✓	✓	✓	✓	✓	✓

“bank” can attend to 9 of 9 words. In the encoder every word sees the whole sentence, including the clue “river”. All 81 pairs allowed.

5.3 What the layers learn

Studies of trained encoders (e.g. BERT) find a rough progression:

- **Lower layers:** surface features — word identity, part of speech, nearby words.
- **Middle layers:** syntax — phrase structure, which word modifies which.
- **Upper layers:** meaning in context — word sense, coreference, task-specific features.

Each layer refines the previous one, like the committee's rounds.

5.4 Pre-training with masked language modelling (MLM)

You can't train a bidirectional encoder with "predict the next word" — each token can already see the next word, so it would just copy it. Instead, BERT-style encoders **hide some words and ask the model to fill them in**:

1. Pick **15%** of the tokens at random.
2. Of those picked: **80%** are replaced with `[MASK]`, **10%** with a **random** token, **10%** are left **unchanged**.
3. The model predicts the **original** token at each picked position.
4. The loss is cross-entropy **only at the picked positions**.

$$\mathcal{L}_{\text{MLM}} = -\frac{1}{|\mathcal{M}|} \sum_{i \in \mathcal{M}} \log P(\text{original token}_i \mid \text{corrupted text})$$

where $\mathcal{M}$ is the set of picked positions.

Why the 80/10/10 split? At fine-tuning and test time, real text contains no `[MASK]` tokens. If the model only ever saw `[MASK]` at prediction positions, it might learn to make good representations *only* there. The random and unchanged cases force it to build a good representation for **every** token.

Other encoder objectives you'll meet:

- **NSP (next sentence prediction):** "does sentence B follow sentence A?" — used by BERT, later dropped (RoBERTa showed it wasn't helping).
- **Replaced token detection (ELECTRA):** a small generator swaps some tokens; the encoder classifies *every* token as original or replaced — learning from all positions, not just 15%.
- **Contrastive training** (sentence-embedding models): pull vectors of matching sentence pairs together and push non-matching pairs apart.

5.5 Using the encoder's output

Task	What you take from H	Head on top
Sentence classification (sentiment, spam)	vector of the special `[CLS]` token (or mean of all tokens)	linear layer → classes
Token tagging (names, parts of speech)	**every** token's vector	linear layer per token → tags
Extractive QA ("find the answer span")	every token's vector	two scores per token: "answer starts here" / "ends here"
Sentence embeddings (search, RAG)	**mean pooling** of all token vectors, usually normalized	none — compare with cosine similarity

6. Worked examples

6.1 Two layers of a tiny encoder

We use our toy 2-D vectors and a **simplified block**: one attention head with identity projections and scaling ($\sqrt{2}$), plus the residual — LayerNorm and FFN left out so the arithmetic stays visible:

$$X^{(\ell)} = X^{(\ell-1)} + \mathrm{softmax}\left(\frac{X^{(\ell-1)} X^{(\ell-1)\top}}{\sqrt{2}}\right) X^{(\ell-1)}$$

Sentence A: "the bat flew", $X^{(0)} = [\,0.2, 0.2\,;\ 1, 1\,;\ 2, 0\,]$.

Layer 1 — attention weights (each row sums to 1; every token sees all three):

	the	bat	flew
the	0.29	0.36	0.36
bat	0.14	0.43	0.43
flew	0.06	0.18	0.76

bat's new vector $= [1,1] + (0.14[0.2,0.2] + 0.43[1,1] + 0.43[2,0]) = [1,1] + [1.32, 0.46] = [2.32, 1.46]$.

The full outputs:

Token	Layer 0	Layer 1	Layer 2
the	[0.20, 0.20]	[1.33, 0.61]	[4.44, 1.21]
bat	[1.00, 1.00]	[2.32, 1.46]	[5.60, 2.00]
flew	[2.00, 0.00]	[3.71, 0.20]	[7.37, 0.43]

Sentence B: "the bat hit" gives the mirror image (the two axes swap): bat → $[1.46, 2.32]$ after layer 1, $[2.00, 5.60]$ after layer 2.

How similar is "bat" in the two sentences? (cosine similarity)

Layer	0	1	2	3
cos(bat in A, bat in B)	**1.00**	0.90	0.63	0.36

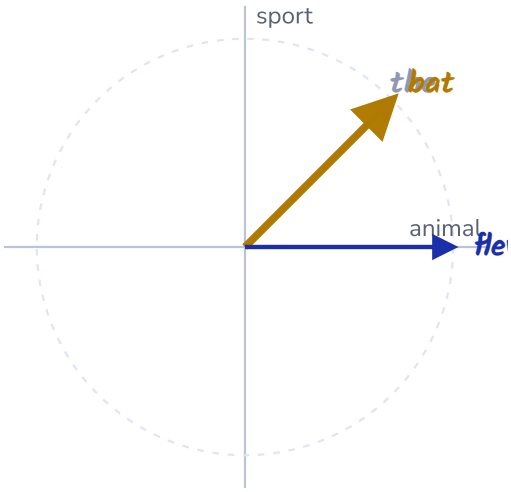
At layer 0, "bat" is the *same* vector in both sentences (it's just the embedding). Each layer pulls it further toward its own sentence's meaning, so the two "bat"s **diverge** — animal-bat and cricket-bat become different representations. This is exactly what "contextual vectors" means.

(Notice the numbers also grow each layer. That's because we left out LayerNorm; real encoders normalize, which keeps the scale steady while preserving these directions.)

Try it: watch "bat" diverge, layer by layer

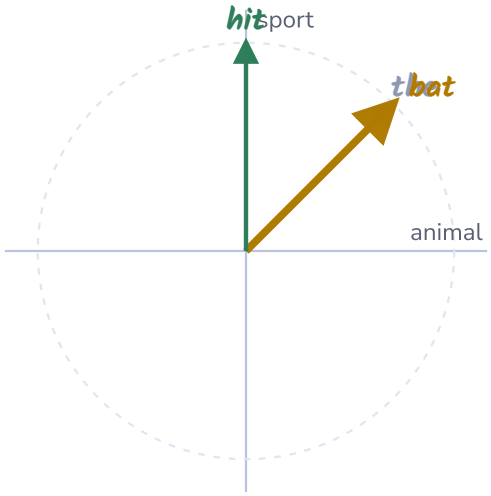
Arrows show each token's **direction** (scaled to the same length, as LayerNorm would keep them comparable). Faint dots trace bat's earlier layers.

"the bat flew"



Layer 0 is just the embeddings — no attention yet.

"the bat hit"



Layer 0 is just the embeddings — no attention yet.

layer 0	layer 1	layer 2	layer 3
1.00	**0.90**	**0.63**	**0.36**
cos(bat, bat)	cos(bat, bat)	cos(bat, bat)	cos(bat, bat)

Layer 0: “bat” is the same embedding [1, 1] in both sentences, so cosine similarity is 1.00. Move the slider up.

6.2 The MLM recipe on a 20-token sentence

Expected numbers for one 20-token sentence:

- Picked: $15\% \times 20 = 3$ tokens.
- Of these, on average: $0.8 \times 3 = 2.4$ become `[MASK]`, 0.3 become a random token, 0.3 stay unchanged.

Suppose the model gives the correct original tokens probabilities 0.6, 0.2 and 0.9 at the three picked positions:

$$\mathcal{L} = \tfrac{1}{3}\big(-\log 0.6 - \log 0.2 - \log 0.9\big) = \tfrac{1}{3}(0.511 + 1.609 + 0.105) = \mathbf{0.7}$$

The other 17 positions contribute **nothing** to the loss — one reason MLM learns less per sentence than next-token prediction (and why ELECTRA's "every token" objective is more sample-efficient).

Try it: corrupt a sentence the BERT way

students	in	hyderabad	often	study	late	at	night was: night
before	[MASK] was: the	final	exams	because	the	syllabus	is
long	and	difficult	purple was: .				

[MASK] random word picked but unchanged not picked (no loss)

How confident is the model about each original word?

pos 7 (“night”)	p = 0.50	-log p = 0.693
pos 9 (“the”)	p = 0.50	-log p = 0.693
pos 19 (“.”)	p = 0.50	-log p = 0.693

loss = (0.693 + 0.693 + 0.693) / 3 = 0.693

3 of 20 tokens picked (15%): 1 masked, 1 random, 1 unchanged. The model must predict the original word at every picked position — even where the word looks normal. The other 17 positions add nothing to the loss. Set the sliders to 0.6, 0.2 and 0.9 to reproduce 6.2.

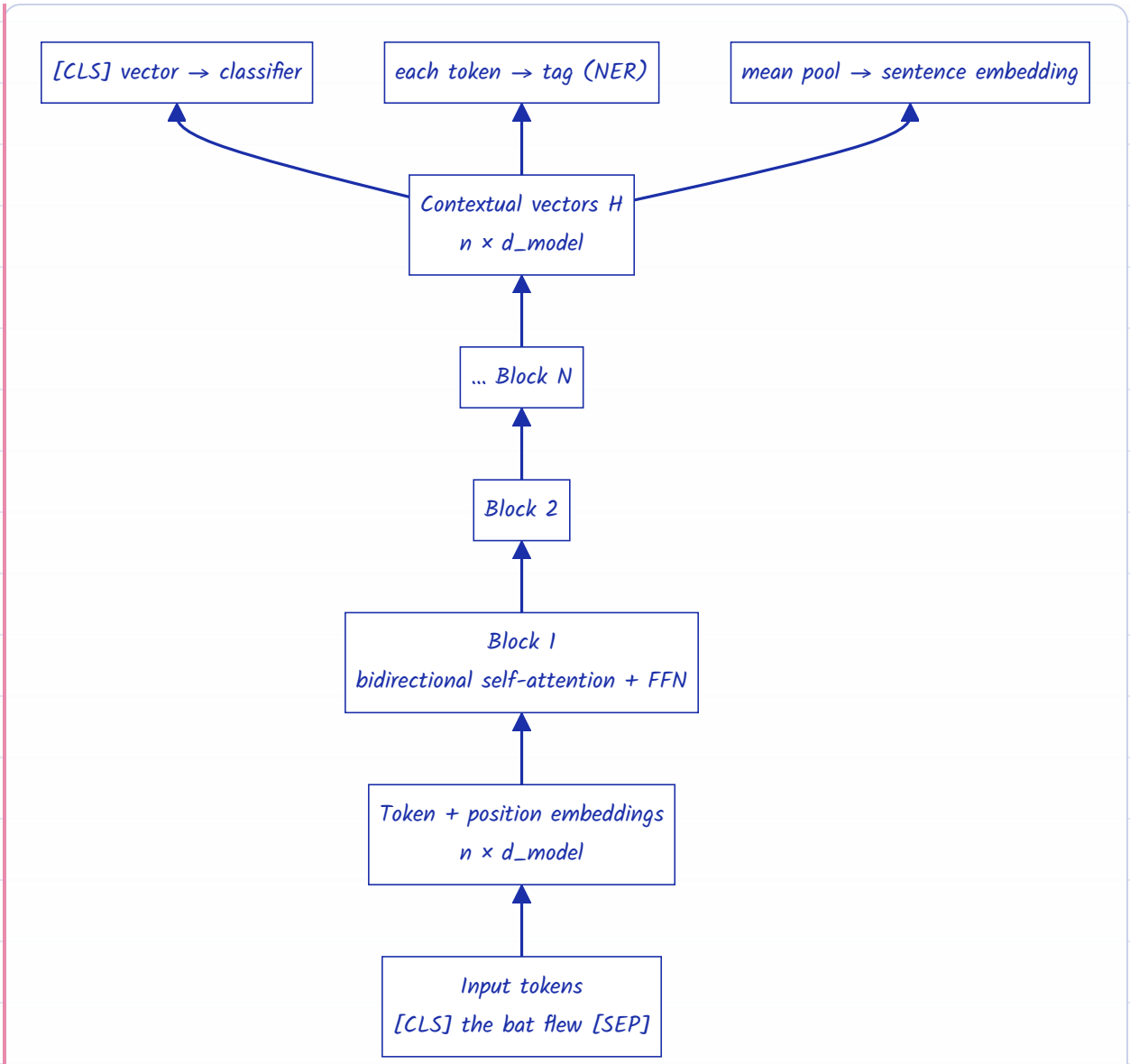
6.3 BERT-base, every parameter accounted for

BERT-base: $d = 768$, 12 layers, 12 heads, $d_{ff} = 3072$, vocabulary 30,522, 512 positions, 2 segment types, post-norm LayerNorm with biases.

Part	Formula	Parameters
Token + position + segment embeddings, embedding LayerNorm	$(30{,}522 + 512 + 2) \times 768 + 2 \times 768$	23,837,184
One layer: attention ($4d^2 + 4d$) + FFN ($2d \cdot d_{ff} + d_{ff} + d$) + 2 LayerNorms ($4d$)		7,087,872
12 layers	$12 \times 7{,}087{,}872$	85,054,464
Pooler (a $d \times d$ layer on `[CLS]`)	$768^2 + 768$	590,592
Total		**109,482,240**

That's BERT-base's "110M". The same recipe with $d = 1024$, 24 layers, $d_{ff} = 4096$ gives BERT-large's **335M**.

7. Diagram



8. In practice

Model	Year	Layers / d_{model}	Pre-training	Notes
BERT-base / large	2018	12 / 768, 24 / 1024	MLM + NSP	the classic encoder
RoBERTa	2019	same as BERT	MLM only, more data, longer training	drops NSP
ELECTRA	2020	various	replaced token detection	learns from every token
DeBERTa	2020	various	MLM	separates content and position in attention
Sentence-embedding models (e.g. E5, BGE)	2022+	BERT-sized	contrastive	power semantic search and RAG
ModernBERT	2024	22 / 768 (base)	MLM	RoPE, 8,192-token context

- **Encoders are still everywhere** even in the LLM era: search engines, spam filters, recommendation, and the **retrieval step of RAG** (Section 18) mostly run on encoders because they are small, fast and excellent at "understanding" tasks.
- **Fine-tuning** an encoder for classification usually means adding one linear layer and training the whole thing for a few epochs on labelled data.
- **Cross-encoders vs bi-encoders:** a *bi-encoder* embeds query and document separately (fast, used for search over millions of documents); a *cross-encoder* feeds both into one encoder together (slower, more accurate, used to **re-rank** the top results).

9. Common mistakes and misconceptions

"An encoder can write text like GPT." Encoders produce vectors for the input they're given; they aren't trained to generate text token by token.

"Bidirectional means it reads left-to-right, then right-to-left." All tokens attend to all others **at once**, in every layer.

"The `[CLS]` vector magically summarizes the sentence." It only becomes a useful summary through training (pre-training objectives and fine-tuning). For raw sentence similarity, mean pooling from a model trained for embeddings usually works much better.

"Encoders need a causal mask." No — only a padding mask. A causal mask would throw away the right-hand context that makes encoders good at understanding.

"The last layer is always the best representation." For some tasks (e.g. syntax), middle layers carry more useful information; the top layers are shaped by the pre-training objective.

10. Summary

- An **encoder** = embeddings + positions → N identical blocks with **bidirectional** self-attention → one **contextual vector** per token ($n \times d_{model}$).
- Every token sees every other token in every layer; only padding is masked.
- Layers refine representations progressively; the same word **diverges** into different vectors in different contexts (bat: cosine 1.00 → 0.36 over three toy layers).
- Encoders are pre-trained with **MLM**: pick 15% of tokens (80% `[MASK]`, 10% random, 10% unchanged) and predict the originals; the loss counts only the picked positions.
- Outputs are used via `[CLS]`/mean pooling (classification, embeddings) or per-token heads (tagging, QA). BERT-base = **109,482,240** parameters.

11. Check your understanding

1. BERT-base has 12 layers with 12 heads each. For a 128-token input, how many attention scores ($q \cdot k$ values) are computed in total across all heads and layers?
2. A 40-token sentence goes through the MLM recipe. How many tokens are picked, and how many of them are expected to become `[MASK]`, random, and unchanged?
3. Why can't a bidirectional encoder be trained with ordinary next-token prediction? What would it learn?

▶ **Show answers**

12. Further reading

1. Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). **"BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding."** *NAACL 2019.* arXiv:1810.04805 — MLM, the 80/10/10 rule, `[CLS]`.
2. Liu, Y. et al. (2019). **"RoBERTa: A Robustly Optimized BERT Pretraining Approach."** arXiv:1907.11692.
3. Clark, K., Luong, M.-T., Le, Q. V., & Manning, C. D. (2020). **"ELECTRA: Pre-training Text Encoders as Discriminators Rather Than Generators."** arXiv:2003.10555.
4. Tenney, I., Das, D., & Pavlick, E. (2019). **"BERT Rediscovers the Classical NLP Pipeline."** arXiv:1905.05950 — what different layers encode.
5. Reimers, N., & Gurevych, I. (2019). **"Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks."** arXiv:1908.10084 — bi-encoders for semantic search.

Transformers & LLMs · Section 10

The Decoder Stack

> **In one line:** a **decoder** is a stack of Transformer blocks with **causal (masked) self-attention**, so each position sees only the past. That lets it be **trained on every position of a sentence in parallel** but **generate text one token at a time**. In translation-style models it also has **cross-attention** to read the encoder's output.

1. Learning objectives
2. Prerequisites
3. Motivation
4. Intuition first
5. The formal idea
6. Worked examples
7. Diagram
8. In practice
9. Common mistakes
10. Summary
11. Check your understanding
12. Further reading

1. Learning objectives

By the end of this section you should be able to:

1. **Describe** the decoder block: masked self-attention, optional cross-attention, FFN — with residuals and norms.
2. **Explain** teacher forcing: how one forward pass gives a training signal at *every* position.
3. **Contrast** training (parallel) with generation (sequential, one forward pass per new token).
4. **Compute** cross-attention by hand, with queries from the decoder and keys/values from the encoder.

5. **Distinguish** decoder-only models (GPT, LLaMA, Claude) from encoder–decoder models (original Transformer, T5).

2. Prerequisites

- **Section 5:** the causal mask — a lower-triangular pattern that sets future scores to $-\infty$.
- **Section 9:** the encoder produces contextual vectors H ($n_{src} \times d_{model}$).
- **Section 8:** a block = sub-layers wrapped in residuals and normalization.
- **Cross-entropy:** loss $= -\log p(\text{correct token})$.

> **Quick refresher — "autoregressive"**
> An **autoregressive** model produces a sequence one item at a time, feeding each output back in as input for the next step — like writing a sentence where every new word depends on all the words you have already written.

3. Motivation: generating text needs a "no peeking" model

To **write** text, a model must answer one question again and again:

> *Given everything so far, what is the next token?*

Two requirements follow:

1. When predicting position t, the model must **not see** positions $t, t+1, \ldots$ — otherwise it would just copy the answer during training and fail at test time, where the future doesn't exist yet.
2. Training must still be **efficient** — we can't afford a separate forward pass for every word of every training sentence.

The causal mask solves both: it hides the future **inside** a single parallel pass. The decoder is the block stack built around this mask.

4. Intuition first: an exam answered line by line

Imagine writing an exam essay one line at a time:

- You may **look at every line you've already written**, but every line **below** the one you're on is covered by a card (the causal mask).
- **Training** is like a teacher grading a model essay: the teacher can check, for *every* line at once, "given the lines above, was this the right next line?" — all lines graded in one sitting.
- **Generating** is like writing a fresh essay: you can't write line 5 until line 4 exists, so you go **one line at a time**.
- **Cross-attention** is an open-book exam for translation: while writing your Hindi answer, you may consult the **entire** English question (the encoder's output) at any time.

> **Where the analogy breaks**
>
> A student revises earlier lines; a decoder never changes tokens it has already produced — each new token is simply appended.

5. The formal idea

5.1 The decoder block

Decoder-only (GPT, LLaMA, Claude) — two sub-layers:

$$A = X + \text{MaskedMHA}(\text{LN}(X))$$
$$X' = A + \text{FFN}(\text{LN}(A))$$

This is exactly the Section 8 block with a **causal mask** in the attention.

Decoder in an encoder–decoder model (original Transformer, T5) — three sub-layers:

$$
\begin{aligned}
A &= X + \mathrm{MaskedMHA}(\mathrm{LN}(X)) \\
B &= A + \mathrm{CrossMHA}\big(\underbrace{\mathrm{LN}(A)}_{\text{queries}}, \underbrace{H}_{\text{keys, values}}\big) \\
X' &= B + \mathrm{FFN}(\mathrm{LN}(B))
\end{aligned}
$$

5.2 Cross-attention

Cross-attention is ordinary attention where the queries and the keys/values come from **different sequences**:

$$
Q = (\text{decoder states})\, W_Q, \qquad K = H\, W_K, \qquad V = H\, W_V
$$

Matrix	Shape
Q (from the n_{tgt} target tokens)	$n_{tgt} \times d_k$
K, V (from the n_{src} source tokens)	$n_{src} \times d_k$
Scores $QK^\top$	$n_{tgt} \times n_{src}$ (**not square**)

Each target token asks: *"which source words matter for what I'm writing now?"* There is **no causal mask** in cross-attention — the whole source sentence is already known.

5.3 The output head

After the last block (and a final norm), a linear layer maps each position to **one score per vocabulary word** (often reusing the embedding matrix — weight tying, Section 2), and softmax turns scores into probabilities:

$$P(\text{next token} \mid \text{tokens} \le t) = \text{softmax}(h_t W_{out}) \qquad (\text{shape } |\mathcal{V}|)$$

5.4 Training: teacher forcing

Take a training sentence and add start/end tokens: `<bos> the bat flew <eos>`.

- **Input** = the sequence **shifted right** (starts with `<bos>`): `<bos> the bat flew`
- **Target** = the same sequence **shifted left**: `the bat flew <eos>`

Position	Sees (input up to here)	Must predict
1	`<bos>`	the
2	`<bos> the`	bat
3	`<bos> the bat`	flew
4	`<bos> the bat flew`	`<eos>`

Because of the causal mask, all four rows are computed in **one forward pass**, each seeing only its allowed prefix. The loss averages over **every** position:

$$\mathcal{L} = -\frac{1}{T}\sum_{t=1}^{T} \log P(x_{t+1} \mid x_{\le t})$$

This is called **teacher forcing**: during training the model is always fed the **true** previous tokens, never its own guesses. Compared with MLM (Section 9), every token is a training signal, not just 15%.

5.5 Inference: the generation loop

At inference time there is no target to copy, so tokens are produced **one per forward pass**:

1. Start with the prompt (e.g. `<bos>`).
2. Run the decoder; take the distribution at the **last** position.

3. Choose a token (greedy, sampling, … — Section 13).
4. **Append** it to the input.
5. Repeat until `<eos>` or a length limit.

	Training	Generation
Tokens processed per forward pass	all T at once	the sequence so far, to get **one** new token
Forward passes for a T-token sentence	**1**	T
What is fed in	the true text (teacher forcing)	the model's own outputs

Recomputing keys and values for the whole prefix at every step would be wasteful; the **KV cache** (Section 14) stores them so each step only processes the newest token.

5.6 Exposure bias

In training the model only ever sees *correct* prefixes; at generation time it sees its *own* outputs, mistakes included. If it makes an unusual choice, it finds itself in a situation it was never trained on, and errors can compound. This mismatch is called **exposure bias**. In practice, large-scale training makes models robust enough for this to matter less than early research feared, and decoding strategies (Section 13) help.

6. Worked examples

6.1 Teacher forcing on one sentence

Sentence: `<bos> the bat flew <eos>`. Suppose the model assigns the correct next token these probabilities:

Position	Context	Target	$P(\text{target})$	$-\log P$
1	`<bos>`	the	0.5	0.693
2	`<bos> the`	bat	0.4	0.916
3	`<bos> the bat`	flew	0.6	0.511
4	`<bos> the bat flew`	`<eos>`	0.7	0.357

$$\mathcal{L} = \tfrac{1}{4}(0.693 + 0.916 + 0.511 + 0.357) = \mathbf{0.619}$$

Four training signals from **one** forward pass of a 4-token input.

6.2 Generating the same sentence

Use a toy model whose next-token probabilities depend only on the last token (a stand-in for a real decoder):

After…	most likely next tokens
`<bos>`	**the 0.50**, a 0.30, …
the	**bat 0.40**, ball 0.30, …
bat	**flew 0.60**, hit 0.25, …
flew	**`<eos>` 0.70**, …

Greedy generation (always take the most likely token):

Pass	Input	Chosen
1	`<bos>`	the
2	`<bos> the`	bat
3	`<bos> the bat`	flew
4	`<bos> the bat flew`	`<eos>`

Same sentence — but **4 sequential passes** instead of 1, because pass $t+1$ needs the token chosen in pass t. Generating 1,000 tokens needs 1,000 passes; this is why text appears word by word in chatbots.

Try it: training (1 pass) vs generation (1 pass per token)

pos 1	`<bos>`	→ `the`
pos 2	`<bos>` `the`	→ `bat`
pos 3	`<bos>` `the` `bat`	→ `flew`
pos 4	`<bos>` `the` `bat` `flew`	→ `<eos>`

Each row sees only the tokens before its target. Press the button: all four predictions come out of a single pass.

6.3 Cross-attention by hand: translating "bat flew"

An encoder has read the English source "bat flew" and produced (toy 2-D, our usual axes) $H = [\,\text{bat} : [1,1];\ \text{flew} : [2,0]\,]$. The decoder is writing the Hindi translation **"चमगादड़ उड़ा"** (*chamgaadad udaa*). Take identity projections ($K = V = H$) and $d_k = 2$, so scores are divided by $\sqrt{2}$.

Step 1 — writing "चमगादड़" (bat). Suppose the decoder's query is $q = [0,2]$ ("looking for the noun").

- Scores: $q \cdot k_{\text{bat}}/\sqrt{2} = 2/1.414 = 1.41$; $q \cdot k_{\text{flew}}/\sqrt{2} = 0$
- Softmax: $e^{1.41} = 4.11$, $e^{0} = 1$ → weights **[0.80, 0.20]**
- Output: $0.80[1,1] + 0.20[2,0] = [1.20, 0.80]$

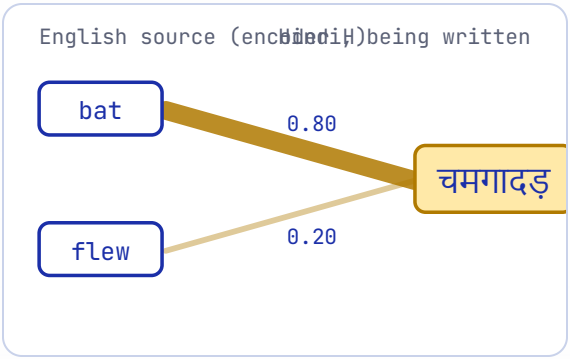
Step 2 — writing "उड़ा" (flew). Query $q = [2,0]$ ("looking for the action").

- Scores: $2/1.414 = 1.41$ and $4/1.414 = 2.83$
- Softmax → weights **[0.20, 0.80]**
- Output: $[1.80, 0.20]$

The cross-attention weights form a **soft alignment** between the two languages: when writing the Hindi word for "bat", the decoder looks mostly at "bat"; when

writing "flew", mostly at "flew". Here the score matrix is 2×2 only because both sentences have two words; in general it is $n_{tgt} \times n_{src}$.

Try it: cross-attention aligns the two languages



query q = [0.0, 2.0]
q·k(bat)/√2 = 2.00/1.414 = 1.41
q·k(flew)/√2 = 0.00/1.414 = 0.00
softmax → weights **[0.80, 0.20]**
output = 0.80·[1, 1] + 0.20·[2, 0] = **[1.20, 0.80]**

decoder self-attention (causal)

	<bos>
<bos>	✓

cross-attention (no mask)

	bat	flew
<bos>	✓	✓

While writing चमगादड़, the decoder looks mostly at “bat” (80%). Left grid: the decoder’s self-attention over its own Hindi words is causal. Right grid: cross-attention to the English source has no mask — every row may look at every source word.

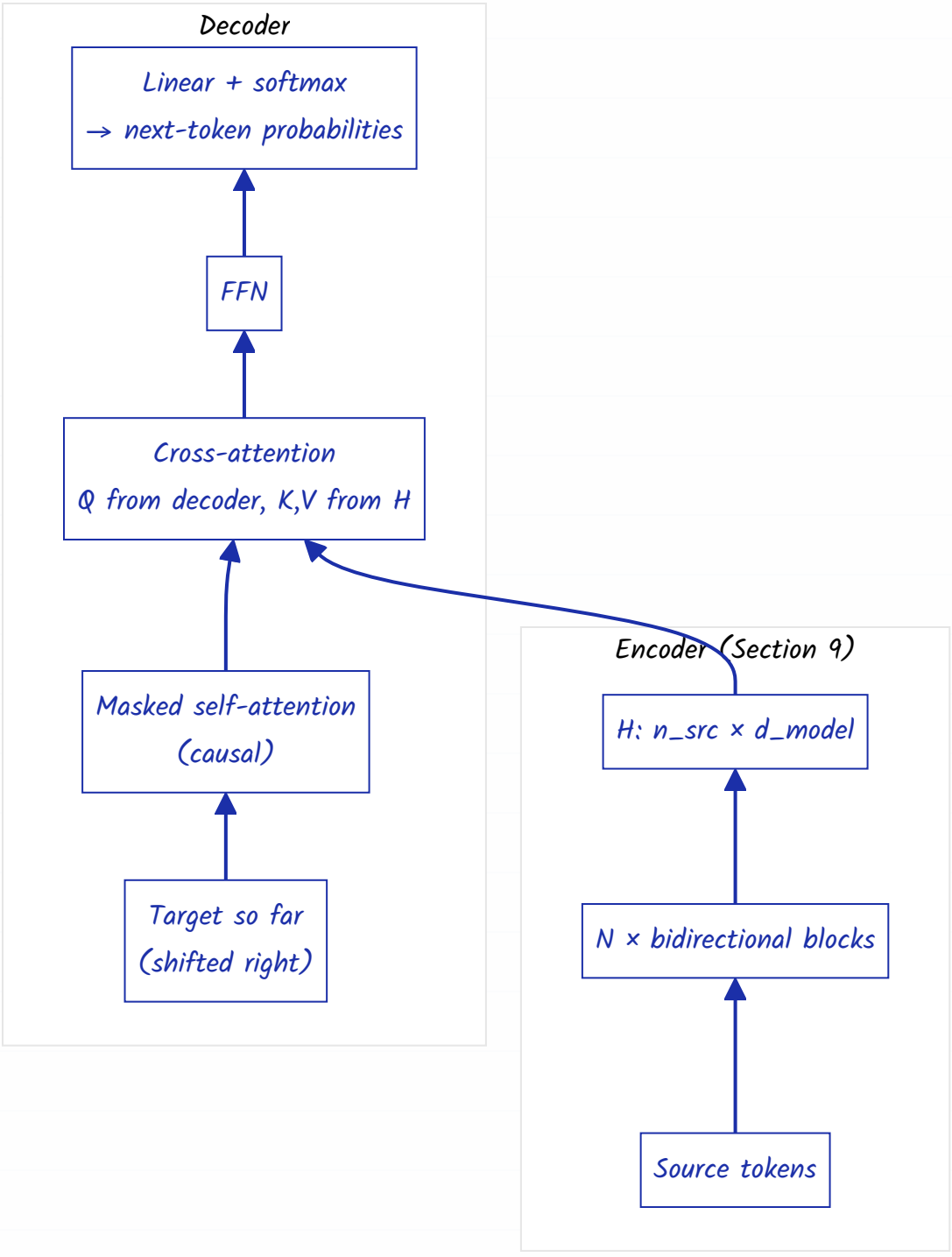
6.4 Counting the original Transformer (encoder–decoder)

Base model: $d = 512$, $d_{ff} = 2048$, 6 encoder + 6 decoder layers, a shared vocabulary of about 37,000 tokens (ignoring biases and norms):

Part	Formula	Parameters
Encoder layer	self-attn $4d^2$ + FFN $2d \cdot d_{ff}$	3,145,728
Decoder layer	self-attn $4d^2$ + **cross-attn** $4d^2$ + FFN $2d \cdot d_{ff}$	4,194,304
6 + 6 layers		44,040,192
Shared embeddings	$37{,}000 \times 512$	18,944,000
Total		**≈ 63.0 M**

Biases and LayerNorms bring it to the paper's reported **65M**. Each decoder layer is a third bigger than an encoder layer because of cross-attention.

7. Diagram



A **decoder-only** model is the right-hand stack **without** the cross-attention box (and with no encoder at all).

8. In practice

Model family	Architecture	Cross-attention?	Typical use
Original Transformer, T5, BART, Whisper	encoder–decoder	**yes**	translation, summarization, speech-to-text
GPT, LLaMA, Mistral, Claude	**decoder-only**	no	chat, writing, code, reasoning

- **Decoder-only dominates LLMs** (Section 11 explains why): the prompt is simply the beginning of the sequence, and "understanding the input" happens through causal self-attention over it.
- **Prefill vs decode:** at inference the prompt is processed in one parallel pass ("prefill", like training), and then new tokens come one per pass ("decode"). Prefill is limited by compute; decode by memory bandwidth (Section 14).
- **Speech and images:** Whisper's decoder cross-attends to audio-encoder outputs; many image-captioning models cross-attend to image features (Section 15).

9. Common mistakes and misconceptions

> **"Decoders are trained one token at a time."** Training is fully parallel: the causal mask lets all positions be predicted in one pass. Only **generation** is sequential.

> **"Cross-attention uses a causal mask."** No — the source sentence is fully known, so every target position may attend to **every** source position. The causal mask applies to the decoder's **self**-attention.

> **"GPT-style models have cross-attention."** Decoder-only models have only masked self-attention + FFN; there is no encoder to attend to.

> **"Teacher forcing is used when the model generates."** Teacher forcing is a **training** procedure. At generation time the model consumes its own outputs.

> **"The output head chooses the next word."** It produces a **probability distribution**; a separate decoding strategy (Section 13) picks the token.

10. Summary

- A **decoder** uses **causal self-attention**: position t sees only positions $\leq t$.
- **Encoder–decoder** decoders add **cross-attention** (Q from the decoder, K/V from the encoder; scores $n_{tgt} \times n_{src}$; no causal mask) — the soft alignment between languages.
- **Teacher forcing:** input shifted right, targets shifted left; one forward pass gives a loss at **every** position.
- **Generation** is autoregressive: one forward pass per new token, appended to the input (sped up by the KV cache).
- Decoder-only models (GPT, LLaMA, Claude) drop cross-attention entirely; encoder–decoder models (T5, Whisper) keep it.

11. Check your understanding

1. A target sentence has 6 tokens (including `<eos>`). How many next-token predictions does one teacher-forced training pass produce? How many forward passes are needed to *generate* those 6 tokens from `<bos>` (without a KV cache)?
2. In a translation model the decoder has produced 5 target tokens and the source has 8 tokens, with $d_k = 64$. Give the shapes of Q, K and the cross-attention score matrix for one head.
3. In 6.3, what are the cross-attention weights if the decoder's query is $q = [1, 1]$? What does that suggest about the step it's at?

▶ Show answers

12. Further reading

1. Vaswani, A. et al. (2017). **"Attention Is All You Need."** *NeurIPS 2017.* arXiv:1706.03762 — Section 3.1 describes the decoder with masked self-attention and encoder–decoder attention.
2. Radford, A. et al. (2018). **"Improving Language Understanding by Generative Pre-Training."** OpenAI — the original GPT, a decoder-only Transformer.
3. Raffel, C. et al. (2020). **"Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer."** *JMLR.* arXiv:1910.10683 — T5, an encoder–decoder model.
4. Bahdanau, D., Cho, K., & Bengio, Y. (2014). **"Neural Machine Translation by Jointly Learning to Align and Translate."** arXiv:1409.0473 — the origin of attention as soft alignment between languages.
5. Bengio, S. et al. (2015). **"Scheduled Sampling for Sequence Prediction with Recurrent Neural Networks."** arXiv:1506.03099 — on the training/inference mismatch (exposure bias).

Prepared by Akhil

Encoder vs Decoder vs Both

> **In one line:** all Transformers are built from the same block; what makes a model an **encoder**, a **decoder**, or an **encoder–decoder** is (1) its **attention mask** and (2) its **training objective**. Each combination is good at different jobs — and decoder-only models became the default for LLMs because one simple objective turns out to cover almost every task.

1. Learning objectives
2. Prerequisites
3. Motivation
4. Intuition first
5. The formal idea
6. Worked examples
7. Diagram
8. In practice
9. Common mistakes
10. Summary
11. Check your understanding
12. Further reading

1. Learning objectives

By the end of this section you should be able to:

1. **Draw** the attention pattern of an encoder, a decoder, a prefix-LM and an encoder–decoder, and count the allowed connections.
2. **Compare** the pre-training objectives — masked LM, causal LM, span corruption — and how many training signals each gives per sentence.
3. **Frame** the same task (e.g. sentiment classification) for all three architectures.
4. **Explain** why decoder-only models dominate modern LLMs, and where the other two still win.

5. **Choose** an architecture for a given application and justify it.

2. Prerequisites

- **Section 9 (encoder):** bidirectional attention, MLM, `[CLS]`/mean pooling.
- **Section 10 (decoder):** causal attention, teacher forcing, generation, cross-attention.
- **Section 5:** masks are added to the scores (0 = allowed, $-\infty$ = blocked).

> ***Quick refresher — the same block everywhere***
>
> Every model in this section uses the Section 8 block: attention + FFN with residuals and norms. Nothing about the block itself changes between architectures; only *who can attend to whom* and *what the model is trained to predict*.

3. Motivation: why not just one design?

Different jobs need different information flows:

Job	What the model needs
"Is this email spam?"	read the **whole** input, output **one label**
"Write a reply to this email"	produce **new text**, one token at a time, never seeing the future
"Translate this email into Hindi"	read the whole input **and** produce new text

An encoder is ideal for the first, a decoder for the second, and an encoder–decoder was designed for the third. Yet today a single decoder-only LLM does all three reasonably well. Understanding *why* is the goal of this section.

4. Intuition first: three kinds of students

- **The reviewer (encoder):** reads a whole essay, can flip back and forth freely, then writes **one verdict**: "Grade A". Great at understanding; never writes essays.
- **The storyteller (decoder):** writes a story one sentence at a time, only ever looking back at what's already written. Asked a question, they simply *continue the text* with an answer — so given the right opening, they can do almost anything.
- **The translator (encoder–decoder):** first reads the whole source document carefully (encoder), then writes the translation line by line (decoder), glancing back at the source whenever needed (cross-attention).

Where the analogy breaks

Real people combine these skills. In models the difference is hard-wired by the mask — a decoder physically *cannot* look at later tokens, however useful they would be.

5. The formal idea

5.1 Four attention patterns

For a sequence of n tokens (row = query, column = key; ✓ = allowed):

Architecture	Pattern	Allowed pairs
Encoder (bidirectional)	full $n \times n$	n^2
Decoder (causal)	lower triangle	$n(n+1)/2$
Prefix-LM	full on the first p tokens ("prefix"), causal after	$p^2 + \sum_{i=p+1}^{n} i$
Encoder–decoder	encoder: full n_{src}^2; decoder self: causal; cross: full $n_{tgt} \times n_{src}$	$n_{src}^2 + \frac{n_{tgt}(n_{tgt}+1)}{2} + n_{tgt}\, n_{src}$

A **prefix-LM** is a decoder that treats the prompt like an encoder (every prompt token sees the whole prompt) and the answer like a decoder (each answer token sees the prompt plus earlier answer tokens).

Try it: who can attend to whom?

bidirectional self-attention

	the	bat	flew	away	fast
the	✓	✓	✓	✓	✓
bat	✓	✓	✓	✓	✓
flew	✓	✓	✓	✓	✓
away	✓	✓	✓	✓	✓
fast	✓	✓	✓	✓	✓

allowed pairs: $5^2 = 25$

Encoder: every token sees every token, including later ones. Good for understanding; useless for generating (it could copy the answer). Click a row label to highlight what that token can see.

5.2 Three pre-training objectives

Objective	Used by	Input → target	Supervised positions
Masked LM (MLM)	encoders (BERT)	text with ~15% hidden → the hidden tokens	~15%
Causal LM (next-token)	decoders (GPT, LLaMA)	text → the same text shifted by one	**100%**
Span corruption / denoising	encoder–decoders (T5, BART)	text with spans replaced by placeholders → the missing spans	the corrupted spans (~15% in T5)

Span corruption (T5): pick short spans of the input, replace each with a unique **sentinel** token (`<X>` , `<Y>` , ...), and train the decoder to output the missing spans, each introduced by its sentinel.

5.3 Why decoder-only won for LLMs

1. **One objective, any text.** Next-token prediction needs no labels and no special corruption — every web page, book and code file is training data, and **every** token is a training signal.
2. **Every task is "continue this text".** Questions, translation, classification, code: put the task in the prompt and let the model write the answer. This is **prompting**, and with scale it gives **in-context learning** — learning a task from a few examples in the prompt.
3. **Simplicity and scaling.** One stack, one mask, one loss. Easy to scale to hundreds of layers and thousands of GPUs; scaling-law research mostly studied this setup.
4. **Efficient generation.** Causal attention means earlier tokens never change, so their keys and values can be cached (the KV cache, Section 14). That makes long chats affordable.
5. **Chat is a single growing sequence.** A conversation is just text appended turn after turn — exactly what a decoder handles natively.

5.4 Where the others still win

- **Encoders:** when you need a **vector or a label**, not text — search embeddings, classification, tagging — encoders are smaller, faster and often more accurate per parameter. RAG retrieval runs on encoders (Section 18).
- **Encoder–decoders: input → output transformations** with a long input and a shorter output (translation, summarization, speech-to-text). The input is read once, bidirectionally, and the decoder can stay small. Whisper (speech recognition) is encoder–decoder.

6. Worked examples

6.1 Counting allowed connections

Sequence of $n = 5$ tokens: a 3-token prompt "the bat flew" followed by a 2-token continuation "away fast".

Pattern	Calculation	Allowed pairs (of 25)
Encoder	5^2	**25**
Decoder	$5 \cdot 6/2$	**15**
Prefix-LM ($p = 3$)	$3^2 + 4 + 5$	**18**

In the prefix-LM, "the" can see "flew" (both are in the prefix) — impossible in a plain decoder — but "away" still cannot see "fast".

Encoder–decoder version (source "the bat flew", 3 tokens; target "away fast", 2 tokens):

- Encoder self-attention: $3 \times 3 = 9$
- Decoder self-attention (causal): $2 \cdot 3/2 = 3$
- Cross-attention: $2 \times 3 = 6$

6.2 Three objectives on one sentence

Sentence (8 tokens): **students often study late at night before exams**

Masked LM (one token hidden — 15% of 8 rounds to 1):

- Input: `students often [MASK] late at night before exams`
- Target: `study` at position 3 → **1** supervised position

Causal LM:

- Input: `<bos> students often study late at night before exams`
- Targets: `students often study late at night before exams <eos>` → **9** supervised positions (every one)

Span corruption (two spans, chosen large here so it's easy to see):

- Input: `students often <X> at night <Y> exams`
- Target: `<X> study late <Y> before <Z>`
- The decoder reconstructs **3** corrupted tokens (plus the sentinels marking where each span goes; `<Z>` marks the end).

Try it: what does each objective supervise?

Masked LM (encoder) — 1 of 8 positions supervised

input: `students` `[MASK]` `study` `late` `at` `night` `before` `exams`

predict: `often @2`

Causal LM (decoder) — 9 of 9 positions supervised

input: `<bos>` `students` `often` `study` `late` `at` `night` `before` `exams`

predict: `students` `often` `study` `late` `at` `night` `before` `exams` `<eos>`

Span corruption (encoder–decoder) — 1 corrupted tokens in 1 span

encoder in: `students` `often` `study` `late` `at` `night` `before` `<X>`

decoder out: `<X>` `exams` `<Y>`

Green tokens are what the model is trained to predict. Causal LM learns from every position (9); MLM from 1; span corruption from 1 (plus sentinels). Gold sentinels mark where each removed span belongs.

6.3 One task, three framings

Task: decide whether the review *"the movie was great"* is positive or negative.

Architecture	Input	Output	Extra parts needed
Encoder (BERT)	`[CLS] the movie was great [SEP]`	label from a classifier on the `[CLS]` vector	a trained classification head
Encoder–decoder (T5)	`sentiment: the movie was great`	the text `positive`	none — the label is generated as text
Decoder-only (GPT/LLaMA)	`Review: the movie was great. Sentiment:`	next token `positive`	none — just a prompt

The encoder needs a task-specific head and fine-tuning; the other two turn the task into **text generation**. The decoder-only version needs no fine-tuning at all if the model is large enough — the prompt alone defines the task.

Try it: the same task, three ways

Encoder (BERT-style)

INPUT

```
[CLS] the movie was great [SEP]
```

OUTPUT

```
class "positive" from a classifier on the [CLS] vector
```

NEEDS

fine-tuned classification head

Encoder–decoder (T5-style)

INPUT

```
sentiment: the movie was great
```

OUTPUT

```
positive
```

NEEDS

nothing extra; the label is generated as text

Decoder-only (GPT-style)

INPUT

```
Review: the movie was great. Sentiment:
```

OUTPUT

```
positive
```

NEEDS

nothing extra; just a prompt

The encoder produces labels or spans through a task-specific head; the encoder–decoder and decoder-only models write the answer as text.

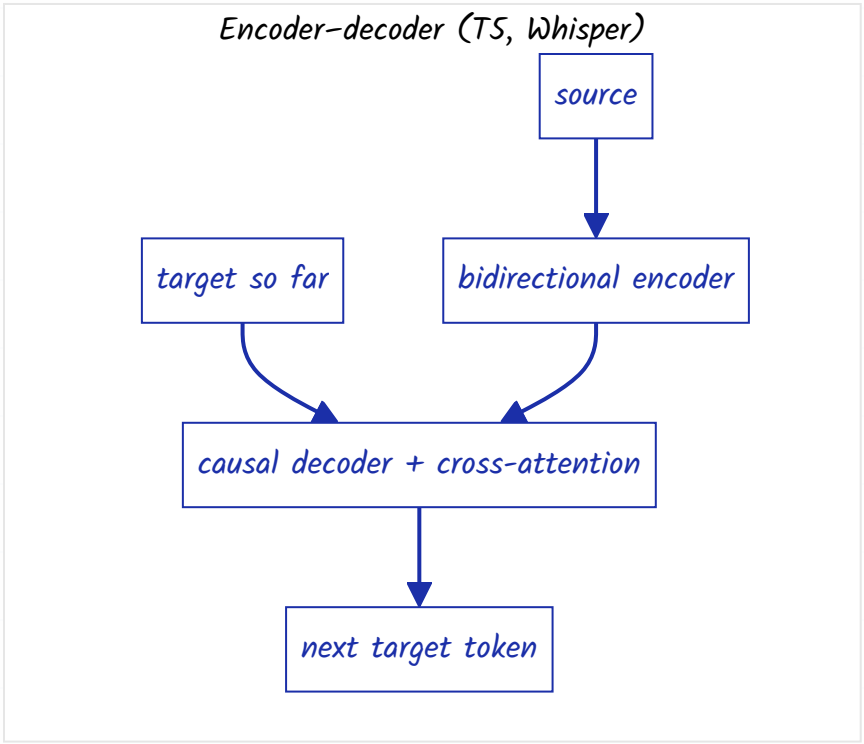
6.4 Training signals per sentence

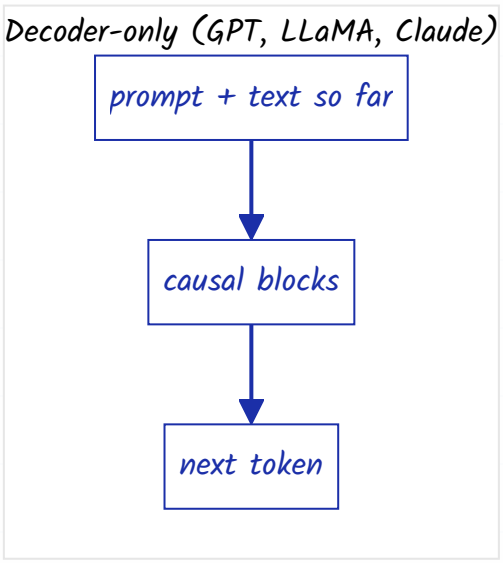
For a 20-token sentence:

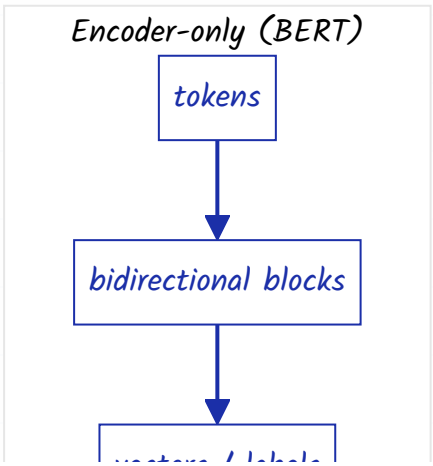
Objective	Supervised predictions
MLM (15%)	$0.15 \times 20 = 3$
Span corruption (15%)	about 3 tokens (in 1–2 spans)
Causal LM	**20** (21 counting the final `<eos>`)

Causal LM extracts the most supervision from each sentence — one reason it scales so well.

7. Diagram


Encoder-decoder (TS, Whisper)
source
target so far
bidirectional encoder
causal decoder + cross-attention
next target token


Decoder-only (GPT, LLaMA, Claude)
prompt + text so far
causal blocks
next token


Encoder-only (BERT)
tokens
bidirectional blocks

vectors / labels

8. In practice

Architecture	Examples	Best at	Weak at
Encoder-only	BERT, RoBERTa, DeBERTa, E5/BGE, ModernBERT	classification, tagging, embeddings, re-ranking	generating text
Decoder-only	GPT series, LLaMA, Mistral, Qwen, Claude	open-ended generation, chat, code, reasoning, few-shot tasks	overkill (large and slow) for simple labelling jobs
Encoder–decoder	T5 / Flan-T5, BART, mT5, Whisper	translation, summarization, speech-to-text	open-ended chat at scale
Prefix-LM / hybrids	UL2, GLM	mixes of understanding and generation	less common today

Choosing in practice:

If you need...	Choose
a label or a score for each input, cheaply, at high volume	**encoder** (fine-tuned)
a vector to search millions of documents	**encoder** (embedding model)
a fixed input→output transform, e.g. speech-to-text or translation, on limited hardware	**encoder–decoder**
open-ended answers, many different tasks, chat, tool use	**decoder-only LLM**

The lines are blurring: some recent embedding models are built from decoder LLMs, and decoder-only LLMs now translate and summarize very well.

9. Common mistakes and misconceptions

"Encoders and decoders use different kinds of blocks." They use the same block; the **mask** (and the objective) differ.

"Decoder-only models can't understand input; they only generate." The prompt *is* the input; causal self-attention over it provides understanding. Each prompt token just can't see later prompt tokens.

"Encoder–decoder is always better because it has both." It's better suited to some transformation tasks, but it's more complex and less convenient for chat and in-context learning. Most frontier LLMs are decoder-only.

"BERT is obsolete." Encoders are still the workhorse for search, classification and RAG retrieval because they're small, fast and accurate.

"Span corruption is the same as MLM." MLM predicts each hidden token in place with an encoder; span corruption removes whole spans and has a **decoder** generate them as a sequence.

10. Summary

- Encoder, decoder and encoder–decoder share the same block; they differ in **attention mask** and **training objective**.
- Masks: encoder **full** (n^2), decoder **causal** ($n(n+1)/2$), prefix-LM **full on the prompt, causal after**, encoder–decoder **full + causal + cross**.
- Objectives: **MLM** (~15% supervised), **causal LM** (100% supervised), **span corruption** (reconstruct removed spans).
- Any task can be framed as text generation; decoder-only LLMs win on simplicity, scaling, prompting/in-context learning and cacheable generation.
- Encoders still win for embeddings and labels; encoder–decoders for long-input → short-output transforms like translation and speech recognition.

11. Check your understanding

1. For $n = 6$ tokens with a prefix of $p = 4$, how many pairs are allowed under (a) bidirectional, (b) causal, and (c) prefix-LM attention?
2. Pick an architecture for each, with one reason: (a) a spam filter scanning one million emails a day; (b) a customer-support chatbot; (c) converting lecture recordings to text.
3. Write the input and target for **span corruption** on "the bat flew out of the cave" if the spans "flew out" and "cave" are removed.

▶ *Show answers*

12. Further reading

1. Devlin, J. et al. (2019). **"BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding."** arXiv:1810.04805 — the encoder-only recipe.
2. Radford, A. et al. (2019). **"Language Models are Unsupervised Multitask Learners."** OpenAI — GPT-2: tasks as text continuation.
3. Brown, T. et al. (2020). **"Language Models are Few-Shot Learners."** arXiv:2005.14165 — GPT-3 and in-context learning.
4. Raffel, C. et al. (2020). **"Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer."** arXiv:1910.10683 — T5, span corruption, and a systematic comparison of architectures and objectives.
5. Lewis, M. et al. (2019). **"BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension."** arXiv:1910.13461.
6. Wang, T. et al. (2022). **"What Language Model Architecture and Pretraining Objective Work Best for Zero-Shot Generalization?"** arXiv:2204.05832.

Training an LLM

> **In one line:** an LLM is made in stages — **pre-training** (predict the next token on trillions of tokens of text), then **supervised fine-tuning** (imitate good answers to instructions), then **preference tuning** (RLHF or DPO: prefer answers people like). Underneath all three is the same loop: forward pass → cross-entropy loss → backpropagation → AdamW update.

1. Learning objectives
2. Prerequisites
3. Motivation
4. Intuition first
5. The formal idea
6. Worked examples
7. Diagram
8. In practice
9. Common mistakes
10. Summary
11. Check your understanding
12. Further reading

1. Learning objectives

By the end of this section you should be able to:

1. **Describe** the three training stages and what data each one uses.
2. **Compute** cross-entropy loss and perplexity for a short sequence, and interpret them.
3. **Perform** one AdamW update by hand and compute a warm-up + cosine learning-rate schedule.
4. **Estimate** training compute ($\approx 6ND$), GPU-hours and memory for a real model, and explain the Chinchilla "20 tokens per parameter" rule.

5. **Explain** SFT loss masking, RLHF, and the DPO loss.

2. Prerequisites

- **Section 10:** teacher forcing — one forward pass gives a next-token loss at every position.
- **Section 8:** parameter counts (LLaMA-2 7B = 6.74 billion).
- **Gradient descent:** $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}$, with learning rate η.
- **Logarithms:** $\ln 1 = 0$, $\ln 0.5 = -0.693$; and $e^{\ln x} = x$.

> **Quick refresher — the sigmoid function**
> $\sigma(z) = \frac{1}{1+e^{-z}}$ squashes any number into $(0, 1)$: $\sigma(0) = 0.5$, large positive $z \rightarrow$ close to 1. It appears in the DPO loss.

3. Motivation: from random numbers to a helpful assistant

A freshly built Transformer has random weights; it outputs gibberish. Turning it into a helpful assistant takes three different kinds of learning:

Stage	What the model learns	Data	Rough scale
1. Pre-training	language, facts, reasoning patterns — "how text continues"	trillions of tokens of web pages, books, code	weeks–months on thousands of GPUs
2. Supervised fine-tuning (SFT)	to follow instructions and answer in a helpful format	~10k–1M (instruction, good response) pairs	hours–days
3. Preference tuning (RLHF / DPO)	to prefer responses people rate as better — more helpful, honest, harmless	pairs of responses ranked by people (or AI)	hours–days

A pre-trained-only model asked "What is the capital of France?" might continue with *"What is the capital of Germany? What is..."* — it's continuing text, not answering. Stages 2 and 3 teach it to **respond**.

4. Intuition first: training a doctor

- **Pre-training = years of reading.** A medical student reads thousands of textbooks and papers. They absorb a huge amount, but don't yet know how to talk to a patient.
- **SFT = residency.** They watch senior doctors handle real cases and imitate them: "when a patient says X, respond like this."
- **Preference tuning = feedback from patients and supervisors.** Two ways of explaining a diagnosis are compared; "this one was clearer and kinder" — and the doctor adjusts towards the preferred style.

Where the analogy breaks

The "reading" stage isn't passive. Every single token of text is a little exam — predict the next word, get graded, adjust — trillions of times.

5. The formal idea

5.1 The training loop

For each step:

1. Take a **batch** of token sequences (e.g. 1,000 sequences × 4,096 tokens ≈ 4 million tokens).
2. **Forward pass:** compute next-token probabilities at every position (teacher forcing).
3. **Loss:** average cross-entropy over all positions.
4. **Backward pass:** backpropagation computes the gradient of the loss for every parameter.
5. **Clip** the gradient if its overall size exceeds a threshold (e.g. 1.0) — protects against rare huge updates.
6. **Update** every parameter with **AdamW**, using the current **learning rate** from the schedule.

5.2 Loss and perplexity

For a sequence of T predictions:

$$\mathcal{L} = -\frac{1}{T}\sum_{t=1}^{T} \log P_\theta(x_{t+1} \mid x_{\le t}), \qquad \text{Perplexity} = e^{\mathcal{L}}$$

Perplexity reads as "the model is as uncertain as if it were choosing uniformly among this many tokens". A model that guesses uniformly over a 50,257-token vocabulary has loss $\ln 50{,}257 = 10.8$ and perplexity 50,257; good LLMs reach perplexities in the single digits on typical text.

5.3 AdamW

Plain gradient descent uses the same step size for every parameter. **Adam** keeps, for each parameter, a running average of its gradient (m, "momentum") and of its squared gradient (v), and scales the step by $1/\sqrt{v}$ — so parameters with consistently large gradients take smaller steps, and rarely-updated ones take bigger steps.

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)\, g_t, \qquad v_t = \beta_2 v_{t-1} + (1-\beta_2)\, g_t^2$$
$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t}$$
$$\theta_t = \theta_{t-1} - \eta \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda\, \theta_{t-1} \right)$$

- $\hat{m}, \hat{v}$ are **bias-corrected** (at the start m and v are 0, which would make them too small).
- **AdamW** applies **weight decay** λ directly to the weights ("decoupled"), gently pulling them towards zero.
- Typical LLM values: $\beta_1 = 0.9$, $\beta_2 = 0.95$, $\epsilon = 10^{-8}$ to 10^{-5}, $\lambda = 0.1$.

5.4 The learning-rate schedule

Two phases:

1. **Warm-up:** increase linearly from 0 to the peak over the first few thousand steps. Early on, Adam's statistics are unreliable and the weights are random; big steps would be unstable.
2. **Cosine decay:** smoothly decrease from the peak to a small minimum (often 10% of the peak):

$$\eta(s) = \eta_{\min} + \tfrac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos\left(\pi\, \frac{s - s_{\text{warm}}}{s_{\text{total}} - s_{\text{warm}}} \right) \right)$$

Large steps early make fast progress; small steps late let the model settle into a good minimum.

5.5 How much compute? $C \approx 6ND$

For a model with N parameters trained on D tokens, training costs about

$$C \approx 6\,N\,D \text{ floating-point operations (FLOPs)}$$

— roughly $2N$ per token for the forward pass and $4N$ for the backward pass.

Scaling laws and Chinchilla. Loss falls smoothly and predictably as N, D and C grow. For a *fixed* compute budget, research on "compute-optimal" training found that parameters and tokens should grow together, at about **20 training tokens per parameter**. Modern models often train far beyond this (e.g. 100+ tokens per parameter): it costs more training compute, but gives a smaller model for the same quality — cheaper to *run* for millions of users.

5.6 Memory for training

With AdamW and mixed precision, each parameter needs roughly:

Item	Bytes per parameter
weights (16-bit)	2
gradients (16-bit)	2
master copy of weights (32-bit)	4
Adam m (32-bit)	4
Adam v (32-bit)	4
Total	**≈ 16**

...plus memory for **activations** (intermediate values kept for the backward pass), which grows with batch size and sequence length. Large models don't fit on one GPU, so these states are sharded across many GPUs (Section 17).

5.7 Supervised fine-tuning (SFT)

Training examples are conversations: an **instruction** (prompt) and a high-quality **response**. The model is trained with the same next-token loss, **but only on the response tokens** — the prompt tokens are masked out of the loss (they're still in the input, as context). Otherwise the model would partly learn to imitate users' questions rather than the assistant's answers.

5.8 Preference tuning: RLHF and DPO

Both start from **preference data**: for a prompt x, two responses — a preferred one y_w ("winner") and a rejected one y_l ("loser").

RLHF (reinforcement learning from human feedback):

1. Train a **reward model** — usually an LLM with a scalar head — to score y_w above y_l.
2. Use reinforcement learning (typically PPO) to make the LLM produce high-reward responses, with a **KL penalty** that keeps it close to the SFT model (so it doesn't drift into strange text that fools the reward model).

DPO (direct preference optimization) skips the reward model and the RL loop. It directly increases the probability of y_w relative to y_l, compared with a frozen reference model π_{ref}:

$$\mathcal{L}_{\text{DPO}} = -\log \sigma\Big(\beta\Big[\underbrace{\log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)}}_{\text{how much more likely the winner became}} - \underbrace{\log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)}}_{\text{how much more likely the lo}}$$

In words: *push the winner's probability up and the loser's down, relative to where they started.* β (e.g. 0.1) controls how far the model may move from the reference.

6. Worked examples

6.1 Loss and perplexity

A model predicts 4 tokens, giving the correct ones probabilities 0.5, 0.25, 0.8 and 0.1:

Position	$P(\text{correct})$	$-\ln P$
1	0.50	0.693
2	0.25	1.386
3	0.80	0.223
4	0.10	2.303

$$\mathcal{L} = \frac{0.693 + 1.386 + 0.223 + 2.303}{4} = \mathbf{1.151}, \qquad \text{Perplexity} = e^{1.151} =$$

On average the model is "as unsure as choosing among about 3 tokens". Notice one confident mistake (position 4, $P = 0.1$) contributes as much loss as the other three combined.

At the start of training (uniform over 50,257 tokens): every $P = 1/50{,}257$, so $\mathcal{L} = \ln 50{,}257 = 10.82$ and perplexity 50,257.

Try it: how loss and perplexity react

P("the") 0.500

P("bat") 0.250

P("flew") 0.800

P("away") 0.100

loss = 1.151 perplexity = 3.16

Red bars show each position's –log P. The worst position ("away") contributes 50% of the total loss. Perplexity 3.16 means the model is about as unsure as picking among 3.2 equally likely tokens.

6.2 One AdamW step by hand

Parameter $\theta = 1.0$, gradient $g = 0.5$, learning rate $\eta = 0.1$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, weight decay $\lambda = 0.01$, first step ($t = 1$, $m_0 = v_0 = 0$):

1. $m_1 = 0.9(0) + 0.1(0.5) = 0.05$ and $v_1 = 0.999(0) + 0.001(0.25) = 0.00025$
2. Bias-correct: $\hat{m}_1 = 0.05/(1 - 0.9) = 0.5$ and $\hat{v}_1 = 0.00025/(1 - 0.999) = 0.25$
3. Adam step: $\eta \cdot \hat{m}_1/\sqrt{\hat{v}_1} = 0.1 \times 0.5/0.5 = 0.1$
4. Weight decay: $\eta\lambda\theta = 0.1 \times 0.01 \times 1.0 = 0.001$
5. New value: $\theta = 1.0 - 0.1 - 0.001 = \mathbf{0.899}$

Notice: the step size ($0.1 = \eta$) didn't depend on the gradient's *size* — a gradient of 50 or 0.005 would give the same first step. Adam normalizes each parameter's step, which is why one learning rate works across very different layers.

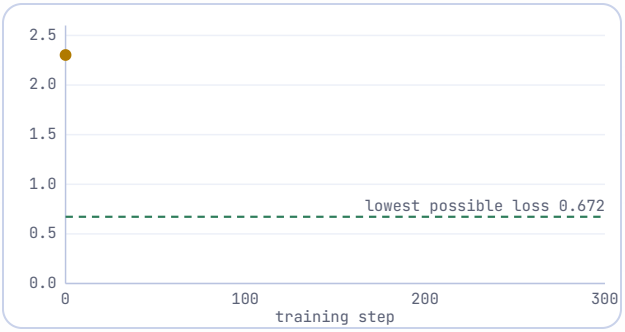
6.3 A learning-rate schedule

Peak 3×10^{-4}, warm-up 2,000 steps, 100,000 steps in total, cosine decay to 3×10^{-5} (10% of peak):

Step	Phase	Learning rate
1,000	warm-up, halfway	1.5×10^{-4}
2,000	peak	3.0×10^{-4}
51,000	halfway through decay ($\cos(\pi/2) = 0$)	$\frac{3\times10^{-4}+3\times10^{-5}}{2} = 1.65 \times 10^{-4}$
100,000	end	3.0×10^{-5}

Try it: train a tiny language model in your browser

A real (but tiny) model: a bigram table of 10 × 10 learnable scores, trained with next-token cross-entropy and AdamW on the five sentences below. Everything is computed live.



Training text

```
<bos> the bat flew <eos>
<bos> the bat hit the ball <eos>
<bos> a bat flew at night <eos>
<bos> the ball flew <eos>
<bos> a ball hit the bat <eos>
```

step	0 / 300
learning rate now	0.0033
loss	2.303
perplexity	10.00
gap to lowest possible	1.631

Text the model writes now (greedy, then sampled)

1. `ball a flew night the the the the`
2. `a night flew at night the`
3. `bat hit bat bat at a the`
4. `ball at flew a flew at bat at`

Untrained: the scores are random, so every next word is roughly equally likely (loss ≈ ln 10 = 2.30) and the generated text is nonsense. Press “Train 300 steps”.

6.4 How expensive was LLaMA-2 7B?

$N = 6.74 \times 10^9$ parameters, $D = 2 \times 10^{12}$ tokens:

$$C \approx 6 \times 6.74 \times 10^9 \times 2 \times 10^{12} = 8.1 \times 10^{22} \text{ FLOPs}$$

An NVIDIA A100 GPU peaks at about 312×10^{12} FLOP/s in 16-bit; real training achieves perhaps 40% of that, i.e. 1.25×10^{14} FLOP/s:

$$\frac{8.1 \times 10^{22}}{1.25 \times 10^{14}} = 6.5 \times 10^{8} \text{ GPU-seconds} \approx \mathbf{180{,}000} \text{ GPU-hours}$$

Meta reported **184,320** A100 GPU-hours for this model — the simple $6ND$ estimate lands within a few percent. On 2,048 GPUs, that's about 3.7 days of continuous running… for the smallest LLaMA-2.

Chinchilla check: $20 \times 6.74 \times 10^{9} \approx 135$ billion tokens would be "compute-optimal"; LLaMA-2 7B used 2 trillion — about **15×** more, trading extra training compute for a cheaper-to-run model.

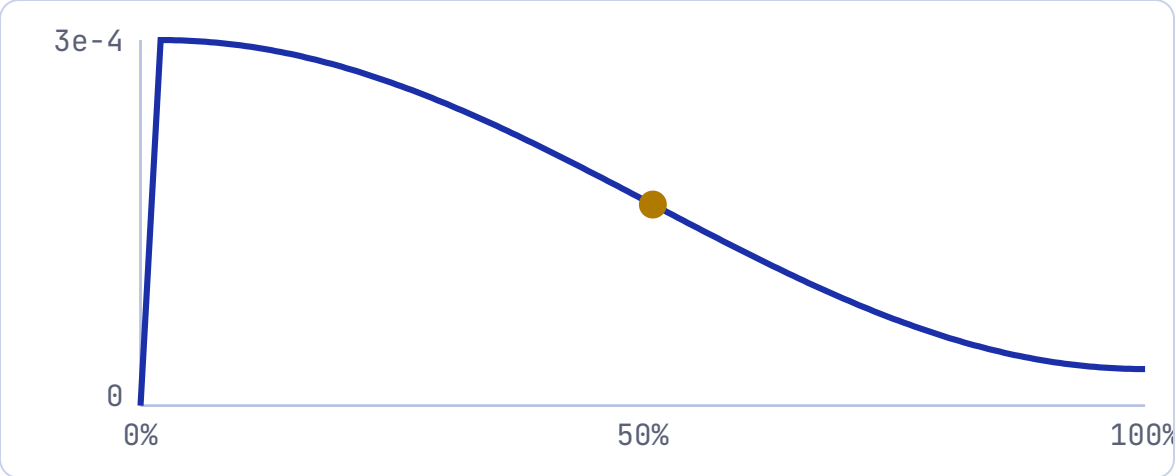
Memory check: $16 \times 6.74 \times 10^{9} \approx 108$ GB of weights, gradients and optimizer states — more than one 80 GB GPU holds, before counting activations. Hence sharding.

Try it: schedule and compute calculator

Quantity	Value
Training compute C ≈ 6ND	`8.09 × 10^22 FLOPs`
Effective speed per GPU	`1.25 × 10^14 FLOP/s`
GPU-hours	`180,021`
Wall-clock time on 2,048 GPUs	`3.7 days`
Chinchilla-optimal tokens (20 N)	`135 B — you are using 14.8× that`
Weights + grads + Adam states (16 bytes/param)	`108 GB (plus activations)`

Meta reported 184,320 A100 GPU-hours for LLaMA-2 7B. The estimate is within 2.3%.

Learning-rate schedule



At 51% of training the learning rate is 1.74e-4. With a 100,000-step run, peak 3e-4, 2% warm-up and 10% final, step 51% gives 1.65e-4 — compare 6.3.

6.5 SFT loss masking

Example: prompt *"What is 2 + 2?"* (5 tokens), response *"2 + 2 = 4."* (7 tokens).

- Positions predicting **prompt** tokens: in the input, **excluded** from the loss.

- Positions predicting the 7 **response** tokens: **included**.

The loss is the average over just those 7 positions.

6.6 One DPO step

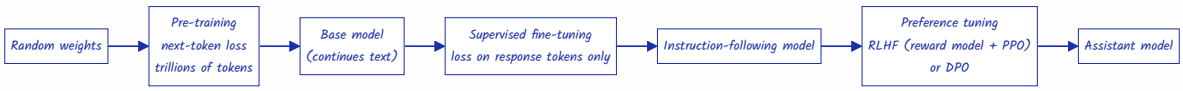
$\beta = 0.1$. After some training, the winner's log-probability has risen by 2.0 relative to the reference, and the loser's has fallen by 1.0:

- Margin inside the brackets: $2.0 - (-1.0) = 3.0$; times β: 0.3
- $\sigma(0.3) = 0.574$, so $\mathcal{L}_{\text{DPO}} = -\ln 0.574 = \mathbf{0.554}$

At the start (policy = reference, both log-ratios 0): $\sigma(0) = 0.5$ and $\mathcal{L} = \ln 2 = 0.693$. The loss falls as the model separates winners from losers.

> **Try it: the DPO loss**
>
> margin = 0.10 × (2.0 − (−1.0)) = 0.30 σ = 0.574
>
> loss = 0.554
>
> The winner has gained more probability than the loser, so the loss is below ln 2 = 0.693. Pushing them further apart lowers it more.

7. Diagram



8. In practice

Model	Parameters	Training tokens	Tokens per parameter
GPT-3 (2020)	175 B	300 B	≈ 1.7
Chinchilla (2022)	70 B	1.4 T	20
LLaMA-2 7B (2023)	6.74 B	2 T	≈ 300
LLaMA-3 8B (2024)	8 B	15 T	≈ 1,900

The trend moved from "big model, fewer tokens" to "smaller model, far more tokens" — because inference cost, paid for every user request, came to dominate.

Typical pre-training settings (LLaMA-2): AdamW with $\beta_1 = 0.9$, $\beta_2 = 0.95$; peak learning rate 3×10^{-4}; 2,000 warm-up steps; cosine decay to 10%; weight decay 0.1; gradient clipping at 1.0; batches of about 4 million tokens; context length 4,096.

Data work is most of the work: removing duplicates, filtering low-quality and unsafe pages, balancing sources (web, code, books, maths, many languages), and removing benchmark test sets from the training data. Data quality often matters more than small architecture changes.

Training instabilities — sudden loss spikes — are handled with gradient clipping, careful learning rates, restarting from a checkpoint before the spike, and skipping problematic batches.

9. Common mistakes and misconceptions

> **"Pre-training teaches the model to be an assistant."** Pre-training teaches it to *continue text*. Following instructions comes from SFT; helpful, safe preferences from RLHF/DPO.

> **"Lower loss always means a better chatbot."** Pre-training loss measures text prediction. Helpfulness, honesty and safety need separate evaluation.

> **"More parameters is always the best use of compute."** For a fixed budget, parameters and data should grow together (Chinchilla); for cheap deployment, a smaller model trained on more tokens is often better.

> **"In SFT the model learns from the whole conversation."** The loss is usually computed only on the assistant's response tokens.

> **"DPO is reinforcement learning."** DPO turns preference learning into a simple classification-style loss on log-probabilities — no reward model, no sampling loop.

10. Summary

- Three stages: **pre-training** (next-token prediction on huge text corpora) → **SFT** (imitate good responses; loss on response tokens only) → **preference tuning** (RLHF with a reward model + PPO, or DPO).
- Loss = average $-\log P$ of the correct next tokens; **perplexity** $= e^{\mathcal{L}}$.
- **AdamW** normalizes each parameter's step and applies decoupled weight decay; learning rate follows **warm-up + cosine decay**.
- Compute $\approx 6ND$ FLOPs (LLaMA-2 7B ≈ 8×10^{22} ≈ 180k GPU-hours); memory ≈ 16 bytes/parameter + activations.
- **Chinchilla:** compute-optimal ≈ 20 tokens/parameter; modern models deliberately train far longer to be cheaper at inference.

11. Check your understanding

1. A model gives the correct next tokens probabilities 0.9, 0.9, 0.9 and 0.01. Compute the loss and perplexity. What does this show about confident mistakes?
2. Using $C \approx 6ND$, how many FLOPs did GPT-3 (175 B parameters, 300 B tokens) need? About how many tokens would Chinchilla's rule suggest for a

175 B model?

3. In 6.2, what would the first AdamW step be if the gradient were $g = 50$ instead of 0.5 (ignore weight decay)? Why?

▶ *Show answers*

12. Further reading

1. Kaplan, J. et al. (2020). **"Scaling Laws for Neural Language Models."** arXiv:2001.08361.
2. Hoffmann, J. et al. (2022). **"Training Compute-Optimal Large Language Models."** arXiv:2203.15556 — the Chinchilla paper.
3. Loshchilov, I., & Hutter, F. (2019). **"Decoupled Weight Decay Regularization."** *ICLR 2019.* arXiv:1711.05101 — AdamW.
4. Touvron, H. et al. (2023). **"Llama 2: Open Foundation and Fine-Tuned Chat Models."** arXiv:2307.09288 — pre-training settings, SFT and RLHF in detail.
5. Ouyang, L. et al. (2022). **"Training language models to follow instructions with human feedback."** arXiv:2203.02155 — InstructGPT / RLHF.
6. Rafailov, R. et al. (2023). **"Direct Preference Optimization: Your Language Model is Secretly a Reward Model."** arXiv:2305.18290.

Prepared by Akhil

Inference & Decoding

> **In one line:** at each step an LLM outputs a **probability for every token in its vocabulary**; a **decoding strategy** — greedy, beam search, or sampling shaped by temperature, top-k, top-p or min-p — decides which token is actually written. The same model can sound robotic, creative or incoherent depending only on these choices.

1. Learning objectives
2. Prerequisites
3. Motivation
4. Intuition first
5. The formal idea
6. Worked examples
7. Diagram
8. In practice
9. Common mistakes
10. Summary
11. Check your understanding
12. Further reading

1. Learning objectives

By the end of this section you should be able to:

1. **Describe** the inference loop, and the difference between the prefill and decode phases.
2. **Apply** temperature to logits and explain its effect on the distribution.
3. **Apply** top-k, top-p (nucleus) and min-p filtering to a distribution by hand.
4. **Run** beam search on a small example and explain when it beats greedy decoding.
5. **Choose** sensible decoding settings for tasks like code, chat and creative writing, and estimate response latency.

2. Prerequisites

- **Section 10:** autoregressive generation — one forward pass per new token, appended to the input.
- **Section 5/12:** softmax turns scores (**logits**) into probabilities; $\log$-probabilities add when probabilities multiply.

> **Quick refresher — logits**
> The model's output head produces one raw score per vocabulary token, called a **logit** z_i. Softmax turns them into probabilities $p_i = e^{z_i} / \sum_j e^{z_j}$. Only the *differences* between logits matter: adding the same number to every logit changes nothing.

3. Motivation: the model doesn't pick words — you do

After reading *"The monsoon rain"*, a model might output:

Token	fell	poured	stopped	began	was	danced
Probability	0.43	0.26	0.16	0.10	0.06	0.01

The model has done its job: it has said how likely each continuation is. Now *we* must choose. Different choices lead to very different behaviour:

- Always take the top token → safe, but repetitive and sometimes stuck in loops.
- Sample in proportion to the probabilities → varied, but occasionally picks something odd like "danced".
- Something in between → usually best.

Decoding is how we control the trade-off between **reliability** and **diversity**.

4. Intuition first: ordering at a restaurant

You visit a new restaurant and the waiter tells you how popular each dish is.

- **Greedy:** always order the most popular dish. Safe; boring after a week.
- **Temperature:** how adventurous you feel today. Low temperature → stick to favourites; high → anything goes.
- **Top-k:** "only consider the top 3 dishes".
- **Top-p:** "only consider the dishes that together make up 90% of orders" — sometimes that's 2 dishes, sometimes 6, depending on how clear the favourites are.
- **Beam search:** planning a whole three-course meal by keeping your best few *combinations* in mind, rather than choosing each course in isolation.

> **Where the analogy breaks**
>
> A person's taste is stable; the model's distribution changes after every single token, because each new word changes the context.

5. The formal idea

5.1 The inference loop and its two phases

1. **Prefill:** the whole prompt is processed in **one parallel pass**. Its keys and values are stored (KV cache, Section 14). The time until the first token appears is the **time to first token (TTFT)**.
2. **Decode:** then one token per forward pass: get logits → apply the decoding strategy → append the chosen token → repeat until a stop condition.

Stop conditions: an end-of-sequence token, a maximum number of new tokens, or a user-defined **stop sequence** (e.g. `"\n\nUser:"`).

5.2 Greedy decoding

$$x_{t+1} = \arg\max_i p_i$$

Deterministic and fast. Good for short, factual or structured outputs. Weaknesses: can **repeat itself** in loops, and choosing the best token *now* doesn't always lead to the best *sequence*.

5.3 Temperature

Divide the logits by a temperature $T > 0$ before the softmax:

$$p_i(T) = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$$

T	Effect
$T \to 0$	all mass on the top token (same as greedy)
$T < 1$	sharper: likely tokens get likelier
$T = 1$	the model's own distribution
$T > 1$	flatter: unlikely tokens get a bigger share

Temperature never changes the **ranking** of tokens, only how peaked the distribution is. (It's the same "divide the scores" idea as $\sqrt{d_k}$ in Section 5.)

5.4 Truncation: top-k, top-p, min-p

Sampling from the *full* distribution occasionally picks a token from the long tail of thousands of unlikely tokens — and one bad token can derail a whole answer. Truncation removes the tail, then **renormalizes** what's left so it sums to 1.

- **Top-k:** keep the k most likely tokens.
- **Top-p (nucleus):** sort tokens by probability and keep the smallest set whose cumulative probability reaches p (e.g. 0.9). The set size **adapts**: few tokens when the model is confident, many when it's unsure.

- **Min-p:** keep every token whose probability is at least $\text{min_p} \times p_{\max}$. Also adaptive, and it behaves well at high temperatures.

A common order of operations: **logits → penalties → temperature → truncation → renormalize → sample**.

Try it: the decoding playground — "The monsoon rain …"

TOKEN	AFTER TEMPERATURE		KEPT?	FINAL (RENORMALIZED)	
fell		0.425	✓		0.425
poured		0.258	✓		0.258
stopped		0.156	✓		0.156
began		0.095	✓		0.095
was		0.058	✓		0.058
danced		0.008	✓		0.008

Kept 6 tokens (fell, poured, stopped, began, was, danced), holding 100.0% of the probability before renormalizing. Greedy would always write “fell”. Press “Sample 1,000 times” to check the final column against real samples.

5.5 Beam search

Instead of committing to one token at a time, keep the B **best partial sequences** ("beams"), scored by their total log-probability:

1. Start with B copies of the prompt.
2. Extend every beam by every possible next token.
3. Keep the B highest-scoring extensions.
4. Repeat until all beams finish; return the best.

Beam search finds higher-probability sequences than greedy. It is standard for **translation** and **speech recognition**, where there's roughly one right answer. For open-ended chat it tends to produce bland, generic, repetitive text — the most *probable* text isn't the most *interesting* — so sampling is preferred there.

Because adding more tokens can only add negative log-probabilities, beam search favours short outputs; implementations usually apply a **length penalty** (divide the score by length$^{\alpha}$).

5.6 Penalties against repetition

- **Presence penalty:** subtract a fixed amount from the logit of any token that has already appeared.
- **Frequency penalty:** subtract an amount proportional to how many times it has appeared.
- **Repetition penalty:** divide positive logits (multiply negative ones) of already-seen tokens by a factor such as 1.1–1.3.

They discourage loops but, if too strong, stop the model from repeating words it legitimately needs (names, code variables).

5.7 Constrained (structured) decoding

To guarantee valid output — e.g. JSON matching a schema, or one of a fixed set of labels — set the logits of every token that would break the format to $-\infty$ at each step (a mask again!). The model can only ever choose tokens that keep the output valid.

6. Worked examples

6.1 Temperature on "The monsoon rain..."

Logits: fell 3.0, poured 2.5, stopped 2.0, began 1.5, was 1.0, danced −1.0.

Token	logit	$T = 0.5$	$T = 1$	$T = 2$
fell	3.0	**0.636**	0.425	0.298
poured	2.5	0.234	0.258	0.232
stopped	2.0	0.086	0.156	0.180
began	1.5	0.032	0.095	0.141
was	1.0	0.012	0.058	0.109
danced	−1.0	0.000	0.008	**0.040**

How to compute (for $T = 1$): $e^{3.0} = 20.09$, $e^{2.5} = 12.18$, $e^{2.0} = 7.39$, $e^{1.5} = 4.48$, $e^{1.0} = 2.72$, $e^{-1} = 0.37$; total 47.23; "fell" $= 20.09/47.23 = 0.425$.

At $T = 0.5$ "fell" dominates; at $T = 2$ even "danced" gets 4%. The ranking never changes.

6.2 Top-k, top-p and min-p (at $T = 1$)

Top-k, $k = 3$: keep fell, poured, stopped (total 0.84) and renormalize: $0.425/0.84 = 0.51$, $0.258/0.84 = 0.31$, $0.156/0.84 = 0.19$.

Top-p, $p = 0.9$: add up in order: fell 0.425 → +poured 0.683 → +stopped 0.839 → +began **0.935** ≥ 0.9, stop. Keep **4** tokens; renormalize by 0.935: fell 0.45, poured 0.28, stopped 0.17, began 0.10.

Min-p, 0.1: threshold $= 0.1 \times 0.425 = 0.0425$. Keep every token with $p \geq 0.0425$ — fell, poured, stopped, began, **was** (0.058) — and drop only "danced" (0.008).

All three remove "danced"; they differ in how much of the middle they keep.

6.3 Beam search beats greedy

A two-step toy problem. First-word probabilities: **the 0.5**, a 0.4, my 0.1. Second-word probabilities depend on the first:

After...	cat	dog	end
the	0.4	0.3	0.3
a	**0.9**	0.05	0.05
my	0.5	0.5	—

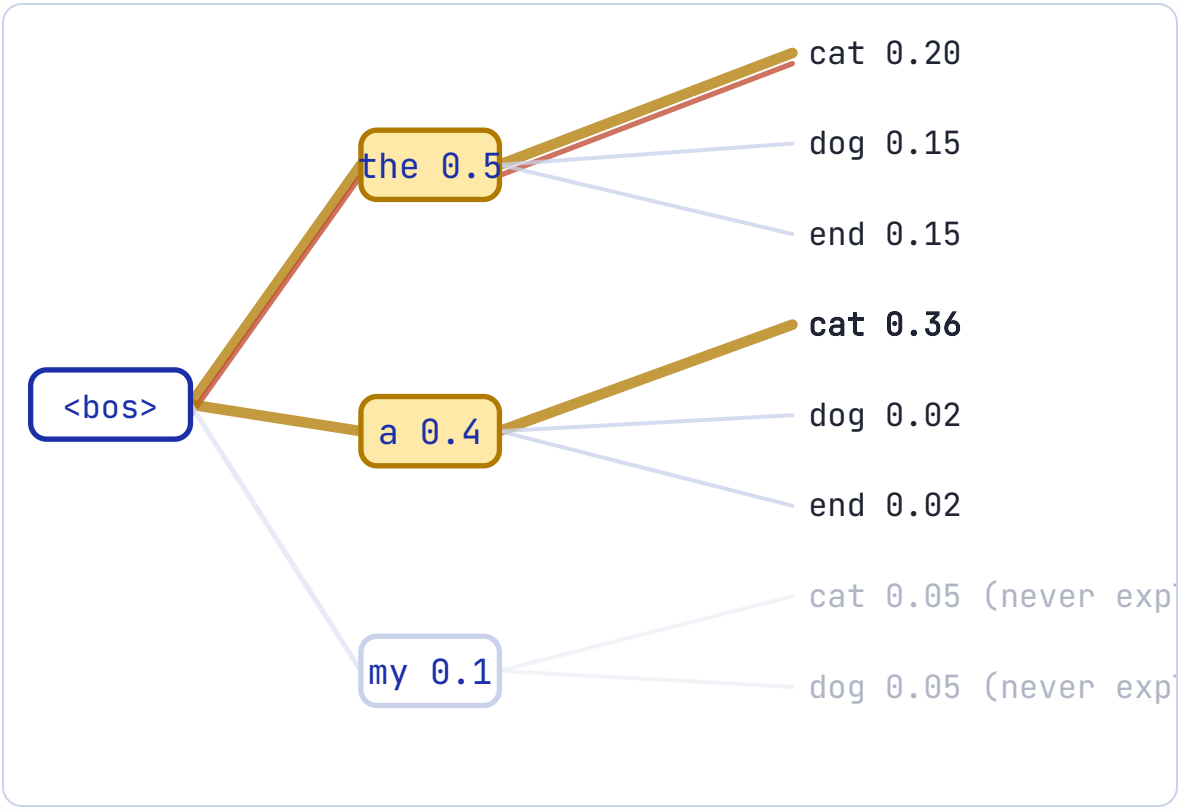
Greedy: step 1 picks "the" (0.5); step 2 picks "cat" (0.4) → "the cat" with probability $0.5 \times 0.4 = 0.20$.

Beam search, $B = 2$:

- Step 1: keep the two best: "the" (0.5) and "a" (0.4).
- Step 2: extend both: the cat 0.20, the dog 0.15, the end 0.15, **a cat 0.36**, a dog 0.02, a end 0.02.
- Keep the best two: **"a cat" (0.36)** and "the cat" (0.20).

Beam search finds "a cat", nearly twice as probable as greedy's answer. Greedy was misled by a slightly better first word that led to weaker continuations. In log-probabilities: $\ln 0.36 = -1.02$ beats $\ln 0.20 = -1.61$.

Try it: beam search vs greedy



B = 2: after step 1 it keeps the + a, expands all of them, and keeps the best 2. Result: “a cat” (0.36, log-prob -1.02). Greedy’s answer: “the cat” (0.20).

6.4 A presence penalty

Suppose "fell" has already been written once and the presence penalty is 0.5. Its logit becomes $3.0 - 0.5 = 2.5$, tying with "poured". New probabilities at $T = 1$: fell 0.31, poured 0.31, stopped 0.19, began 0.11, was 0.07, danced 0.01. The model is now less likely to repeat itself.

Try it: temperature over whole sentences

The toy next-word model from Section 10 (after "the": bat 0.40, ball 0.30, ...), generating 20 sentences from <bos> by sampling.

1. the bat bat
2. the bat flew
3. (empty)
4. the the
5. the bat flew ball
6. a bat hit the bat hit a
7. ball hit a bat flew
8. the flew bat hit the bat the flew hit the bat flew
9. ball flew hit the ball hit
10. the bat flew
11. a ball
12. the flew ball the bat flew the the bat flew the bat

different sentences	**16 / 20**
average log-prob per token (under the model)	**-1.48**
most common	**the bat flew**

Moderate temperature: a mix of the common sentence and plausible variations.

6.5 How long will the answer take?

A 500-token prompt and a 200-token answer, on a server that prefills 5,000 tokens/s and decodes at 30 ms per token:

- **TTFT:** $500/5{,}000 = 0.1$ s
- **Decode:** $200 \times 0.03 = 6.0$ s
- **Total:** about **6.1 s** — almost all of it spent generating, one token at a time.

That's why answers are **streamed**: users start reading after 0.1 s instead of waiting 6 s.

Try it: where does the time go?

```
decode 6.0 s
```

first token after 0.10 s · full answer after 6.1 s · 33 tokens/s

7. Diagram



8. In practice

Task	Typical settings	Why
Code, maths, extraction, classification	temperature 0–0.2 (near greedy)	one correct answer; consistency matters
General chat	temperature 0.6–0.8, top-p 0.9–0.95	natural, varied but focused
Brainstorming, stories	temperature 0.9–1.2, top-p 0.95 or min-p 0.05	more diversity
Translation, speech-to-text	beam search, B = 4–5	find the single most likely output

- **Determinism:** temperature 0 (or a fixed random seed) gives repeatable outputs in principle, but GPU arithmetic and batching can still cause tiny differences between runs.

- **Speculative decoding:** a small fast "draft" model proposes several tokens; the big model checks them all in one pass and keeps the ones it agrees with. Same output distribution, often 2–3× faster (Section 17).
- **Sampling several answers** and picking the most common final answer ("self-consistency") improves accuracy on reasoning problems, at extra cost.
- **Latency metrics:** TTFT (time to first token) and time per output token / tokens per second are the numbers serving systems optimize (Section 18).

9. Common mistakes and misconceptions

"Temperature 0 makes the model more accurate." It makes it more *consistent*. If the most likely token is wrong, greedy decoding is consistently wrong.

"Higher temperature makes the model more creative." It makes it more *random*. Past a point, output becomes incoherent rather than creative.

"Top-k and top-p remove bad tokens and keep the probabilities as they were." The survivors are **renormalized** to sum to 1, so each one's probability goes up.

"Beam search is always better than sampling." Beam search maximizes probability, which suits translation; for open-ended text it produces bland, repetitive output.

"The model chooses its words." The model only outputs a distribution. The decoding strategy — set by the developer — makes the choice.

10. Summary

- Each decode step turns **logits** into a choice: penalties → **temperature** → **truncation** (top-k / top-p / min-p) → renormalize → sample (or argmax).
- **Temperature** reshapes the distribution (sharper below 1, flatter above 1) without changing the ranking.
- **Top-k** keeps a fixed number of tokens; **top-p** and **min-p** keep an adaptive set based on probability.
- **Beam search** tracks the B best partial sequences and can beat greedy (0.36 vs 0.20 in our example); great for translation, bland for chat.
- Inference = **prefill** (parallel, sets TTFT) + **decode** (one token per pass, dominates total time); responses are streamed.

11. Check your understanding

1. Using the $T = 1$ probabilities in 6.1, which tokens are kept by top-p with $p = 0.7$? What are their renormalized probabilities?
2. Two logits are 4.0 and 2.0. Compute their probabilities at $T = 1$ and at $T = 2$ (consider only these two tokens).
3. In 6.3, what would beam search with $B = 1$ return? With $B = 3$?

▶ *Show answers*

12. Further reading

1. Holtzman, A. et al. (2020). **"The Curious Case of Neural Text Degeneration."** *ICLR 2020.* arXiv:1904.09751 — introduces nucleus (top-p) sampling and explains why beam search degenerates for open-ended text.
2. Fan, A., Lewis, M., & Dauphin, Y. (2018). **"Hierarchical Neural Story Generation."** arXiv:1805.04833 — popularized top-k sampling.
3. Keskar, N. S. et al. (2019). **"CTRL: A Conditional Transformer Language Model for Controllable Generation."** arXiv:1909.05858 — the repetition

penalty.

4. Leviathan, Y., Kalman, M., & Matias, Y. (2023). **"Fast Inference from Transformers via Speculative Decoding."** arXiv:2211.17192.
5. Wang, X. et al. (2023). **"Self-Consistency Improves Chain of Thought Reasoning in Language Models."** arXiv:2203.11171.

Prepared by Akhil

KV Cache & Fast Attention

> **In one line:** during generation, a token's keys and values never change once computed, so we **cache** them instead of recomputing the whole prefix every step. That makes decoding fast but turns memory into the bottleneck — solved with smaller caches (GQA, quantization, sliding windows, paging) and with **FlashAttention**, which computes exact attention without ever storing the $n \times n$ score matrix.

1. Learning objectives
2. Prerequisites
3. Motivation
4. Intuition first
5. The formal idea
6. Worked examples
7. Diagram
8. In practice
9. Common mistakes
10. Summary
11. Check your understanding
12. Further reading

1. Learning objectives

By the end of this section you should be able to:

1. **Explain** why keys and values can be cached during generation, and count the work saved.
2. **Calculate** the size of a KV cache for a real model, context length and batch size.
3. **Explain** why decoding is limited by memory bandwidth, and estimate a lower bound on time per token.

4. **Describe** the main ways to shrink the cache: GQA/MQA, quantization, sliding windows, PagedAttention, prefix caching.
5. **Compute** softmax in chunks with the **online softmax** trick, and explain how FlashAttention uses it.

2. Prerequisites

- **Section 10:** generation runs one forward pass per new token; causal attention means earlier tokens never see later ones.
- **Section 6:** GQA — groups of query heads share key/value heads.
- **Section 5:** attention scores, softmax, and the "subtract the max" trick for numerical stability.
- **Section 13:** prefill vs decode.

> **Quick refresher — bandwidth vs compute**
>
> A GPU can do arithmetic extremely fast (hundreds of trillions of operations per second), but it can only **read memory** at a few terabytes per second. If a job needs very few calculations per byte it reads, it is **memory-bound**: its speed is set by how fast data can be loaded, not by how fast the GPU can calculate.

3. Motivation: generation repeats itself

Without any caching, generating token $t + 1$ means running the model on **all** t previous tokens again — recomputing every key and value for the whole prefix — only to use the output at the last position.

But with a causal mask, the key and value of token j depend only on tokens $1 \dots j$. Adding new tokens later **never changes them**. Recomputing them is pure waste.

Generating 500 tokens after a 1,000-token prompt	K/V computations (per layer, per head)
Without caching	$1000 + 1001 + \cdots + 1499 = 624{,}750$
With a KV cache	$1000 + 499 = 1{,}499$ (≈ **417× fewer**)

So every LLM server keeps a **KV cache**. But the cache itself costs memory — often more than the model weights — and that creates the next set of problems.

4. Intuition first: a running class notebook

A teacher dictates a long passage, one sentence at a time, and after each sentence asks the class a question about everything so far.

- **No cache:** after every new sentence, students rewrite their notes on the *whole* passage from scratch before answering. Exhausting, and it gets slower with every sentence.
- **KV cache:** students keep a **notebook**. For each new sentence they add one new line of notes, then answer by reading back through the notebook. Fast — but the notebook keeps growing, and with 100 students' notebooks the shelf fills up.
- **FlashAttention:** instead of spreading the whole notebook across a big table to read it (which needs a huge table), a student reads it **one page at a time**, keeping a small running tally — and gets exactly the same answer.

Where the analogy breaks

The "notebook" isn't a summary; it's an exact copy of every past token's key and value vectors, in every layer and every key/value head.

5. The formal idea

5.1 Decoding with a KV cache

Prefill: run the whole prompt once; store each layer's K and V (shape $n_{\text{prompt}} \times d_k$ per KV head) in the cache.

Each decode step, in every layer:

1. Compute q_t, k_t, v_t for the **new token only**.
2. Append k_t, v_t to the cache.
3. Attend: $\text{softmax}\left(q_t K_{\text{cache}}^{\top} / \sqrt{d_k}\right) V_{\text{cache}}$ — one query row against all cached keys.

The output is **identical** to recomputing everything; the cache just avoids redundant work. Per step, the cost of the projections no longer grows with the context; the attention itself still reads all t cached keys and values.

Try it: generation with and without a KV cache

without cache p1 p2 p3 p4 p5

with KV cache p1 p2 p3 p4 p5

K/V recomputed this step read from cache new token: computed once, then cached

tokens generated	0
K/V computations without cache	0
K/V computations with cache	0
saving	NaN×

A 5-token prompt. Press “Generate one token”: the first pass (prefill) reads the whole prompt and produces generated token g1.

5.2 How big is the cache?

$$\text{bytes} = \underbrace{2}_{K \text{ and } V} \times \text{layers} \times \text{KV heads} \times d_{\text{head}} \times \text{bytes per number} \times \text{tok}$$

It grows **linearly** with context length and with the number of simultaneous users.

5.3 Why decoding is memory-bound

At each decode step with batch size 1, the GPU must **read every weight once** (and the whole KV cache) to produce **one** token. That's only about 2 floating-point operations per weight read — far too few to keep the GPU's arithmetic units busy. So:

$$\text{time per token} \gtrsim \frac{\text{bytes of weights} + \text{bytes of KV cache}}{\text{memory bandwidth}}$$

Batching helps enormously: one read of the weights can serve many users' tokens at once, so throughput rises almost linearly with batch size — until the KV caches of all those users fill the GPU's memory. That's why cache size matters so much.

5.4 Shrinking the cache

Technique	Idea	Saving
GQA / MQA (Section 6)	fewer key/value heads, shared by groups of query heads	h/g (e.g. 4–8×)
KV quantization	store cached values in 8-bit (or fewer) instead of 16-bit	2× or more
Sliding-window attention	each layer attends only to the last W tokens, so older entries are dropped	cache capped at W tokens
PagedAttention	store the cache in fixed-size blocks ("pages") anywhere in memory, like an OS's virtual memory	almost no wasted (fragmented) memory → more users per GPU
Prefix caching	reuse the cached K/V of a shared prompt (e.g. a long system prompt) across requests	skip repeated prefill

5.5 FlashAttention: exact attention without the $n \times n$ matrix

Standard attention writes the full score matrix $S = QK^\top$ ($n \times n$) to GPU main memory, reads it back for the softmax, writes A, reads it again for AV. For long sequences that's a lot of memory **traffic** — and memory traffic, not arithmetic, is the slow part.

FlashAttention splits Q, K and V into **tiles** small enough to fit in the GPU's tiny but very fast on-chip memory (SRAM), and processes the keys/values tile by tile. The problem: softmax needs the maximum and the sum over the **whole** row before any weight is known. The solution is the **online softmax**.

Online softmax. Process scores in chunks, keeping three running quantities per query row:

- m: the largest score seen so far,
- ℓ: the sum of e^{s-m} so far,

- o: the sum of $e^{s-m}\,v$ so far.

When a new chunk arrives with a larger maximum m', rescale the old totals by $e^{m-m'}$, then add the new chunk's contributions. At the end, the output is o/ℓ — **exactly** the same as normal softmax attention.

Results:

- **Memory:** $O(n)$ instead of $O(n^2)$ — the full score matrix is never stored.
- **Speed:** typically 2–4× faster in practice, because far less data moves between memory and the chip.
- **Exactness:** not an approximation; same output up to floating-point rounding.

5.6 Beyond full attention (brief)

Other ideas trade some exactness or generality for lower cost: **sparse attention** (each token attends to a selected subset), **linear attention** (rewrite attention to avoid the n^2 term), and **state-space / recurrent models** (e.g. Mamba) with a fixed-size state instead of a growing cache. Several recent models mix such layers with standard attention.

6. Worked examples

6.1 One decode step with a cache

Prompt "the bat" (2 tokens). After prefill, each layer's cache holds $k_{\text{the}}, k_{\text{bat}}$ and $v_{\text{the}}, v_{\text{bat}}$.

To generate the next token after "flew" has just been chosen:

1. Compute $q_{\text{flew}}, k_{\text{flew}}, v_{\text{flew}}$ — **one** new set of projections.
2. Cache becomes $K = [k_{\text{the}}; k_{\text{bat}}; k_{\text{flew}}]$, V likewise.
3. Attention for "flew": one row of scores $q_{\text{flew}} \cdot [k_{\text{the}}, k_{\text{bat}}, k_{\text{flew}}]$ → softmax → mix the three cached values.

Small count: generating 3 tokens after a 5-token prompt needs $5 + 6 + 7 = 18$ K/V computations without a cache, but only $5 + 1 + 1 = 7$ with it.

6.2 Cache sizes for real models (16-bit numbers)

Model	Layers	KV heads	d_{head}	Per token	4,096-token context
LLaMA-2 7B (MHA)	32	32	128	$2 \cdot 32 \cdot 32 \cdot 128 \cdot 2 = 524{,}288$ B = **0.5 MB**	**2 GiB**
Mistral 7B (GQA)	32	8	128	0.125 MB	0.5 GiB
LLaMA-2 70B (GQA)	80	8	128	0.31 MB	1.25 GiB

Serving 32 users of LLaMA-2 7B at 4,096 tokens each needs $32 \times 2 = 64$ GiB (≈ 69 GB) of cache — about **5×** the model's own weights (13.5 GB). Mistral's GQA brings that down to 16 GiB.

6.3 A lower bound on decode speed

LLaMA-2 7B in 16-bit: $6.74 \times 10^9 \times 2 = 13.5$ GB of weights. On an A100 GPU with about 2.0 TB/s of memory bandwidth, batch size 1, short context:

$$\frac{13.5 \times 10^9 \text{ bytes}}{2.0 \times 10^{12} \text{ bytes/s}} = 6.75 \text{ ms per token} \;\Rightarrow\; \text{at most} \approx 150 \text{ tokens/s}$$

No amount of extra arithmetic speed can beat this — the weights simply can't be read faster. Ways around it: **quantize** the weights (8-bit → half the bytes → about twice as fast, Section 17), and **batch** many users so each weight read produces many tokens.

Try it: cache memory and decode speed



Fits: 15.6 GB needed of 80 GB

Quantity	Value
Cache per token	524,288 bytes = 0.500 MB
Tokens kept per user	4,096
Cache for all 1 user	2.00 GiB
Bytes read per decode step (weights + cache)	15.6 GB
Lower bound on time per step	7.8 ms
Tokens/s per user · total	128 · 128

Each decode step must read all the weights plus every user's cache, so more users raise total throughput (128 tokens/s) until memory runs out. Try 32 users at 4,096 tokens for LLaMA-2 7B, then switch to Mistral 7B (GQA) or 8-bit.

6.4 Online softmax by hand

One query, four keys with scores $s = [2, 1, 3, 0]$ and (scalar) values $v = [1, 2, 3, 4]$. Process the keys in **two chunks** of two.

Chunk 1 (scores 2, 1):

- $m = 2$
- $\ell = e^{2-2} + e^{1-2} = 1 + 0.3679 = 1.3679$
- $o = 1 \cdot 1 + 0.3679 \cdot 2 = 1.7358$

Chunk 2 (scores 3, 0) — new maximum $m' = 3$:

- Rescale the old totals by $e^{2-3} = 0.3679$: $\ell = 0.5032$, $o = 0.6386$
- Add the new terms: $e^{3-3} = 1$ (value 3) and $e^{0-3} = 0.0498$ (value 4): $\ell = 0.5032 + 1 + 0.0498 = 1.5530$; $o = 0.6386 + 3 + 0.1991 = 3.8377$

Output: $o/\ell = 3.8377/1.5530 = \mathbf{2.4711}$.

Check with ordinary softmax: weights $[0.2369, 0.0871, 0.6439, 0.0321]$, output $= 0.2369(1) + 0.0871(2) + 0.6439(3) + 0.0321(4) = \mathbf{2.4711}$. Identical — but we never needed all four scores at once.

Try it: online softmax, one chunk at a time

score	2	1	3	0	–	–
value	1	2	3	4	–	–
chunk						

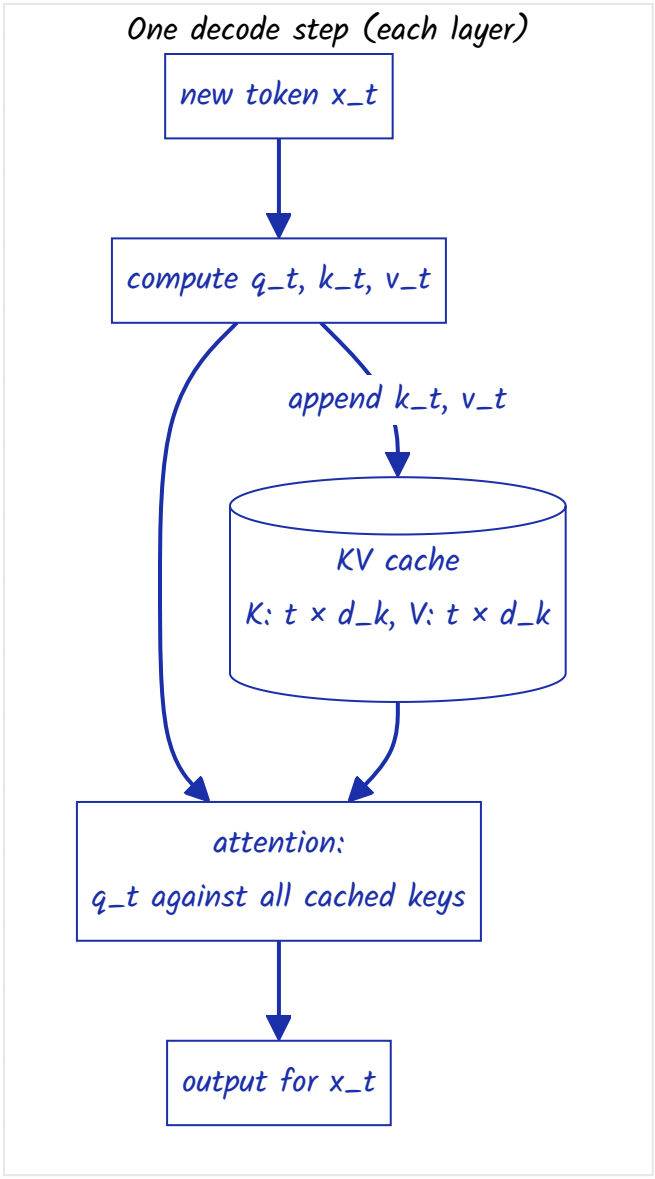
chunk (items)	rescale old totals	running max m	running sum ℓ	running output o

Processed 0 of 4 items. Each new chunk may raise the running max; when it does, the old sum and output are shrunk by e^(old max − new max) so everything stays on the same scale.

6.5 Why the score matrix is a problem

For a 32,768-token context, one head's score matrix in 16-bit is $32{,}768^2 \times 2 \approx 2.1$ GB — per head, per layer. A model with 32 heads would need about 69 GB *per layer* just to hold scores. FlashAttention never builds this matrix, so the memory stays proportional to n.

7. Diagram





8. In practice

- **Every production LLM server** uses a KV cache; systems like **vLLM** add PagedAttention and **continuous batching** (new requests join the running batch at every step) to serve many users per GPU (Section 18).

- **Long-context costs:** a 128k-token conversation needs a cache 32× larger than at 4k tokens. Long contexts are expensive mainly because of cache memory and the time to read it every step, not the weights.
- **FlashAttention** (and successors) is the default attention implementation in modern training and inference libraries; in PyTorch, `scaled_dot_product_attention` selects it automatically when it can.
- **Prefix caching** makes repeated system prompts and multi-turn chats cheaper: the shared beginning is prefilled once and reused.
- **Speculative decoding** (Section 13/17) tackles the same memory-bound problem from another angle: verify several drafted tokens per weight read.

9. Common mistakes and misconceptions

> **"The KV cache is an approximation."** It gives exactly the same results as recomputing everything; it only removes redundant work.

> **"Queries are cached too."** Only keys and values. Each past query was used once, at its own step, and is never needed again.

> **"Decoding is slow because GPUs can't compute fast enough."** For small batches it's limited by memory **bandwidth** — reading the weights and cache — not arithmetic.

> **"FlashAttention is a cheaper, approximate attention."** It computes **exact** attention; it's faster because it moves less data, not because it computes less.

> **"Bigger context only costs more compute."** It costs **memory**: the KV cache grows linearly with context for every user, limiting how many users fit on a GPU.

10. Summary

- Causal attention means past keys and values never change, so the **KV cache** stores them; each decode step computes q, k, v only for the new token (417× fewer K/V computations in our example).
- Cache size $= 2 \times \text{layers} \times \text{KV heads} \times d_{\text{head}} \times \text{bytes} \times \text{tokens} \times \text{batch}$ — LLaMA-2 7B: 0.5 MB per token, 2 GiB per 4k-token conversation.
- Decoding is **memory-bandwidth bound**: ≥ (weights + cache) / bandwidth per step (≈ 6.75 ms/token for a 7B model on an A100); batching and quantization help.
- Shrink the cache with **GQA/MQA**, **quantization**, **sliding windows**, **PagedAttention** and **prefix caching**.
- **FlashAttention** tiles the computation and uses the **online softmax** (running max, sum, output with rescaling) to get exact attention in $O(n)$ memory.

11. Check your understanding

1. Without a cache, how many K/V computations are needed to generate 4 tokens after a 10-token prompt? With a cache?
2. A model has 40 layers, 8 KV heads, $d_{\text{head}} = 128$, 16-bit cache. How much cache does one token need? How much for a batch of 16 users at 8,192 tokens each?
3. Redo the online softmax of 6.4 with chunks in the order (3, 0) then (2, 1). Do you get the same output? Which running quantity changes on the second chunk?

▶ Show answers

12. Further reading

1. Pope, R. et al. (2022). **"Efficiently Scaling Transformer Inference."** arXiv:2211.05102 — memory-bandwidth limits, KV cache, and batching trade-offs.

2. Dao, T. et al. (2022). **"FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness."** arXiv:2205.14135.
3. Milakov, M., & Gimelshein, N. (2018). **"Online normalizer calculation for softmax."** arXiv:1805.02867 — the online softmax.
4. Kwon, W. et al. (2023). **"Efficient Memory Management for Large Language Model Serving with PagedAttention."** arXiv:2309.06180 — vLLM.
5. Ainslie, J. et al. (2023). **"GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints."** arXiv:2305.13245.
6. Jiang, A. Q. et al. (2023). **"Mistral 7B."** arXiv:2310.06825 — sliding-window attention with GQA.

Prepared by Akhil

Vision Transformers (ViT)

> **In one line:** a Vision Transformer cuts an image into small square **patches**, turns each patch into a vector (a "visual token"), adds position information, and feeds the sequence to a standard Transformer **encoder** — the same architecture as BERT, applied to pictures instead of words.

1. Learning objectives
2. Prerequisites
3. Motivation
4. Intuition first
5. The formal idea
6. Worked examples
7. Diagram
8. In practice
9. Common mistakes
10. Summary
11. Check your understanding
12. Further reading

1. Learning objectives

By the end of this section you should be able to:

1. **Convert** an image into a sequence of patch tokens and compute the number of tokens for any image and patch size.
2. **Compute** patch embeddings by hand as a linear projection of flattened pixel values.
3. **Count** the parameters of ViT-B/16 using the formulas from earlier sections.
4. **Compare** ViTs with convolutional networks (CNNs): built-in assumptions, data needs, and cost.

5. **Explain** the key ViT-family ideas: DeiT, Swin, MAE, CLIP, and how vision–language models connect a ViT to an LLM.

2. Prerequisites

- **Section 9:** the encoder — bidirectional blocks, a `[CLS]` token for classification.
- **Section 7:** learned absolute position embeddings.
- **Section 2:** an embedding is a vector; cosine similarity.
- **Images as numbers:** a colour image of height H and width W is an $H \times W \times 3$ array (red, green, blue values for each pixel).

> **Quick refresher — flattening**
>
> "Flattening" a small block of numbers means listing them in one row, e.g. the 2×2 block $\begin{bmatrix} a & b \\ c & d \end{bmatrix}$ becomes $[a, b, c, d]$. A $16 \times 16 \times 3$ patch flattens to $16 \cdot 16 \cdot 3 = 768$ numbers.

3. Motivation: can attention replace convolutions?

For a decade, image recognition was dominated by **convolutional neural networks (CNNs)**, which slide small filters over the image. CNNs build in two strong assumptions:

- **Locality:** nearby pixels matter most.
- **Translation equivariance:** a cat in the corner is detected the same way as a cat in the centre.

Meanwhile Transformers had taken over language. The question was: *can the same architecture — with no image-specific assumptions — do vision too?*

The naive approach fails: treating every **pixel** as a token gives $224 \times 224 = 50{,}176$ tokens, and attention's n^2 cost makes that about 2.5 billion scores per head per layer. The **ViT** trick (2020): use **patches**, not pixels, as tokens. A 224×224 image becomes just **196** tokens.

4. Intuition first: a jigsaw puzzle

Cut a photograph into a 14×14 grid of jigsaw pieces.

- Each **piece** is a token. On its own it shows only a little — some fur, an edge, part of an eye.
- Each piece has a **position** written on the back (row 3, column 7) — the position embedding.
- **Self-attention** lets every piece "look at" every other piece: the piece with an ear figures out it belongs with the pieces showing whiskers and eyes — "together we're a cat".
- A special **summary card** (`[CLS]`) looks at all the pieces and announces the answer.

> **Where the analogy breaks**
>
> A jigsaw solver *moves* pieces to fit them. A ViT keeps every patch in place and instead updates each patch's **vector** to reflect what the whole image shows.

5. The formal idea

5.1 Image → patches → tokens

For an image of size $H \times W \times C$ and patch size P:

$$N = \frac{H}{P} \times \frac{W}{P} \quad \text{patches, each with } P^2 C \text{ numbers}$$

1. **Split** the image into N non-overlapping $P \times P$ patches.
2. **Flatten** each patch into a vector of length P^2C.
3. **Project** each flattened patch with a learned matrix E of shape $P^2C \times d_{model}$ (plus a bias) → a **patch embedding**.

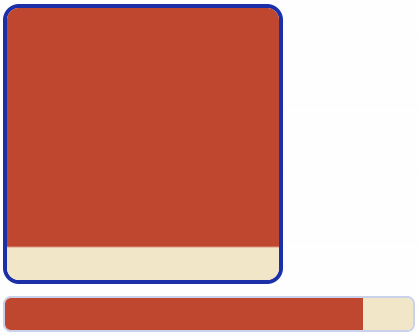
In code this is usually written as a convolution with kernel size P and stride P — mathematically the same linear projection, applied to every patch.

Try it: cut an image into tokens



grid	14 × 14
patches N	196
numbers per patch	16·16·3 = 768
tokens with [CLS]	197
patches the encoder sees	196
attention cost vs 196	1.00×

Click a patch to inspect it



Patch at row 7, column 9 → token #93 (token #0 is [CLS]). Flattened: 768 numbers (strip shows the pixels in order).

The standard ViT setting: 196 patches of 768 numbers each. Try P = 8 (784 patches, 16× the attention cost) or set the mask to 75% like MAE.

5.2 The full ViT

$$
\begin{aligned}
z_0 &= [\,x_{\text{cls}};\ x_1 E;\ x_2 E;\ \ldots;\ x_N E\,] + E_{\text{pos}} \quad (N+1) \times d_{model} \\
z_\ell &= \text{Block}_\ell(z_{\ell-1}), \quad \ell = 1, \ldots, L \\
y &= \text{softmax}\big(\text{LN}(z_L^{\text{cls}})\, W_{\text{head}}\big)
\end{aligned}
$$

- x_{cls}: a learned `[CLS]` vector prepended to the sequence (as in BERT).
- E_{pos}: learned position embeddings, one per token ($N+1$ of them).
- Blocks: standard **encoder** blocks — bidirectional self-attention + FFN (Section 8), no mask.
- Classification: a linear head on the final `[CLS]` vector (some models average all patch vectors instead).

5.3 ViT vs CNN

	CNN	ViT
Built-in assumptions	locality, translation equivariance	almost none — learned from data
Each layer sees	a small neighbourhood	the **whole image** from layer 1
Small datasets	usually better	worse (needs lots of data or tricks)
Very large datasets	improvements level off	keeps improving with scale
Cost vs resolution	grows with number of pixels	grows with **(number of patches)²** in attention

The original ViT matched or beat strong CNNs only when pre-trained on very large datasets (hundreds of millions of images). With less data, a CNN's built-in assumptions are an advantage.

5.4 The ViT family

- **DeiT:** training recipes (heavy data augmentation, and distillation from a CNN teacher) that make ViTs work well on ImageNet-size data alone.
- **Swin Transformer:** attention inside **local windows** that shift between layers, plus a **hierarchy** that merges patches as the network deepens. Cost grows linearly with image size, and the multi-scale features suit detection and segmentation.
- **MAE (Masked Autoencoder):** hide **75%** of the patches and train the model to reconstruct the missing pixels — the vision version of masked language modelling. The encoder only processes the visible 25%, which makes pre-training cheap.
- **CLIP:** an image encoder (often a ViT) and a text encoder trained together so that matching image–caption pairs get **similar embeddings** (contrastive learning on hundreds of millions of pairs). Result: **zero-shot** classification — compare an image's embedding with the embeddings of texts like *"a photo of a dog"*.
- **Vision–language models (e.g. LLaVA-style):** a ViT turns an image into patch vectors, a small **projector** maps them into the LLM's embedding space, and they're placed in the LLM's input like ordinary tokens. The LLM can then answer questions about the image.

5.5 Higher resolution

Patch size stays fixed, so a bigger image means more patches: 384×384 with $P = 16$ gives $24^2 = 576$ tokens instead of 196. Two consequences:

- **Cost:** attention grows with $576^2/196^2 \approx 8.6\times$.
- **Positions:** a model trained at 224 has only 196 learned position embeddings, so they are **interpolated** (resized as a 2-D grid) to 576 before fine-tuning at the new resolution.

6. Worked examples

6.1 Patch arithmetic

Image	Patch size	Patches N	Numbers per patch	Tokens (with `[CLS]`)	Attention cost vs 196
224 × 224 × 3	16	$14^2 = 196$	$16 \cdot 16 \cdot 3 = 768$	197	1×
224 × 224 × 3	8	$28^2 = 784$	192	785	**16×**
224 × 224 × 3	32	$7^2 = 49$	3,072	50	0.06×
384 × 384 × 3	16	$24^2 = 576$	768	577	8.6×

Halving the patch size gives finer detail but quadruples the token count and multiplies attention cost by 16.

6.2 Patch embeddings by hand

A tiny 4×4 grey-scale image (one channel, brightness 0–8), patch size $P = 2$:

$$\text{image} = \begin{bmatrix} 0 & 8 & 8 & 8 \\ 0 & 8 & 8 & 8 \\ 0 & 0 & 0 & 8 \\ 0 & 0 & 0 & 8 \end{bmatrix}$$

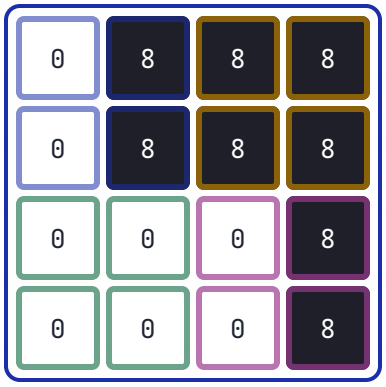
That's $N = (4/2)^2 = 4$ patches of $2 \cdot 2 = 4$ numbers each (flattened row by row). Use a projection to $d_{model} = 2$ whose columns we've chosen to be meaningful (a real model learns them):

$$E = \begin{bmatrix} 0.25 & -0.5 \\ 0.25 & 0.5 \\ 0.25 & -0.5 \\ 0.25 & 0.5 \end{bmatrix} \qquad \begin{matrix} \text{column 1: average brightness} \\ \text{column 2: right minus left (a vertical-edge detect} \end{matrix}$$

Patch	Pixels (flattened)	Embedding $[\text{brightness}, \text{edge}]$
top-left	[0, 8, 0, 8]	$[0.25(16),\ 0.5(16)] = [4, 8]$
top-right	[8, 8, 8, 8]	$[8, 0]$
bottom-left	[0, 0, 0, 0]	$[0, 0]$
bottom-right	[0, 8, 0, 8]	$[4, 8]$

The top-left and bottom-right patches both contain a dark-to-bright vertical edge, so they get the **same** embedding $[4, 8]$. Without position embeddings the model couldn't tell where each one is — exactly the order problem of Section 7, now in 2-D.

Try it: compute patch embeddings yourself



Click a pixel to cycle 0 → 4 → 8.

patch	flattened	x·E = [brightness, edge]
top-left	[0, 8, 0, 8]	[4, 8]
top-right	[8, 8, 8, 8]	[8, 0]
bottom-left	[0, 0, 0, 0]	[0, 0]
bottom-right	[0, 8, 0, 8]	[4, 8]

Highlighted rows have identical embeddings: the model can't tell which part of the image they came from. Switch on positions.

6.3 Every parameter of ViT-B/16

ViT-B/16: $d = 768$, 12 layers, 12 heads, $d_{ff} = 3072$, patch size 16, 224 × 224 input, 1,000 classes.

Part	Formula	Parameters
Patch embedding	$768 \times 768 + 768$ (input 768 = $16 \cdot 16 \cdot 3$)	590,592
`[CLS]` token	768	768
Position embeddings	197×768	151,296
12 encoder blocks	$12 \times 7{,}087{,}872$ (same block as BERT-base, Section 9)	85,054,464
Final LayerNorm	2×768	1,536
Classification head	$768 \times 1000 + 1000$	769,000
Total		**86,567,656**

That's ViT-B/16's reported **86M**. Notice almost everything is in the Transformer blocks — the "vision-specific" part (patch embedding) is under 1%.

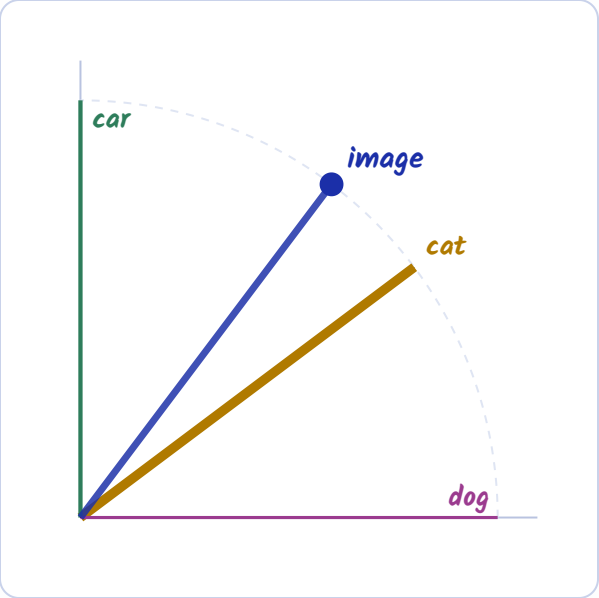
6.4 CLIP zero-shot classification (toy)

Use 2-D unit vectors. An image embedding $[0.6, 0.8]$; text embeddings for three prompts:

Prompt	Text embedding	Cosine with image	$\times 10$	Softmax
"a photo of a cat"	[0.8, 0.6]	0.96	9.6	**0.81**
"a photo of a car"	[0, 1]	0.80	8.0	0.16
"a photo of a dog"	[1, 0]	0.60	6.0	0.02

The image is classified as a **cat** — with no cat-specific training, just by comparing it with text. Real CLIP multiplies cosines by a learned scale of about 100, which makes the softmax much sharper (here "cat" would get essentially 1.00).

Try it: zero-shot classification with text prompts



a photo of a cat 0.816 cos 0.96

a photo of a car 0.161 cos 0.80

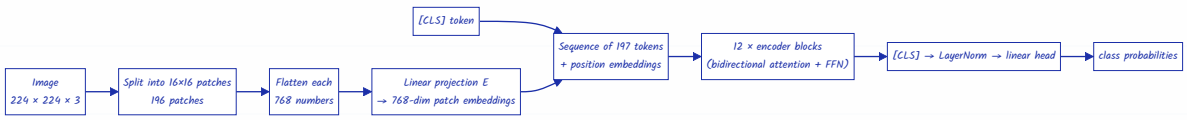
a photo of a dog 0.023 cos 0.60

The image embedding is [0.60, 0.80]. Prediction: “a photo of a cat” (82%). Raise the logit scale to see how CLIP sharpens small cosine differences. At 53° and scale 10 you get 6.4’s numbers.

6.5 Why MAE pre-training is cheap

With 196 patches and 75% masked, the encoder sees only $196 \times 0.25 = 49$ patches. Attention cost: 49^2 instead of 196^2 — **16× less** per layer. A small decoder then reconstructs the hidden patches. (Set the mask slider in the first demo to 75%.)

7. Diagram



8. In practice

Model	Year	Idea	Used for
ViT	2020	patches + standard encoder	image classification
DeiT	2021	data-efficient training, distillation	ViTs without huge datasets
Swin	2021	shifted windows, hierarchy	detection, segmentation
CLIP	2021	contrastive image–text training	zero-shot classification, image search, text-to-image guidance
MAE	2021	mask 75% of patches, reconstruct	self-supervised pre-training
DINOv2	2023	self-distillation	strong general-purpose image features
LLaVA-style VLMs	2023+	ViT features → projector → LLM	chat about images

Common sizes: ViT-B/16 (86M), ViT-L/16 (about 300M, 24 layers, $d = 1024$), ViT-H/14 (about 630M). The "/16" or "/14" is the patch size.

Beyond images: the same "patchify and use a Transformer" idea works for **audio** (spectrogram patches or frames — Whisper's encoder), **video** (space-time patches), and even **diffusion image generators**, many of which now use Transformer backbones on patches.

9. Common mistakes and misconceptions

> **"ViTs treat every pixel as a token."** They use **patches** (e.g. 16×16 pixels each); per-pixel attention would be far too expensive.

> **"The patch embedding is a CNN, so ViT is really a CNN."** The "convolution" with stride = kernel size is just a convenient way to apply the same linear

projection to every non-overlapping patch. There are no stacked convolutional layers.

"ViTs are always better than CNNs." With limited data and no special training tricks, CNNs often win; ViTs shine with large-scale pre-training.

"Patch embeddings already know where they are." Two identical patches get identical embeddings (6.2); position embeddings are required.

"CLIP was trained on a fixed list of classes." It learned from free-form captions, so you can classify into any set of classes by writing text prompts.

10. Summary

- A **ViT** splits an image into $N = (H/P)(W/P)$ patches, flattens and linearly projects each into a token, adds a `[CLS]` token and position embeddings, and runs a standard **encoder**.
- 224 × 224 with $P = 16$ → **196 patches + `[CLS]` = 197 tokens**; halving P multiplies attention cost by 16.
- ViT-B/16 has **86,567,656** parameters — 98% of them in ordinary Transformer blocks.
- ViTs have fewer built-in assumptions than CNNs: they need more data but scale better and see the whole image from the first layer.
- **DeiT** (efficient training), **Swin** (local windows, hierarchy), **MAE** (masked patches), **CLIP** (image–text contrastive, zero-shot) and **VLMs** (ViT → projector → LLM) extend the idea.

11. Check your understanding

1. A $512 \times 512 \times 3$ image is split into 32×32 patches. How many patch tokens are there, and how many numbers does each flattened patch contain?

How does attention cost compare with the standard 196-token ViT?

2. In 6.2, what would the bottom-right patch's embedding be if its pixels were $[8, 0, 8, 0]$ (bright on the left)? What does the sign of the second number tell you?
3. A ViT trained at 224 × 224 ($P = 16$) is fine-tuned at 448 × 448. How many position embeddings are needed now, and where do the extra ones come from?

▶ *Show answers*

12. Further reading

1. Dosovitskiy, A. et al. (2021). **"An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale."** *ICLR 2021.* arXiv:2010.11929 — the ViT paper.
2. Touvron, H. et al. (2021). **"Training data-efficient image transformers & distillation through attention."** arXiv:2012.12877 — DeiT.
3. Liu, Z. et al. (2021). **"Swin Transformer: Hierarchical Vision Transformer using Shifted Windows."** arXiv:2103.14030.
4. He, K. et al. (2021). **"Masked Autoencoders Are Scalable Vision Learners."** arXiv:2111.06377 — MAE.
5. Radford, A. et al. (2021). **"Learning Transferable Visual Models From Natural Language Supervision."** arXiv:2103.00020 — CLIP.
6. Liu, H. et al. (2023). **"Visual Instruction Tuning."** arXiv:2304.08485 — LLaVA.

Prepared by Akhil

Fine-Tuning & LoRA

> **In one line:** fine-tuning adapts a pre-trained model to your task or domain. Updating **all** the weights is expensive, so most practitioners use **parameter-efficient fine-tuning** — above all **LoRA**, which freezes the original weights and learns a tiny **low-rank** correction $\Delta W = BA$, often less than 1% of the parameters.

1. Learning objectives
2. Prerequisites
3. Motivation
4. Intuition first
5. The formal idea
6. Worked examples
7. Diagram
8. In practice
9. Common mistakes
10. Summary
11. Check your understanding
12. Further reading

1. Learning objectives

By the end of this section you should be able to:

1. **Choose** between prompting, full fine-tuning and parameter-efficient methods for a given situation.
2. **Explain** LoRA: why a low-rank update works, how B and A are initialised, and how the update is merged after training.
3. **Compute** a LoRA forward pass by hand and count LoRA's trainable parameters.
4. **Estimate** GPU memory for full fine-tuning, LoRA and QLoRA.

5. **Describe** adapters and prompt tuning, and the practical risks of fine-tuning (overfitting, forgetting).

2. Prerequisites

- **Section 12:** training with AdamW; ≈ 16 bytes per trainable parameter for weights, gradients and optimizer states.
- **Section 4/8:** the attention projections W_Q, W_K, W_V, W_O and FFN matrices are where most parameters live.
- **Matrix rank:** the **rank** of a matrix is the number of independent rows (or columns). A $d \times k$ matrix written as BA with B of shape $d \times r$ and A of shape $r \times k$ has rank at most r.

> **Quick refresher — the outer product**
> A column vector b ($d \times 1$) times a row vector a ($1 \times k$) gives a $d \times k$ matrix ba of **rank 1**: every row is a multiple of a. It's described by only $d + k$ numbers instead of $d \cdot k$.

3. Motivation: adapting a giant model cheaply

A pre-trained LLM knows a lot, but you may want it to:

- answer in your company's format and tone,
- handle a specialist domain (law, medicine, your codebase),
- do a narrow task very reliably (extract fields from invoices).

Options, from cheapest to most expensive:

Approach	Trainable parameters	Cost	Good when
Prompting / few-shot examples	0	none	the model can already do it with instructions
Retrieval (RAG, Section 18)	0	low	the model needs **facts** it doesn't know
LoRA / QLoRA	~0.1–1%	low	you need new **behaviour**, style, format or task skill
Full fine-tuning	100%	very high	big domain shift, lots of data and hardware

Full fine-tuning of a 7B model needs about $16 \times 6.74 \times 10^9 \approx 108$ GB just for weights, gradients and Adam states — more than one large GPU — and produces a full 13.5 GB copy of the model for every task. LoRA fixes both problems.

4. Intuition first: sticky notes on a textbook

You own an excellent textbook (the pre-trained model). You're taking a specialised course and want to adapt it.

- **Full fine-tuning** = rewriting the whole textbook by hand. Thorough, but slow, and you end up with one rewritten copy per course.
- **LoRA** = leaving the book untouched and adding a few **sticky notes** with corrections. The notes are tiny, you can keep different sets for different courses, and you can swap them in and out. When you're done, you can even copy the notes into the book permanently (merging).
- **Prompt tuning** = writing a short cheat-sheet at the front of the book that changes how you read the rest.

Where the analogy breaks

LoRA's "notes" aren't sentences; they're small matrices whose product slightly changes how every input passes through a layer.

5. The formal idea

5.1 Full fine-tuning

Start from the pre-trained weights and continue training on task data, updating every parameter (usually with a smaller learning rate than pre-training). Most powerful, most expensive. Every task needs its own full copy of the model.

5.2 LoRA: Low-Rank Adaptation

LoRA keeps each chosen weight matrix W_0 (shape $d \times k$) **frozen** and adds a trainable low-rank update (using the paper's convention $h = Wx$ with column vectors):

$$h = W_0 x + \frac{\alpha}{r} BAx, \qquad B \in \mathbb{R}^{d \times r},\ A \in \mathbb{R}^{r \times k},\ r \ll \min(d, k)$$

- **Rank r:** typically 4–64.
- **Initialisation:** A random (small Gaussian), $B = 0$. So at the start $BA = 0$ and the model behaves **exactly** like the original — training begins from the pre-trained model, not from a damaged one.
- **Scaling** α/r: a constant that keeps the update's size roughly independent of r (e.g. $\alpha = 16, r = 8 \rightarrow$ factor 2).
- **Trainable parameters** for one matrix: $r(d + k)$ instead of dk.
- **Where:** usually the attention projections (W_Q, W_V or all four), often the FFN matrices too.

Why low rank is enough: experiments show that the *change* needed to adapt a pre-trained model to a task tends to be much simpler than the weights themselves — it can be captured well by a few directions. A rank-r update can only change the layer's behaviour along r directions, but those turn out to be the ones that matter.

Merging: after training, compute $W = W_0 + \frac{\alpha}{r} BA$ once. The merged model has exactly the original shape and **no extra inference cost**. Or keep the small A, B files separate and **swap** them per task — many LoRA "adapters" can share one base model.

5.3 QLoRA

QLoRA stores the frozen base model in **4-bit** precision (a special format called NF4) and trains LoRA matrices on top in 16-bit. The base weights take about 0.5 bytes per parameter instead of 2, so a 65B-parameter model can be fine-tuned on a single 48 GB GPU, and a 7B model on a consumer GPU.

5.4 Other parameter-efficient methods

- **Adapters:** small bottleneck networks inserted inside each block: down-project $d \to m$, non-linearity, up-project $m \to d$, plus a residual. Effective, but they add a little inference latency because they can't be merged away.
- **Prompt tuning / prefix tuning:** learn a handful of "virtual token" embeddings (prompt tuning) or per-layer key/value vectors (prefix tuning) that are prepended to the input. Extremely few parameters; works best on large models.
- **(IA)³, DoRA and others:** variations that scale activations or split weights into magnitude and direction.

5.5 Practical points

- **Data quality beats quantity:** a few thousand clean, consistent examples often beat a large noisy set.

- **Use the model's chat template** exactly; mismatched formatting is a common silent failure.
- **Learning rate:** LoRA typically uses about 10^{-4} to 2×10^{-4}, much higher than full fine-tuning ($\approx 10^{-5}$), because only a few parameters move.
- **Catastrophic forgetting:** narrow fine-tuning can erase general abilities. Keep a held-out evaluation of general skills, use few epochs, or mix in general data. LoRA forgets somewhat less, since the base weights are frozen.
- **Overfitting:** watch validation loss; 1–3 epochs is usually enough.

6. Worked examples

6.1 Counting LoRA parameters

One 4096×4096 matrix ($d = k = 4096$):

	Parameters	Share of the matrix
Full matrix	$4096^2 = 16{,}777{,}216$	100%
LoRA, $r = 8$	$8 \times (4096 + 4096) = 65{,}536$	**0.39%**
LoRA, $r = 16$	$131{,}072$	0.78%

Whole model (LLaMA-2 7B): LoRA with $r = 16$ on W_Q, W_K, W_V, W_O in all 32 layers:

$$32 \times 4 \times 16 \times (4096 + 4096) = 16{,}777{,}216 \text{ trainable parameters} \approx 0.25$$

The whole adapter file is about 34 MB in 16-bit — versus 13.5 GB for a fully fine-tuned copy.

6.2 A LoRA forward pass by hand

Take a tiny layer with $d = k = 3$, frozen weights $W_0 = I$ (the identity), rank $r = 1$, and $\alpha/r = 1$. After training, suppose

$$A = \begin{bmatrix} 1 & 0 & 1 \end{bmatrix} \ (1 \times 3), \qquad B = \begin{bmatrix} 0.5 \\ 0 \\ -0.5 \end{bmatrix} \ (3 \times 1)$$

Input $x = [1, 2, 3]^\top$.

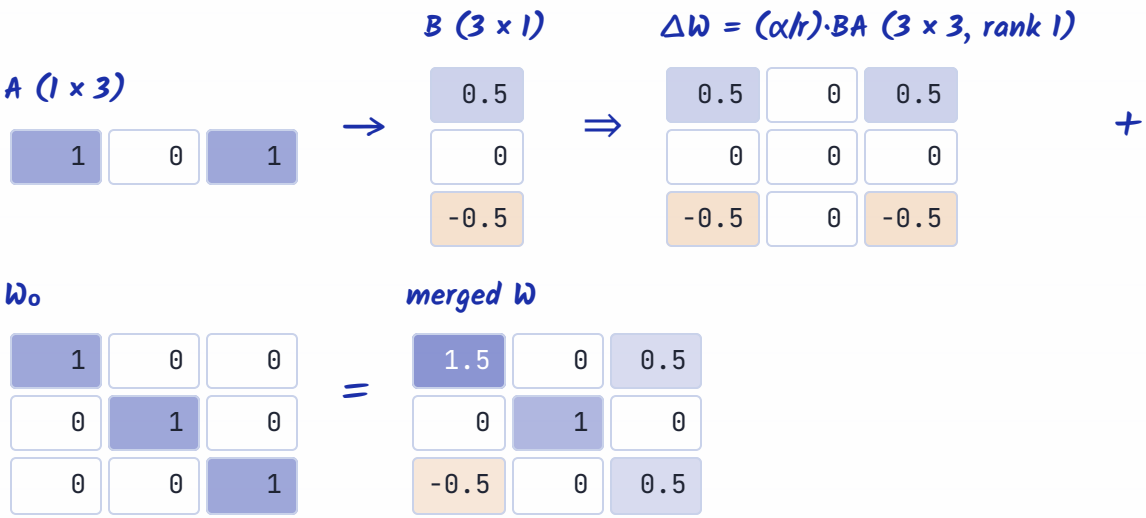
1. Frozen path: $W_0 x = [1, 2, 3]^\top$
2. LoRA path, step 1: $Ax = 1(1) + 0(2) + 1(3) = 4$ (one number, because $r = 1$)
3. LoRA path, step 2: $B(Ax) = 4 \times [0.5, 0, -0.5]^\top = [2, 0, -2]^\top$
4. Add: $h = [1, 2, 3] + [2, 0, -2] = [\mathbf{3}, \mathbf{2}, \mathbf{1}]$

Merging: $\Delta W = BA = \begin{bmatrix} 0.5 & 0 & 0.5 \\ 0 & 0 & 0 \\ -0.5 & 0 & -0.5 \end{bmatrix}$ (rank 1 — every row is a multiple of $[1, 0, 1]$).

$W = W_0 + \Delta W = \begin{bmatrix} 1.5 & 0 & 0.5 \\ 0 & 1 & 0 \\ -0.5 & 0 & 0.5 \end{bmatrix}$, and $Wx = [1.5 + 1.5,\ 2,\ -0.5 + 1.5] = [3, 2, 1]$ — **the same output**, with a single matrix.

At initialisation ($B = 0$): $h = W_0 x = [1, 2, 3]$ exactly — the model starts unchanged.

Try it: a LoRA layer, two ways



During training: frozen W_0 + LoRA path

x = [1, 2, 3]

W_0x = [1, 2, 3]

Ax = 1·1 + 0·2 + 1·3 = **4**

(α/r)·B·(Ax) = [2, 0, −2]

h = **[3, 2, 1]**

After merging: one matrix $W = W_0 + (\alpha/r)\cdot BA$

h = Wx = **[3, 2, 1]**

✓ identical to the LoRA path

The LoRA path adds a correction along B, scaled by how much x points along A. Merging gives one matrix with exactly the same effect.

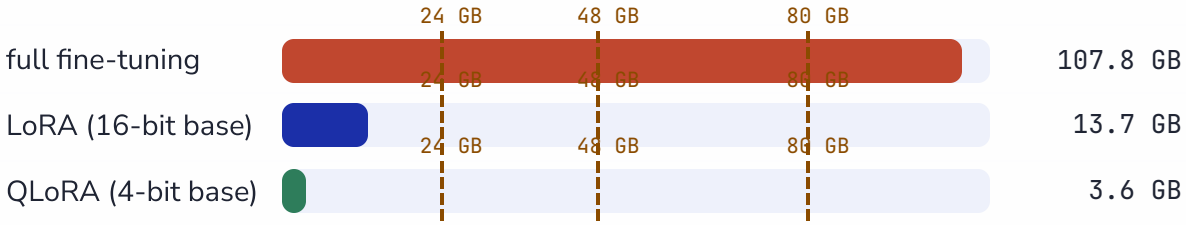
6.3 Memory: full fine-tuning vs LoRA vs QLoRA (7B model)

Method	Frozen base weights	Trainable parameters × 16 bytes	Total before activations
Full fine-tuning	—	$6.74\text{B} \times 16 = 108$ GB	**≈ 108 GB**
LoRA ($r = 16$, attention)	16-bit: $6.74\text{B} \times 2 = 13.5$ GB	$16.8\text{M} \times 16 = 0.27$ GB	**≈ 13.8 GB**
QLoRA (same LoRA)	4-bit: $6.74\text{B} \times 0.5 = 3.4$ GB	0.27 GB	**≈ 3.7 GB**

Activations come on top and depend on batch size and sequence length, but the difference is dramatic: full fine-tuning needs several data-centre GPUs; QLoRA fits on a single consumer GPU.

Try it: how many parameters, how much memory?

16,777,216 trainable parameters (0.25% of the model) · adapter file ≈ 34 MB



Dashed lines: common GPU sizes (24, 48, 80 GB). Bars exclude activations.

LoRA on q, k, v, o at rank 16. Before activations: full fine-tuning needs several GPUs, LoRA a 24 GB GPU, QLoRA a 24 GB GPU. (Attention matrices are treated as d×d here; models with GQA have smaller K/V projections.)

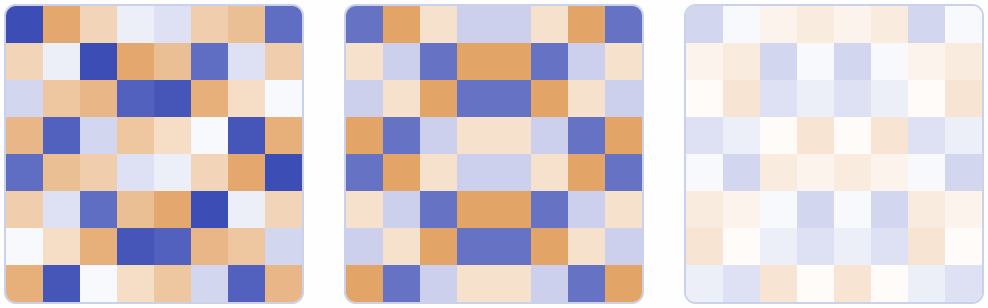
6.4 How much rank do you need?

Suppose the "ideal" update for some 8×8 weight matrix is a sum of five rank-1 pieces with strengths $8, 4, 2, 1, 0.5$ (in orthogonal directions). The best rank-r approximation keeps the r strongest pieces:

Rank r	Parameters $r(8+8)$	Relative error
0	0	100%
1	16	50%
2	32	25%
3	48	12%
4	64	5%
5	80	0%

When an update is dominated by a few strong directions — as task adaptations tend to be — a small rank captures most of it.

Try it: approximating an update with rank r



target ΔW (rank 5) rank-2 approximation what's left (error)

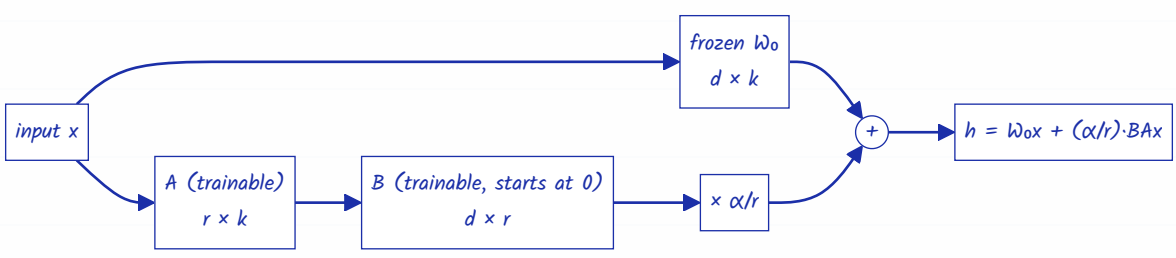
parameters: 2 × (8 + 8) = 32 of 64 · relative error: 25%

Each extra rank adds the next-strongest direction. Because the strengths fall off (8, 4, 2, 1, 0.5), the first few ranks remove most of the error.

6.5 Other methods, counted

- **Adapters** in BERT-base ($d = 768$, bottleneck $m = 64$, two per layer, 12 layers): each adapter $2 \times 768 \times 64 + 64 + 768 = 99{,}136$ parameters; total $24 \times 99{,}136 = 2{,}379{,}264 \approx$ **2.2%** of BERT-base.
- **Prompt tuning** with 20 virtual tokens on a model with $d = 4096$: $20 \times 4096 = 81{,}920$ parameters — about 0.001% of a 7B model.

7. Diagram



After training: $W = W_0 + \frac{\alpha}{r} BA$ → one ordinary matrix, no extra cost.

8. In practice

Situation	Typical choice
Teach a consistent format, tone or task on a 7–70B model	LoRA / QLoRA on attention (and often FFN) matrices, r = 8–64
One GPU with limited memory	QLoRA
Serve many customers, each with their own customisation	one base model + many swappable LoRA adapters
Big domain shift, plenty of data and compute	full fine-tuning (or continued pre-training)
Model just lacks up-to-date facts	RAG first (Section 18), not fine-tuning

- **Libraries:** Hugging Face **PEFT** provides LoRA, QLoRA (with bitsandbytes), adapters and prompt tuning with a few lines of configuration.
- **Instruction tuning** (Section 12's SFT) and **preference tuning** (DPO) are both commonly run with LoRA.
- **Evaluate before and after** on your task *and* on general ability, to catch forgetting.

9. Common mistakes and misconceptions

> **"Fine-tuning is the best way to add new knowledge."** Fine-tuning is better at teaching **behaviour and format**; for facts that change or need citations, retrieval (RAG) is usually more reliable.

> **"LoRA adds inference cost."** Only if you keep the adapter separate. After merging, the model is exactly the original size and speed.

"LoRA starts from a random model." $B = 0$ at the start, so the adapted model initially equals the pre-trained one.

"A higher rank is always better." Beyond a modest rank, gains are small and overfitting risk rises; small ranks often match full fine-tuning on narrow tasks.

"QLoRA trains a 4-bit model." The **base** is stored in 4-bit and frozen; the LoRA weights are trained in 16-bit.

10. Summary

- **Full fine-tuning** updates every weight — powerful but ≈ 16 bytes/parameter of memory and a full model copy per task.
- **LoRA** freezes W_0 and learns $\Delta W = \frac{\alpha}{r} BA$ with B ($d \times r$, starts at 0) and A ($r \times k$); parameters $r(d + k)$ — e.g. 0.39% of a 4096×4096 matrix at $r = 8$.
- The update can be **merged** into W_0 (no inference cost) or kept as a small, swappable adapter.
- **QLoRA** stores the frozen base in 4-bit, bringing 7B fine-tuning to ≈ 4 GB plus activations.
- **Adapters** and **prompt/prefix tuning** are other parameter-efficient options; good data, the right template and checking for forgetting matter more than the method.

11. Check your understanding

1. A weight matrix is $4096 \times 11{,}008$ (an FFN projection in LLaMA-2 7B). How many trainable parameters does LoRA with $r = 8$ add? What fraction of the matrix is that?
2. In 6.2, what is h if $x = [2, 0, -2]^\top$? Explain the result using the rank of ΔW.
3. Why is B initialised to zero rather than A and B both random?

▶ *Show answers*

12. Further reading

1. Hu, E. J. et al. (2021). **"LoRA: Low-Rank Adaptation of Large Language Models."** arXiv:2106.09685.
2. Dettmers, T. et al. (2023). **"QLoRA: Efficient Finetuning of Quantized LLMs."** arXiv:2305.14314.
3. Houlsby, N. et al. (2019). **"Parameter-Efficient Transfer Learning for NLP."** arXiv:1902.00751 — adapters.
4. Lester, B., Al-Rfou, R., & Constant, N. (2021). **"The Power of Scale for Parameter-Efficient Prompt Tuning."** arXiv:2104.08691.
5. Li, X. L., & Liang, P. (2021). **"Prefix-Tuning: Optimizing Continuous Prompts for Generation."** arXiv:2101.00190.
6. Aghajanyan, A., Zettlemoyer, L., & Gupta, S. (2020). **"Intrinsic Dimensionality Explains the Effectiveness of Language Model Fine-Tuning."** arXiv:2012.13255 — why low-dimensional updates suffice.

Prepared by Akhil

Optimization & Quantization

> **In one line:** large models are only practical because of a toolbox of efficiency tricks — **lower-precision numbers** (BF16, INT8, INT4), **memory-saving training** (mixed precision, gradient checkpointing, ZeRO/FSDP sharding, model parallelism), and **cheaper inference** (quantization, distillation, pruning, speculative decoding).

1. Learning objectives
2. Prerequisites
3. Motivation
4. Intuition first
5. The formal idea
6. Worked examples
7. Diagram
8. In practice
9. Common mistakes
10. Summary
11. Check your understanding
12. Further reading

1. Learning objectives

By the end of this section you should be able to:

1. **Compare** number formats (FP32, FP16, BF16, FP8, INT8, INT4) by range and precision, and explain why BF16 is preferred for training.
2. **Quantize** a list of weights to INT8 and INT4 by hand (absmax scaling), and explain why outliers and group-wise scales matter.
3. **Calculate** per-GPU training memory under data parallelism and ZeRO stages 1–3.
4. **Explain** gradient checkpointing, gradient accumulation, and tensor/pipeline parallelism.

5. **Estimate** the speed-up from speculative decoding, and describe distillation and pruning.

2. Prerequisites

- **Section 12:** AdamW training needs ≈ 16 bytes per parameter (weights, gradients, master weights, m, v).
- **Section 14:** decoding is memory-bandwidth bound — fewer bytes per weight means faster tokens.
- **Binary numbers:** with b bits you can represent 2^b different values (8 bits → 256).

> **Quick refresher — floating-point numbers**
> A floating-point number is stored like scientific notation in base 2: a **sign**, an **exponent** (sets the range — how big or small) and a **mantissa** (sets the precision — how many significant digits). More exponent bits → bigger range; more mantissa bits → finer steps.

3. Motivation: the numbers are too big

A 70-billion-parameter model:

- **Weights alone** in 32-bit floats: $70 \times 10^9 \times 4 = 280$ GB. In 16-bit: 140 GB. Neither fits on one 80 GB GPU.
- **Training** with AdamW: ≈ $16 \times 70 \times 10^9 = 1{,}120$ GB of weights, gradients and optimizer states — before activations.
- **Generation** reads every weight for every token (Section 14), so the number of bytes directly sets the speed.

Every byte saved per parameter is multiplied by tens of billions. This section is about saving those bytes — and the compute that goes with them — without

losing much accuracy.

4. Intuition first: packing for a trip

You're moving a huge library to a new city.

- **Lower precision** = writing measurements with fewer decimal places. "1.23456789 m" becomes "1.23 m": less space, slightly less exact, usually good enough.
- **Quantization** = sorting books onto a fixed set of shelves (only 16 shelves for INT4) instead of placing each at its exact height. One enormous book (an **outlier**) forces you to space all the shelves far apart, and small books all end up on the bottom shelf — unless each section of the library gets its own shelf spacing (**group-wise scales**).
- **ZeRO / sharding** = several friends each carry one part of the library instead of everyone carrying a full copy.
- **Gradient checkpointing** = not photocopying every page you read; if you need a page again later, re-read it.
- **Distillation** = a great teacher writes a short study guide that a smaller student learns from.
- **Speculative decoding** = a fast assistant drafts the next few sentences; the expert just checks them, which is quicker than writing.

> ### Where the analogy breaks
> Quantization error isn't random clutter — it systematically shifts each weight to its nearest allowed value, and errors in thousands of weights can add up.

5. The formal idea

5.1 Number formats

Format	Bits (sign / exponent / mantissa)	Largest value	Step just above 1.0	Typical use
FP32	1 / 8 / 23	$\approx 3.4 \times 10^{38}$	1.2×10^{-7}	master weights, optimizer states
FP16	1 / 5 / 10	65,504	9.8×10^{-4}	older mixed-precision training, inference
BF16	1 / 8 / 7	$\approx 3.4 \times 10^{38}$	7.8×10^{-3}	**training today**
FP8 (E4M3)	1 / 4 / 3	448	0.125	newest GPUs: training and inference
INT8	integer	127	1 (with a scale)	inference weights/activations
INT4	integer	7	1 (with a scale)	inference weights

Why BF16 for training: it has the **same range as FP32** (8 exponent bits), so values rarely overflow or underflow; it gives up precision, which training tolerates well. FP16 has more precision but a tiny range: gradients smaller than about 6×10^{-8} round to zero, so FP16 training needs **loss scaling** (multiply the loss by e.g. 1,024 before the backward pass, divide the gradients afterwards).

Mixed precision: do the heavy matrix multiplications in BF16/FP16 (fast), but keep a **32-bit master copy** of the weights and the optimizer states, so tiny updates aren't lost to rounding.

Try it: how does each format store your number?

format	stored value	relative error	status
FP32	1.0007	3.0e-7%	ok
FP16	1.0009766	0.028%	ok
BF16	1	0.070%	ok
FP8 (E4M3)	1	0.070%	ok

Compare the relative errors: BF16 is coarser than FP16 (7 vs 10 mantissa bits) but covers a far larger range.

5.2 Quantization

Quantization maps real numbers onto a small set of integers plus a **scale**. The simplest scheme is **symmetric absmax** quantization to b bits:

$$s = \frac{\max_i |w_i|}{q_{\max}}, \quad q_{\max} = 2^{b-1} - 1 \qquad q_i = \text{round}\left(\frac{w_i}{s}\right) \qquad \hat{w}_i = s \cdot q_i$$

- Store the small integers q_i (8 or 4 bits each) plus one scale s per group.
- At use time, **dequantize** $\hat{w}_i = s\, q_i$ (or compute directly with integers on supporting hardware).
- The rounding error per weight is at most $s/2$.

Granularity: one scale for a whole tensor is cheapest but fragile; **per-channel** or **per-group** scales (e.g. one scale per 64 or 128 weights) keep errors small, at a small storage cost.

The outlier problem: a single very large value inflates s for its whole group, so ordinary small weights all round to 0. Fixes: smaller groups, keeping outliers in higher precision (as in LLM.int8()), or rescaling channels before quantizing (as in SmoothQuant/AWQ).

When to quantize:

- **Post-training quantization (PTQ):** quantize a finished model; methods like **GPTQ** and **AWQ** choose roundings/scales that minimise the error on sample data. Takes minutes to hours; INT8 is nearly lossless and 4-bit usually loses only a little.
- **Quantization-aware training (QAT):** simulate quantization during training so the model learns to tolerate it; better at very low bit widths.

5.3 Saving memory during training

- **Gradient accumulation:** run several small batches, add up their gradients, then do one optimizer step — same effective batch size with less activation memory.
- **Gradient (activation) checkpointing:** store activations only at a few layer boundaries and **recompute** the rest during the backward pass. Saves most activation memory for about one extra forward pass (≈ 33% more compute, since a training step ≈ 1 forward + 2 forward-equivalents of backward).
- **Data parallelism:** every GPU holds a full model copy and processes different data; gradients are averaged (all-reduce) each step.
- **ZeRO / FSDP:** remove the redundant copies across data-parallel GPUs by **sharding**:

Stage	What is split across the G GPUs	Bytes per parameter per GPU
none (plain data parallel)	nothing	$2 + 2 + 12 = 16$
ZeRO-1	optimizer states (12 B)	$2 + 2 + 12/G$
ZeRO-2	+ gradients	$2 + 14/G$
ZeRO-3 / FSDP	+ weights	$16/G$

(2 bytes 16-bit weights, 2 bytes 16-bit gradients, 12 bytes for the FP32 master copy, m and v.) Higher stages save more memory but communicate more.

- **Tensor parallelism:** split each big matrix multiplication across GPUs (e.g. different attention heads or columns of W on different GPUs). Needs very fast links; used within a server.
- **Pipeline parallelism:** put different layers on different GPUs, passing activations along like an assembly line; micro-batches keep all stages busy.

Real large-scale training combines all of these ("3-D parallelism").

5.4 Cheaper inference

- **Weight quantization** (INT8/INT4): fewer bytes → less memory and, because decoding is bandwidth-bound, faster generation.
- **Knowledge distillation:** train a small **student** to match a large **teacher**'s output probabilities (softened with a temperature), not just the correct labels. The student learns the teacher's "dark knowledge" about which wrong answers are nearly right.
- **Pruning:** remove weights (unstructured) or whole heads/neurons/layers (structured) that matter least. Structured pruning gives real speed-ups on ordinary hardware; unstructured pruning needs special support.
- **Speculative decoding:** a small **draft** model proposes k tokens; the large model checks all of them in **one** forward pass and keeps the longest prefix it agrees with (plus one token of its own). The output distribution is exactly the large model's. If each drafted token is accepted with probability α, the expected number of tokens per large-model pass is

$$E = \frac{1 - \alpha^{k+1}}{1 - \alpha}$$

6. Worked examples

6.1 What can each format represent?

- **Step size near 1.0:** FP16 can represent 1 and $1 + 2^{-10} = 1.000977$ but nothing in between; BF16 only 1 and $1 + 2^{-7} = 1.0078$. BF16 is coarser — but its range is as large as FP32's.
- **A tiny gradient** of 10^{-8}: below FP16's smallest positive number ($\approx 6 \times 10^{-8}$), so it **underflows to 0**. With loss scaling by 1,024 it becomes 1.024×10^{-5} — safely representable — and is divided back after the backward pass. In BF16 it's representable without any scaling.
- **A large activation** of 100,000: overflows FP16 (max 65,504) but fits easily in BF16.

6.2 INT8 and INT4 quantization by hand

Weights: $w = [0.52, -1.17, 0.03, 2.40, -0.71, 0.15]$. Largest magnitude: 2.40.

INT8 ($q_{\max} = 127$): scale $s = 2.40/127 = 0.0189$.

w	w/s	q	$\hat{w} = sq$	error
0.52	27.52	28	0.529	0.009
−1.17	−61.91	−62	−1.172	0.002
0.03	1.59	2	0.038	0.008
2.40	127.00	127	2.400	0
−0.71	−37.57	−38	−0.718	0.008
0.15	7.94	8	0.151	0.001

Average error **0.005** — tiny.

INT4 ($q_{\max} = 7$): scale $s = 2.40/7 = 0.343$.

w	q	$\hat{w}$	error
0.52	2	0.686	0.166
−1.17	−3	−1.029	0.141
0.03	0	0	0.030
2.40	7	2.400	0
−0.71	−2	−0.686	0.024
0.15	0	0	0.150

Average error **0.085** — 18× larger: only 15 levels are available. Small weights (0.03, 0.15) become exactly 0.

6.3 One outlier ruins a group

Replace 0.15 with an outlier, **24.0**. Now $s = 24/7 = 3.43$ for INT4: 0.52, −1.17, 0.03 and −0.71 all round to **0**, and 2.40 becomes 3.43. Five of six weights are badly wrong.

Group-wise scales (groups of 3):

- Group 1 $[0.52, -1.17, 0.03]$: $s = 1.17/7 = 0.167 \rightarrow [0.50, -1.17, 0]$ — good again.
- Group 2 $[2.40, -0.71, 24.0]$: still dominated by the outlier → $[3.43, 0, 24.0]$.

The damage is now confined to the outlier's own group. Real 4-bit methods use groups of 64–128 weights and extra tricks to protect outliers.

Try it: quantize six weights

w	group	scale s	w/s	q	ŵ = s·q	error
0.52	1	0.0189	27.52	28	0.529	0.009
-1.17	1	0.0189	-61.91	-62	-1.172	0.002
0.03	1	0.0189	1.59	2	0.038	0.008
2.40	1	0.0189	127.00	127	2.400	0.000
-0.71	1	0.0189	-37.57	-38	-0.718	0.008
0.15	1	0.0189	7.94	8	0.151	0.001

average error 0.0046 · 255 levels · ≈ 10.7 bits per weight incl. 16-bit scales

With 8 bits there are 255 levels, so every weight lands very close to its true value.

6.4 Memory for the weights

Model	FP16 / BF16	INT8	INT4
LLaMA-2 7B (6.74B)	13.5 GB	6.7 GB	3.4 GB (≈ 3.8 GB with group scales)
70B	140 GB	70 GB	35 GB

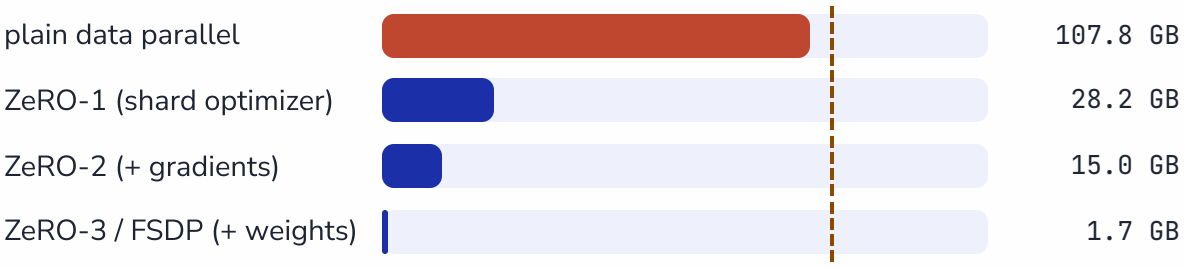
A 70B model in INT4 fits on a single 48 GB GPU (with room for some KV cache); in FP16 it needs at least two 80 GB GPUs. And since decoding is bandwidth-bound, 4-bit weights can make generation roughly 2–3× faster in practice.

6.5 ZeRO on 64 GPUs (training LLaMA-2 7B)

$N = 6.74 \times 10^9, G = 64$:

Setup	Bytes per parameter per GPU	Per-GPU memory (before activations)
Plain data parallel	16	**108 GB** — doesn't fit on 80 GB
ZeRO-1	$4 + 12/64 = 4.19$	28.2 GB
ZeRO-2	$2 + 14/64 = 2.22$	15.0 GB
ZeRO-3 / FSDP	$16/64 = 0.25$	**1.7 GB**

Try it: training memory per GPU



Dashed line: one GPU's 80 GB. The first setup that fits (before activations) is ZeRO-1 (shard optimizer). Each higher stage needs more communication between GPUs.

6.6 Speculative decoding

A draft model proposes $k = 4$ tokens; each is accepted with probability $\alpha = 0.8$:

$$E = \frac{1 - 0.8^5}{1 - 0.8} = \frac{1 - 0.328}{0.2} = 3.36 \text{ tokens per large-model pass}$$

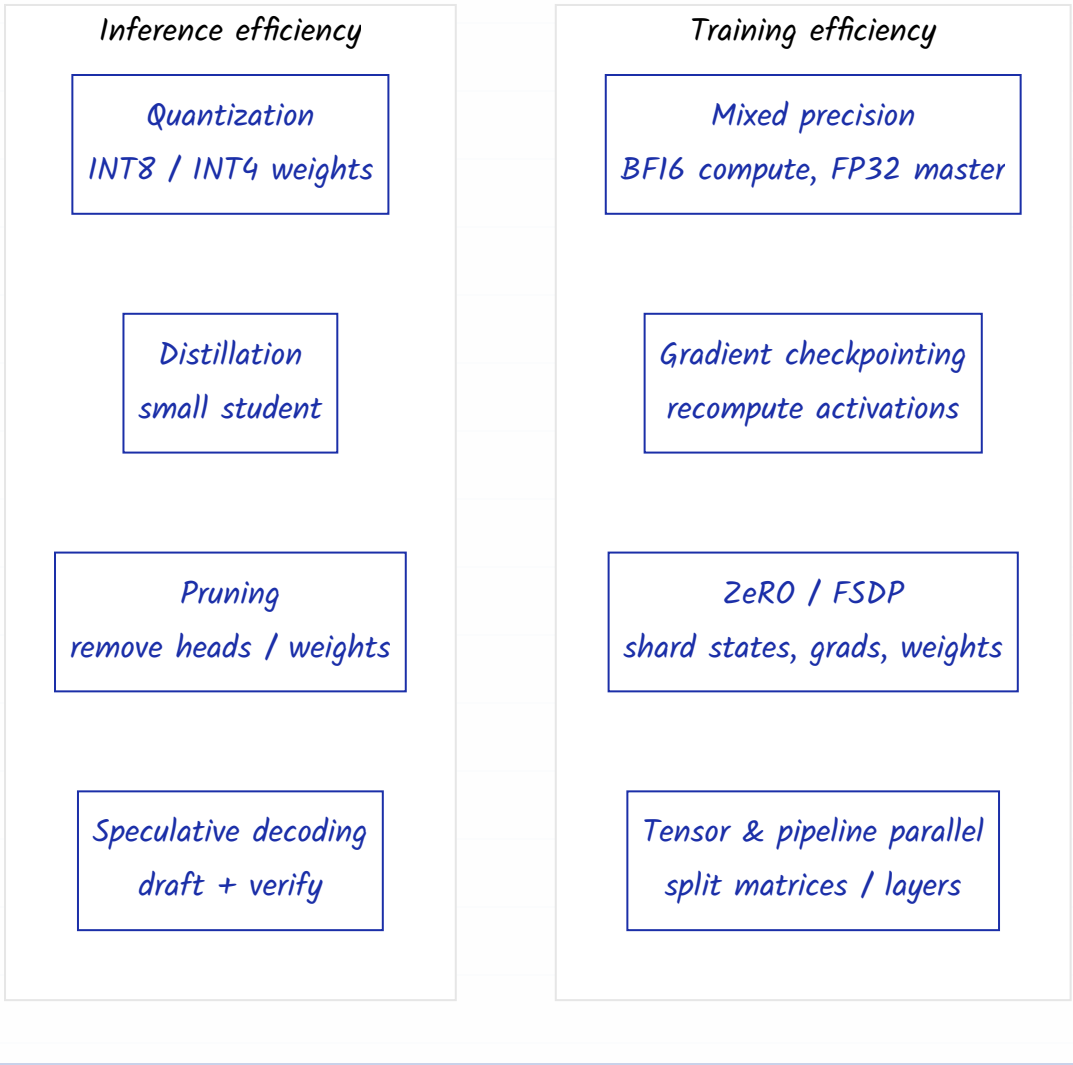
If one draft-model step costs 5% of a large-model step, each round costs $1 + 4 \times 0.05 = 1.2$ large-model steps, giving a speed-up of about $3.36/1.2 \approx \mathbf{2.8}\times$ — with exactly the same output distribution.

Try it: speculative decoding

E = (1 − 0.80^5) / (1 − 0.80) = 3.36 tokens per big-model pass · speed-up ≈ 2.80×

Each round costs 1 big-model pass plus 4 draft steps (0.20 big-model equivalents). Longer drafts only pay off when acceptance is high.

7. Diagram



8. In practice

Goal	Common choice
Pre-training a large model	BF16 mixed precision, FSDP/ZeRO-3, tensor + pipeline parallelism, gradient checkpointing; FP8 on the newest GPUs
Fine-tuning on one GPU	QLoRA (4-bit base, Section 16), gradient checkpointing, gradient accumulation
Serving in a data centre	FP8/INT8 weights (and often KV cache), continuous batching, speculative decoding
Running on a laptop or phone	4-bit (or lower) weights in formats like GGUF via llama.cpp, or a distilled small model

- **Measure accuracy after quantizing**, on your own tasks — some abilities (maths, long reasoning, rare languages) degrade before average benchmarks do.
- **Small distilled models** (and small models trained on outputs of larger ones) are now competitive for many narrow tasks at a fraction of the cost.
- **Bits keep falling:** research on 2-bit and even ~1.6-bit (ternary) models shows promise, especially when the model is trained that way from the start.

9. Common mistakes and misconceptions

"FP16 and BF16 are interchangeable." Same size, different trade-off: FP16 has more precision but a tiny range (needs loss scaling); BF16 has FP32's range with less precision.

"Quantization just truncates decimals." It maps values to a few levels with a shared **scale**; the scale's choice (and outliers) decides the error.

"4-bit always ruins a model." With good methods (GPTQ, AWQ, group-wise scales), 4-bit weights usually cost only a small amount of quality — but

always check on your tasks.

"ZeRO-3 is free memory." It trades memory for **communication**: weights must be gathered from other GPUs before each layer.

"Speculative decoding changes the model's answers." With the standard acceptance rule, the output distribution is exactly the large model's; it only changes speed.

10. Summary

- **BF16** (FP32's range, less precision) is the standard training format; FP16 needs loss scaling; master weights and Adam states stay in FP32 (**mixed precision**).
- **Quantization**: $s = \max|w|/q_{\max}$, $q = \text{round}(w/s)$, $\hat{w} = sq$. INT8 is nearly lossless; INT4 needs care — **outliers** inflate the scale, so use **group-wise scales** and methods like GPTQ/AWQ.
- Weight memory: 7B = 13.5 GB (16-bit) → 6.7 GB (INT8) → 3.4 GB (INT4); 70B INT4 fits on one 48 GB GPU.
- Training memory: **ZeRO/FSDP** shards optimizer states, gradients and weights (7B on 64 GPUs: 108 GB → 1.7 GB per GPU); **gradient checkpointing** trades ≈ 33% more compute for much less activation memory.
- Cheaper inference: quantization, **distillation**, **pruning**, and **speculative decoding** ($E = (1-\alpha^{k+1})/(1-\alpha)$ tokens per pass).

11. Check your understanding

1. Quantize $w = [1.0, -0.5, 0.25]$ to INT4 with one absmax scale. Give s, the integers q, and the dequantized values.
2. A 13B model is trained on 32 GPUs with ZeRO-2. How much memory per GPU do weights, gradients and optimizer states need?

3. With a draft length of $k = 3$ and acceptance probability $\alpha = 0.6$, how many tokens per large-model pass do you expect? Is speculative decoding worth it if a draft step costs 10% of a large step?

▶ *Show answers*

12. Further reading

1. Micikevicius, P. et al. (2018). **"Mixed Precision Training."** arXiv:1710.03740 — FP16 training and loss scaling.
2. Rajbhandari, S. et al. (2020). **"ZeRO: Memory Optimizations Toward Training Trillion Parameter Models."** arXiv:1910.02054.
3. Chen, T. et al. (2016). **"Training Deep Nets with Sublinear Memory Cost."** arXiv:1604.06174 — gradient checkpointing.
4. Dettmers, T. et al. (2022). **"LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale."** arXiv:2208.07339 — outliers in LLMs.
5. Frantar, E. et al. (2022). **"GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers."** arXiv:2210.17323. And Lin, J. et al. (2023). **"AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration."** arXiv:2306.00978.
6. Hinton, G., Vinyals, O., & Dean, J. (2015). **"Distilling the Knowledge in a Neural Network."** arXiv:1503.02531.
7. Leviathan, Y., Kalman, M., & Matias, Y. (2023). **"Fast Inference from Transformers via Speculative Decoding."** arXiv:2211.17192.

Prepared by Akhil

Serving & RAG

> **In one line: serving** is the engineering of running an LLM for many users at once — fast, cheap and reliably (continuous batching, KV-cache management, streaming). **RAG** (retrieval-augmented generation) gives the model the right documents at question time, so it can answer from your own, up-to-date data and cite its sources.

1. Learning objectives
2. Prerequisites
3. Motivation
4. Intuition first
5. The formal idea
6. Worked examples
7. Diagram
8. In practice
9. Common mistakes
10. Summary
11. Check your understanding
12. Further reading

1. Learning objectives

By the end of this section you should be able to:

1. **Describe** the path of a request through an LLM serving system and the metrics used to judge it (TTFT, time per output token, throughput, cost per million tokens).
2. **Explain** why continuous batching beats static batching, and compute GPU utilisation for both.
3. **Draw** the RAG pipeline — chunk, embed, index, retrieve, (re-rank), prompt, generate — and explain each step.

4. **Compute** chunk counts, cosine-similarity retrieval, and retrieval metrics (recall@k, MRR) by hand.
5. **Diagnose** common RAG failures and choose between RAG and fine-tuning.

2. Prerequisites

- **Sections 13–14:** prefill vs decode, TTFT, the KV cache, memory-bandwidth limits.
- **Section 9:** encoders produce sentence embeddings by mean pooling; bi-encoders vs cross-encoders.
- **Section 2:** cosine similarity.
- **Section 16:** fine-tuning teaches behaviour; this section shows how to supply knowledge.

> ***Quick refresher — percentiles***
>
> The **p50** latency is the median: half of requests are faster. The **p99** latency is slower than 99% of requests — it describes the unlucky users. Serving systems care about p99, not just the average.

3. Motivation: two practical gaps

Gap 1 — one model, many users. A model that answers one question in 6 seconds on a GPU is not a product. A real service must handle thousands of users at once, keep the first token fast, stream the rest, and cost cents rather than dollars per conversation.

Gap 2 — the model doesn't know your data. A pre-trained LLM:

- has a **knowledge cutoff** — nothing after its training data,
- has never seen your **private documents** (college circulars, company policies, your codebase),

- can **hallucinate** — state plausible-sounding but wrong facts,
- can't easily **cite** where an answer came from.

Fine-tuning doesn't fix these well (Section 16): facts change, and the model still can't point to a source. **RAG** fixes them by looking up relevant text at question time and putting it in the prompt.

4. Intuition first

Serving = a busy restaurant kitchen. Orders (requests) arrive at random. A bad kitchen cooks in fixed rounds — nobody's new order starts until every dish in the current round is done (static batching). A good kitchen starts a new dish the moment a burner frees up (continuous batching). The fridge space (GPU memory for KV caches) limits how many dishes can be in progress at once.

RAG = an open-book exam. Instead of answering from memory, the student first looks up the relevant pages in the textbook, then writes an answer using them — and can say "see page 42". The quality depends on two things: finding the **right pages** (retrieval) and **writing a good answer** from them (generation).

Where the analogies break

A kitchen's burners are independent; on a GPU, all requests in a batch share each step, so one very long request still occupies a slot until it finishes. And unlike a student, the model only "reads" the pages you put in its context window — it can't flip to others.

5. The formal idea

5.1 Anatomy of a serving system

1. **API gateway:** authentication, rate limits, routing.
2. **Scheduler:** queues requests, decides which ones join the next batch.

3. **Inference engine** (e.g. vLLM, TGI, TensorRT-LLM, SGLang): runs prefill and decode on the GPUs, manages KV-cache memory (PagedAttention, prefix caching — Section 14).
4. **Streaming:** sends each token to the user as soon as it's generated.

Key metrics:

Metric	Meaning	Mostly set by
TTFT (time to first token)	how long before the answer starts appearing	queueing + prefill
TPOT (time per output token)	speed of the streamed answer	decode (memory bandwidth, batch size)
Throughput	tokens per second across all users	batching, KV-cache capacity
Cost per million tokens	GPU price ÷ tokens produced	throughput
p99 latency	the slow tail of requests	load, long requests, queueing

Total time for one answer of n tokens $\approx \mathrm{TTFT} + n \times \mathrm{TPOT}$.

5.2 Static vs continuous batching

- **Static batching:** collect B requests, run them together until **all** finish. Short requests sit idle, padded, while the longest one finishes; new requests wait for the whole batch.
- **Continuous (in-flight) batching:** the batch is rebuilt **every decode step** — finished requests leave immediately and waiting requests take their slots.

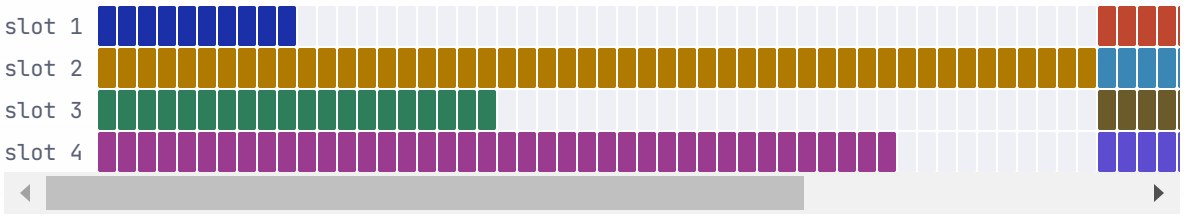
GPU slot utilisation for static batching:

$$\text{utilisation} = \frac{\sum_i \text{length}_i}{B \times \max_i \text{length}_i}$$

Continuous batching keeps slots nearly full whenever there's a queue, typically giving several times the throughput.

Try it: static vs continuous batching

8 queued requests needing 10, 50, 20, 40, 15, 30, 8, 25 output tokens (all arrive at step 0).



r1 (10) r2 (50) r3 (20) r4 (40) r5 (15) r6 (30) r7 (8) r8 (25) idle slot

steps to finish all 8	80
slot utilisation	62%
average finish step	49.8

Static: each group of 4 runs until its longest request finishes, so grey slots sit idle and the next group waits. Switch to continuous.

5.3 The RAG pipeline

Indexing (offline, done once and updated as documents change):

1. **Load** documents (PDF, web pages, wiki, tickets...).
2. **Chunk** them into passages of a few hundred tokens, with some **overlap** so sentences cut at a boundary appear whole in one chunk.
3. **Embed** each chunk with an embedding model (an encoder, Section 9) → a vector.
4. **Store** vectors (plus the text and metadata like source and page) in a **vector index**.

Answering (online, every question):

1. **Embed the question** with the same embedding model.
2. **Retrieve** the top-k chunks by similarity (usually cosine).

3. Optionally **re-rank** them with a cross-encoder (Section 9) that reads question and chunk together — slower but more accurate.
4. **Build the prompt:** instructions + retrieved chunks (numbered, with sources) + the question.
5. **Generate** an answer, asking the model to use only the provided context and to cite chunk numbers.

5.4 Retrieval details

- **Exact search** compares the query with every vector — fine for thousands of chunks, too slow for hundreds of millions.
- **Approximate nearest-neighbour (ANN)** indexes such as **HNSW** (a navigable graph) or **IVF** (clustering) find almost the same top results in milliseconds.
- **Keyword search (BM25)** matches exact words — great for names, codes and rare terms that embeddings can blur.
- **Hybrid search** runs both and merges the lists, often with **reciprocal rank fusion (RRF)**:

$$\mathrm{RRF}(d) = \sum_{\text{lists}} \frac{1}{k + \mathrm{rank}(d)}, \quad k \approx 60$$

5.5 Evaluating RAG

Evaluate the two halves separately:

Part	Metric	Question it answers
Retrieval	**recall@k**	Is a relevant chunk in the top k?
Retrieval	**MRR** (mean reciprocal rank)	How high is the first relevant chunk? (average of $1/\text{rank}$)
Generation	**faithfulness**	Is every claim in the answer supported by the retrieved text?
Generation	**answer relevance / correctness**	Does it actually answer the question correctly?

Common failures: the right chunk isn't retrieved (bad chunking, vocabulary mismatch → try hybrid search, better chunking, query rewriting); it's retrieved but buried among irrelevant chunks (→ re-ranking, smaller k); the model ignores or contradicts the context (→ clearer instructions, citations, a stronger model).

6. Worked examples

6.1 Static vs continuous batching

Four requests in one batch ($B = 4$) need **10, 50, 20 and 40** output tokens.

Static: the batch runs for 50 steps (the longest request). Slot-steps used: $10 + 50 + 20 + 40 = 120$ out of $4 \times 50 = 200$.

$$\text{utilisation} = 120/200 = \mathbf{60\%}$$

40% of the GPU's batch capacity is wasted on finished requests, and any new request waits all 50 steps.

Continuous: when the 10-token request finishes at step 10, a waiting request takes its slot at step 11, and so on. With a steady queue, slots stay close to 100% busy.

6.2 What does a token cost?

A GPU rented at $2 per hour that sustains **2,000 output tokens per second** across all users:

$$\text{tokens per hour} = 2{,}000 \times 3{,}600 = 7.2 \text{ million} \qquad \text{cost} = \frac{2}{7.2\text{M}} \approx \mathbf{0.28}$$

Doubling throughput (better batching, quantization) halves the cost.

Latency for one user: TTFT 0.2 s, TPOT 25 ms, a 300-token answer → $0.2 + 300 \times 0.025 = \mathbf{7.7}$ **s** — but streaming means they start reading after 0.2 s.

Try it: cost and latency

tokens per hour	7,200,000
cost per million tokens	$0.278
cost of this answer	$0.000083
first token after	0.20 s
full answer after	7.70 s
streaming speed	40 tokens/s

6.3 Chunking a document

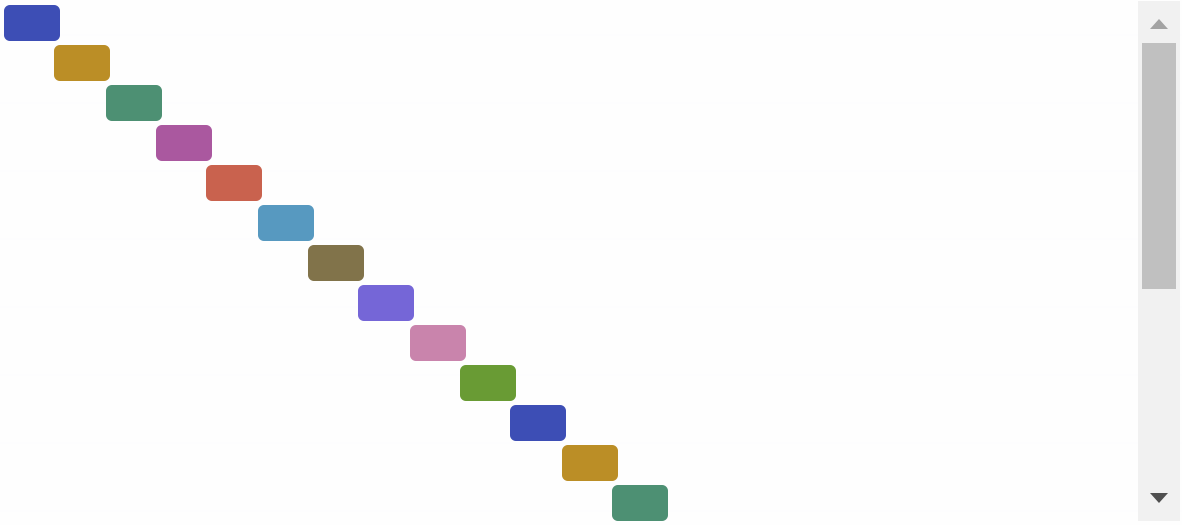
A 10,000-token document, chunk size 500 tokens, overlap 50 tokens → each new chunk starts $500 - 50 = 450$ tokens after the previous one:

- Chunk starts: 0, 450, 900, ..., 9,450 (22 chunks), plus one final chunk 9,900–10,000 to cover the end.
- Total: **23 chunks**.

Storage for the index: with 768-dimensional 32-bit embeddings, each chunk needs $768 \times 4 = 3{,}072$ bytes; one million chunks need about **3 GB** of vectors (plus the text).

Try it: chunk a document

23 chunks · step 450 tokens · index ≈ 70,656 bytes of vectors



Each bar is one chunk placed along the document (grey bar). Overlaps let a sentence cut at one boundary appear whole in the next chunk. Total tokens stored: 11,100 (11.0% extra because of overlap).

6.4 Retrieval by cosine similarity

A tiny index of college notices (3-D toy embeddings):

Chunk	Text	Embedding
c1	"Library hours: 8 am to 10 pm on weekdays."	[0.9, 0.1, 0.2]
c2	"The hostel mess serves dinner from 7 pm."	[0.2, 0.9, 0.1]
c3	"On Sundays the library closes at 6 pm."	[0.6, 0.3, 0.6]
c4	"The exam fee deadline is 15 March."	[0.1, 0.2, 0.9]

Question: *"When is the library open?"* → embedding $q = [0.9, 0.2, 0.2]$, with $\|q\| = 0.943$.

Chunk	$q \cdot c$	$\|c\|$	Cosine
c1	$0.81 + 0.02 + 0.04 = 0.87$	0.927	**0.994**
c3	$0.54 + 0.06 + 0.12 = 0.72$	0.900	**0.848**
c2	$0.18 + 0.18 + 0.02 = 0.38$	0.927	0.434
c4	$0.09 + 0.04 + 0.18 = 0.31$	0.927	0.354

With $k = 2$, retrieval returns **c1 and c3** — both library notices. The prompt then becomes:

```
Answer using only the context. Cite sources as [1], [2].
[1] Library hours: 8 am to 10 pm on weekdays.
[2] On Sundays the library closes at 6 pm.
Question: When is the library open?
```

and a good answer is: *"8 am to 10 pm on weekdays [1]; on Sundays it closes at 6 pm [2]."*

Try it: a working mini-RAG over ten college notices

This demo really retrieves: it turns every notice and your question into a word-weight vector (TF-IDF — a simple stand-in for a neural embedding model), ranks notices by cosine similarity, and builds the prompt an LLM would receive.

d2	On Sundays the library opens at 10 am and closes at 6 pm.	0.617
d1	Library hours: the central library is open 8 am to 10 pm on weekdays.	0.402
d10	The computer lab is open until 8 pm and needs a valid student ID card.	0.099
d3	The hostel mess serves dinner from 7 pm to 9:30 pm every day.	0.000
d4	Students who return late can collect a packed dinner from the hostel office until 11 pm.	0.000
d5	The last date to pay the semester exam fee is 15 March; a late fee applies after that.	0.000
d6	Exam hall tickets can be downloaded from the student portal one week before exams.	0.000
d7	A college bus leaves the railway station for the campus at 7:30 am and 8:15 am.	0.000
d8	The sports ground is closed for maintenance during the first week of April.	0.000
d9	Wi-Fi passwords for the hostel are reset on the first day of every month.	0.000

Prompt sent to the LLM

```
Answer using only the context below. Cite sources as [1], [2], … If the
context does not contain the answer, say you don't know.

[1] On Sundays the library opens at 10 am and closes at 6 pm.
[2] Library hours: the central library is open 8 am to 10 pm on
weekdays.

Question: When is the library open on Sunday?
```

Question terms used: library, open, sunday

Highlighted notices go into the prompt with numbers, so the LLM can cite them. Try the “bus” question, or ask something with words the notices don’t use.

6.5 Retrieval metrics

Three test questions; the first relevant chunk appears at rank **1**, rank **3**, and **not at all** in the top 5:

- recall@1 $= 1/3 = 0.33$; recall@3 $= 2/3 = 0.67$
- MRR $= \frac{1}{3}\left(\frac{1}{1} + \frac{1}{3} + 0\right) = \mathbf{0.44}$

6.6 Hybrid search with RRF ($k = 60$)

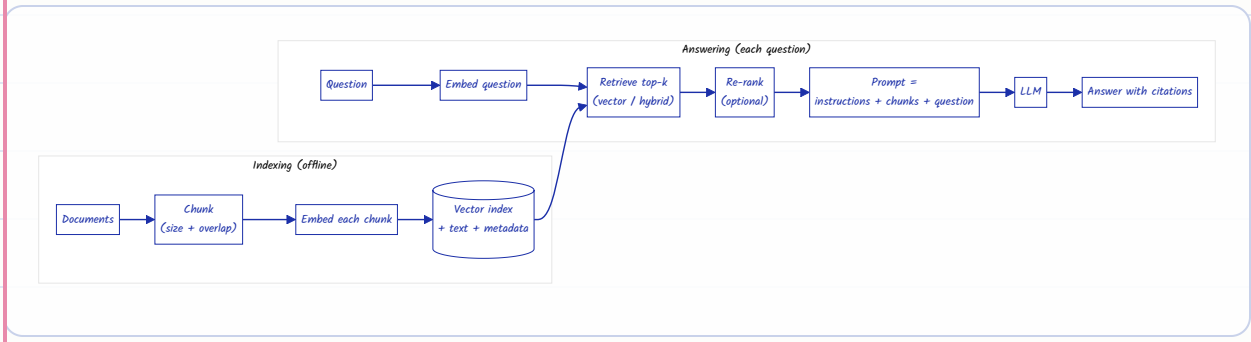
Document	BM25 rank	Vector rank	RRF score
A	1	3	$1/61 + 1/63 = 0.03227$
B	2	1	$1/62 + 1/61 = 0.03252$
C	—	2	$1/62 = 0.01613$

Final order: **B, A, C**. B wins because it's near the top of *both* lists; C appears in only one.

6.7 Fitting it into the context window

A 4,096-token model: system instructions 200 + five chunks × 500 + question 50 + room for a 500-token answer = **3,250 tokens** — it fits, with about 850 to spare. Ten chunks would not. Retrieval quality matters more than stuffing in more chunks.

7. Diagram



8. In practice

Serving stack choices:

Need	Typical tool
High-throughput GPU serving	vLLM, TGI, TensorRT-LLM, SGLang
Local / laptop	llama.cpp, Ollama (quantized GGUF models)
Managed	a cloud provider's hosted model API

RAG building blocks:

Component	Examples
Embedding models	sentence-transformers / E5 / BGE family, hosted embedding APIs
Vector stores	FAISS (library), pgvector (PostgreSQL), Qdrant, Weaviate, Milvus, Chroma
Keyword search	BM25 (e.g. in Elasticsearch / OpenSearch)
Re-rankers	cross-encoder models

Good practice:

- Start simple: decent chunking + a good embedding model + top-5 retrieval + a clear prompt with citations. Add hybrid search and re-ranking when evaluation shows retrieval misses.
- **Build a small evaluation set** (questions with known answers and source chunks) before tuning anything.
- Keep **metadata** (source, date, page, access rights) with every chunk — for citations, filtering, and making sure users only retrieve documents they're allowed to see.
- **RAG vs fine-tuning:** RAG for *knowledge* (changing facts, private documents, citations); fine-tuning for *behaviour* (format, tone, a narrow skill). They combine well.

- **Long-context models** can sometimes take whole documents instead of chunks, but cost grows with every token (Section 14), and retrieval still helps when the collection is large.

9. Common mistakes and misconceptions

> **"Bigger batches always mean better service."** Bigger batches raise throughput but can raise each user's latency; serving balances the two, within KV-cache memory.

> **"RAG makes hallucinations impossible."** It reduces them, but the model can still misread, ignore or over-generalise the context — measure faithfulness.

> **"More retrieved chunks is always better."** Irrelevant chunks dilute the context and can mislead the model; quality over quantity.

> **"Vector search understands everything."** Embeddings can blur exact names, numbers and codes; hybrid search with keywords catches these.

> **"RAG and fine-tuning are alternatives."** They solve different problems (knowledge vs behaviour) and are often used together.

10. Summary

- **Serving** turns a model into a service: gateway → scheduler → inference engine → streaming. Judge it by **TTFT**, **TPOT**, **throughput**, **p99 latency** and **cost per million tokens**.
- **Continuous batching** refills slots every step; static batching wastes capacity (60% utilisation in our example).
- **RAG** = chunk → embed → index (offline) and embed question → retrieve top-k → (re-rank) → prompt with sources → generate (online).

- Retrieval uses cosine similarity over embeddings, ANN indexes at scale, and **hybrid** keyword + vector search merged with **RRF**.
- Evaluate retrieval (**recall@k**, **MRR**) and generation (**faithfulness**, correctness) separately; use RAG for knowledge and fine-tuning for behaviour.

11. Check your understanding

1. A static batch of 4 requests needs 30, 30, 30 and 90 output tokens. What is the slot utilisation? What would help most?
2. A 6,000-token document is split with chunk size 400 and overlap 100. How many chunks are produced?
3. Using 6.4, which chunks are retrieved for the question *"What time is dinner?"* with embedding $q = [0.2, 0.9, 0.2]$ and $k = 1$? Compute the winning cosine.

▶ **Show answers**

12. Further reading

1. Lewis, P. et al. (2020). **"Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks."** arXiv:2005.11401 — the original RAG paper.
2. Kwon, W. et al. (2023). **"Efficient Memory Management for Large Language Model Serving with PagedAttention."** arXiv:2309.06180 — vLLM and continuous batching.
3. Yu, G.-I. et al. (2022). **"Orca: A Distributed Serving System for Transformer-Based Generative Models."** *OSDI 2022* — iteration-level (continuous) batching.
4. Karpukhin, V. et al. (2020). **"Dense Passage Retrieval for Open-Domain Question Answering."** arXiv:2004.04906.
5. Malkov, Y. A., & Yashunin, D. A. (2016). **"Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs."** arXiv:1603.09320 — HNSW.

6. Cormack, G. V., Clarke, C. L. A., & Büttcher, S. (2009). **"Reciprocal Rank Fusion outperforms Condorcet and individual Rank Learning Methods."** *SIGIR 2009.*

Prepared by Akhil

Interview Revision

> **In one line:** this section pulls the whole course together — a one-page formula sheet, the numbers worth remembering, a question bank with model answers, mixed numerical problems, and two system-design questions — so you can revise for exams and interviews in one place.

1. Learning objectives
2. How to use this section
3. Formula sheet
4. Numbers worth remembering
5. Question bank
6. Numerical problems
7. System design
8. Revision checklist
9. Common interview mistakes
10. Summary
11. Final self-test
12. Further reading

1. Learning objectives

By the end of this section you should be able to:

1. **Recall** the core formulas of the Transformer from memory, with shapes.
2. **Answer** common conceptual interview questions clearly in two or three sentences.
3. **Solve** mixed numerical problems: attention, parameter counts, KV cache, compute, LoRA, quantization, decoding.
4. **Sketch** an end-to-end design for an LLM application (a RAG assistant, a serving setup) and justify the choices.
5. **Identify** your weak topics and know which section to revisit.

2. How to use this section

1. **Cover the answers.** Try each question before reading the model answer — or use the flashcards below.
2. **Say answers out loud.** Interviews reward clear, short explanations: *definition → why it matters → one example or number*.
3. **Do the numerical problems on paper.** Then check them with the calculator and the drill below.
4. **Mark weak spots** and go back to the section listed next to each question.

A good answer has three parts

- *What* it is (one sentence).
- *Why* it exists — the problem it solves.
- *A concrete detail* — a formula, a number, or a real model that uses it.

3. One-page formula sheet

Idea	Formula	Section
Embedding lookup	$X = E[\text{ids}]$, E: $\lvert\mathcal{V}\rvert \times d_{model}$	2
Cosine similarity	$\cos(a,b) = \frac{a \cdot b}{\lVert a\rVert \lVert b\rVert}$	2
Projections	$Q = XW_Q,\ K = XW_K,\ V = XW_V$	4
Scaled dot-product attention	$\text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) V$	5
Multi-head attention	$[\text{head}_1 \mid \cdots \mid \text{head}_h]\, W_O, d_k = d_{model}/h$	6
Sinusoidal positions	$p_{pos,2i} = \sin\left(pos/10000^{2i/d}\right), p_{pos,2i+1} = \cos(\cdot)$	7
Pre-norm block	$A = X + \text{MHA}(\text{LN}(X)), X' = A + \text{FFN}(\text{LN}(A))$	8
LayerNorm	$\gamma \frac{x-\mu}{\sqrt{\sigma^2+\epsilon}} + \beta$	8
FFN	$\phi(xW_1 + b_1)W_2 + b_2, d_{ff} \approx 4d$	8
Parameters per block	$\approx 12\, d_{model}^2$ (attention $4d^2$ + FFN $8d^2$)	8
Next-token loss	$\mathcal{L} = -\frac{1}{T}\sum_t \log P(x_{t+1} \mid x_{\le t})$; perplexity $e^{\mathcal{L}}$	10, 12
Training compute	$C \approx 6ND$; Chinchilla $D \approx 20N$	12
Training memory (AdamW)	≈ 16 bytes / parameter	12
Temperature	$p_i \propto e^{z_i/T}$	13
KV cache	$2 \times \text{layers} \times \text{KV heads} \times d_{head} \times \text{bytes} \times \text{tokens}$	14
Decode time bound	$\geq (\text{weights} + \text{cache})/\text{bandwidth}$	14
ViT patches	$N = (H/P)(W/P)$, each P^2C numbers	15
LoRA	$h = W_0x + \frac{\alpha}{r}BAx$; params $r(d+k)$	16
Quantization	$s = \max\lvert w\rvert/q_{\max}, q = \text{round}(w/s)$	17
Speculative decoding	$E = (1-\alpha^{k+1})/(1-\alpha)$	17

RRF	$\sum 1/(k + \text{rank}), k \approx 60$	18

4. Numbers worth remembering

Fact	Value
Original Transformer (base)	6+6 layers, $d = 512$, 8 heads, ≈ 65M params
BERT-base	12 layers, $d = 768$, 12 heads, **109,482,240** params
GPT-2 small	12 layers, $d = 768$, **124,439,808** params
LLaMA-2 7B	32 layers, $d = 4096$, 32 heads, d_{ff} = 11,008, **6.74B** params, trained on 2T tokens
ViT-B/16	196 patches + `[CLS]`, **86.6M** params
Typical d_{head}	64 or 128
English rule of thumb	1 token ≈ ¾ word ≈ 4 characters
KV cache, LLaMA-2 7B	0.5 MB per token, 2 GiB at 4,096 tokens
Weights, 7B model	13.5 GB (16-bit), 6.7 GB (INT8), 3.4 GB (INT4)
LLaMA-2 7B training	≈ 8×10^{22} FLOPs, ≈ 184k A100 GPU-hours
Chinchilla	≈ 20 training tokens per parameter
BERT MLM	15% picked; 80% `[MASK]` / 10% random / 10% unchanged
MAE	75% of patches masked

5. Question bank with model answers

Click a question to reveal its answer — or practise with the flashcards after the list.

Foundations (Sections 1–3)

▶ 1. What problem did the Transformer solve compared with RNNs?

▶ 2. Why sub-word tokenization?

▶ 3. What does self-attention compute, in one sentence?

▶ 4. Is self-attention aware of word order?

Attention (Sections 4–7)

▶ 5. Why separate Q, K and V?

▶ 6. Why divide by $\sqrt{d_k}$?

▶ 7. What does the causal mask do and why is it $-\infty$?

▶ 8. Why multiple heads?

▶ 9. What is GQA?

▶ 10. Why is RoPE popular?

The block and architectures (Sections 8–11)

▶ 11. What do residual connections do?

▶ 12. LayerNorm vs BatchNorm?

▶ 13. What does the FFN do, and how many parameters does it have?

▶ 14. Pre-norm vs post-norm?

▶ 15. Encoder vs decoder?

▶ 16. Why are most LLMs decoder-only?

▶ 17. What is teacher forcing?

Training and inference (Sections 12–14)

▶ 18. What are the training stages of a chat model?

▶ 19. What does AdamW add over plain SGD?

▶ 20. What is the Chinchilla result?

▶ 21. Temperature vs top-p?

▶ 22. When would you use beam search?

▶ 23. What is the KV cache and why is decoding memory-bound?

▶ 24. What does FlashAttention change?

Applications (Sections 15–18)

▶ 25. How does a ViT turn an image into tokens?

▶ 26. How does CLIP do zero-shot classification?

▶ 27. Explain LoRA.

▶ 28. Why does one outlier hurt quantization?

▶ 29. RAG or fine-tuning?

▶ 30. Why does continuous batching help?

6. Mixed numerical problems

P1 — Attention (Section 5). One query $q = [1, 1]$, keys $k_1 = [1, 0]$, $k_2 = [0, 2]$, values $v_1 = [1, 0]$, $v_2 = [0, 1]$, $d_k = 2$. Find the weights and the output.

Scores: $1/\sqrt{2} = 0.707$ and $2/\sqrt{2} = 1.414$. Softmax: $e^{0.707} = 2.03$, $e^{1.414} = 4.11$ → weights **[0.33, 0.67]**. Output $= 0.33[1, 0] + 0.67[0, 1] =$ **[0.33, 0.67]**.

P2 — Parameter count (Section 8). A decoder has $d = 2048$, 24 layers, standard FFN ($d_{ff} = 4d$), a tied vocabulary of 32,000 tokens. Ignoring biases, norms and positions, how many parameters?

Blocks: $12 \times 2048^2 \times 24 = 1{,}207{,}959{,}552$. Embeddings: $32{,}000 \times 2048 = 65{,}536{,}000$. Total **1,273,495,552 ≈ 1.27B**.

P3 — KV cache (Section 14). The same model uses 16 heads of $d_{head} = 128$ (standard multi-head attention), 16-bit cache. Cache per token and for an 8,192-token context?

$2 \times 24 \times 16 \times 128 \times 2 =$ **196,608 bytes** per token; × 8,192 = 1,610,612,736 bytes = **1.5 GiB**.

P4 — Compute (Section 12). How many tokens would be "Chinchilla-optimal" for this model, and how much training compute?

$D \approx 20N = 20 \times 1.27 \times 10^9 \approx$ **25.5B tokens**. $C \approx 6ND = 6 \times 1.27 \times 10^9 \times 2.55 \times 10^{10} \approx$ **1.9 × 10²⁰ FLOPs**.

P5 — LoRA (Section 16). LoRA with $r = 8$ on W_Q and W_V (2048×2048 each) in all 24 layers. Trainable parameters?

$24 \times 2 \times 8 \times (2048 + 2048) =$ **1,572,864** ≈ 0.12% of the model.

P6 — Quantization (Section 17). Weight memory of this model in 16-bit and in INT4? Lower bound on decode time per token at batch 1 on a 2 TB/s GPU (16-bit)?

16-bit: $1.27 \times 10^9 \times 2 =$ **2.5 GB**; INT4: **0.64 GB**. Time ≥ $2.55 \times 10^9 / 2 \times 10^{12} \approx$ **1.3 ms/token** (≈ 790 tokens/s at most).

P7 — Decoding (Section 13). Logits [2, 1, 0] at temperature 0.5?

> Divide by 0.5 → [4, 2, 0]; $e^4 = 54.6$, $e^2 = 7.39$, $e^0 = 1$ → **[0.867, 0.117, 0.016]**.

P8 — Loss (Section 12). A model's average cross-entropy on a test set is 2.0 nats per token. Perplexity?

> $e^{2.0} =$ **7.39** — as unsure as choosing among about 7 tokens.

P9 — Positions (Section 7). Sinusoidal encoding for position 2 with $d_{model} = 4$?

> $[\sin 2, \cos 2, \sin 0.02, \cos 0.02] =$ **[0.909, −0.416, 0.020, 1.000]**.

P10 — ViT (Section 15). A 448 × 448 image with 14 × 14 patches: how many patches, and how does attention cost compare with 196 patches?

> $(448/14)^2 = 32^2 =$ **1,024** patches; cost ratio $(1024/196)^2 \approx$ **27×**.

7. System-design questions

7.1 "Design a question-answering assistant over our college's circulars."

A strong answer covers:

1. **Requirements:** questions in English (and maybe Telugu/Hindi), answers must cite the circular, data changes weekly, students may only see documents meant for them.
2. **Indexing:** extract text from PDFs → chunk (~300–500 tokens, ~10% overlap) → embed with a multilingual embedding model → store vectors + text +

metadata (date, department, audience) in a vector database; re-index new circulars automatically.

3. **Retrieval:** hybrid search (BM25 for exact terms like course codes + vectors for meaning), filter by the student's access rights and date, re-rank the top ~20 with a cross-encoder, keep the top ~5.
4. **Generation:** a prompt that says "answer only from the context, cite [n], say you don't know otherwise"; a mid-sized instruction-tuned LLM.
5. **Evaluation:** a test set of real student questions with known answers; track recall@5, faithfulness and answer correctness; monitor "I don't know" rates.
6. **Risks:** outdated circulars (prefer the newest by date), missing documents, prompt injection inside documents (treat retrieved text as data, not instructions).

7.2 "Serve a 7B chat model to 1,000 concurrent users."

A strong answer covers:

1. **Memory:** 13.5 GB weights in 16-bit (or ~4 GB in 4-bit); KV cache 0.5 MB/token for LLaMA-2-style MHA — GQA models need far less. 1,000 users × 2,000 tokens × 0.5 MB = 1 TB of cache in the worst case, so you need GQA, an 8-bit cache, and several GPUs or replicas.
2. **Engine:** vLLM/TGI-style continuous batching with PagedAttention and prefix caching for the shared system prompt.
3. **Latency targets:** TTFT under ~1 s, TPOT ~30–50 ms; stream tokens; autoscale replicas on queue length.
4. **Cost:** measure tokens/s per GPU and compute cost per million tokens; consider quantization and speculative decoding.
5. **Safety and reliability:** rate limits, input/output filtering, logging, fallbacks when a replica fails.

8. Revision checklist

Tick each when you can do it **without notes** (your ticks are remembered in this browser):

- [ ] Write the scaled dot-product attention formula and give every shape.
- [ ] Explain Q, K, V with an analogy and say why they're separate.
- [ ] Explain $\sqrt{d_k}$ with the variance argument.
- [ ] Draw a pre-norm decoder block and label residuals, norms, attention and FFN.
- [ ] Compute a 2-token attention example by hand.
- [ ] Count the parameters of a model from d, layers and vocabulary.
- [ ] Compute a KV-cache size for a given model and context.
- [ ] Explain MLM vs causal LM vs span corruption.
- [ ] Explain pre-training → SFT → RLHF/DPO.
- [ ] Apply temperature, top-k and top-p to a small distribution.
- [ ] Explain the KV cache and why decoding is memory-bound.
- [ ] Explain LoRA's BA, its initialisation and merging.
- [ ] Quantize a few numbers to INT4 and explain outliers.
- [ ] Draw the RAG pipeline and name two evaluation metrics.

0 of 14 ticked.

9. Common interview mistakes

Reciting buzzwords without the "why". Always say what problem a technique solves.

Mixing up training and inference. Training is parallel (teacher forcing); generation is sequential (one pass per token, with a KV cache).

Forgetting shapes. Interviewers often ask "what's the shape of $QK^\top$?" — answer $n \times n$, and say why that's the $O(n^2)$ cost.

Over-claiming. Say "reduces" hallucination (RAG), "usually small" accuracy loss (4-bit), "roughly" 20 tokens per parameter.

Ignoring cost and latency in design questions. Mention memory, throughput and cost per token.

10. Summary

- The whole course rests on a few formulas: embeddings, $\mathrm{softmax}(QK^\top/\sqrt{d_k}+M)V$, multi-head concatenation, residual + norm + FFN blocks, and next-token cross-entropy.
- A handful of numbers ($12d^2$ per block, 16 bytes/param for training, $6ND$, 0.5 MB/token KV cache for a 7B MHA model, ~20 tokens/param) let you sanity-check almost any claim.
- Good answers follow *what → why → a concrete detail*.
- Design answers should cover requirements, data/memory, the pipeline, evaluation, cost and risks.

11. Final self-test

1. In one sentence each, explain: the KV cache, LoRA, and RAG.
2. A model has $d = 4096$, 32 layers, tied vocabulary 32,000, standard FFN. Estimate its parameters (ignore biases and norms).
3. Why can't you train a bidirectional encoder with next-token prediction?

▶ Show answers

12. Further reading

The papers below are the "spine" of the course; each earlier section lists more.

1. Vaswani, A. et al. (2017). **"Attention Is All You Need."** arXiv:1706.03762.
2. Devlin, J. et al. (2019). **"BERT."** arXiv:1810.04805.
3. Brown, T. et al. (2020). **"Language Models are Few-Shot Learners."** arXiv:2005.14165.
4. Hoffmann, J. et al. (2022). **"Training Compute-Optimal Large Language Models."** arXiv:2203.15556.
5. Touvron, H. et al. (2023). **"Llama 2: Open Foundation and Fine-Tuned Chat Models."** arXiv:2307.09288.
6. Jurafsky, D., & Martin, J. H. ***Speech and Language Processing*** (3rd edition draft) — the Transformer and LLM chapters, for a textbook-length treatment.

Prepared by Akhil